Adaptively-Secure Proxy Re-Encryption with Tight Security

Chen Qian [1,2,3], Shuo Chen[1,2,3], and Shuai Han [4]

[1] School of Cyber Science and Technology, Shandong University, Qingdao, Shandong, 266237, China

[2] State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Jinan, 250100, China

[3] Key Laboratory of Cryptologic Technology and Information Security of Ministry of Education, Shandong University, Jinan, Shandong, 250100, China

[4] School of Computer Science, Shanghai Jiao Tong University, Shanghai, China

[5] State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an, China

Abstract. (Bi-Directional) Proxy Re-Encryption (PRE) is a public-key encryption scheme that allows a proxy, holding a re-encryption key from i to j, to transform a ciphertext intended for i into one intended for j. PRE has numerous applications, including secure data sharing and cloud computing. However, most existing PRE schemes experience significant security degradation when adversaries are allowed to adaptively corrupt re-encryption or secret keys. Prior to this work, only a few PRE schemes achieved quasi-polynomial security loss in the adaptive setting, and even those were limited to restricted re-encryption strategies.

In this paper, we propose four distinct PRE schemes with tight security guarantees in the adaptive setting, based on the MDDH assumption:

- PREA, PREB: Single- and multi-challenge aHRA-secure PRE schemes with tight security focusing on efficient constructions.
- PREC, PRED: Single- and multi-challenge aCCA-secure PRE schemes with (almost) tight security focusing on CCA-type security.

To achieve tightly CCA-secure PRE schemes, we introduce a novel concept called tag-based language-malleable NIZK with special simulation soundness. This primitive provides simulation-sound NIZK while preserving a restricted form of malleability. We construct both one-time and unbounded versions of this primitive under the MDDH(Matrix Decisional Diffie-Hellman) assumption.

Keywords. Proxy Re-Encryption, Adaptive Security, NIZK, Simulation-Soundness, Malleability

1 Introduction

Proxy re-encryption (PRE) was first introduced by Blaze, Bleumer, and Strauss in 1998 [BBS98] as a mechanism for securely transforming ciphertexts between different users. Since then, PRE has been extensively studied in the context of secure data sharing, cloud computing, and access control scenarios.

Uni-Directional VS Bi-Directional. There are two different notions of PRE schemes, called uni-directional PRE [LV08] and bi-directional PRE [BBS98]. The uni-directional PRE means that the re-encryption key d can only be used to transform ciphertexts from Alice to Bob, but not vice versa. However, in many applications, bi-directional PRE is sufficient, where the re-encryption key d can be used to transform ciphertexts in both directions. In fact, as we will explain, the bi-directional proxy re-encryption already implies updatable encryption schemes and public key encryption schemes with adaptive key corruption, which has many applications in real life scenarios. Moreover, the relaxation from uni-directional to bi-directional leads to much more efficient PRE schemes as [CH07]. Therefore, we only focus on bi-directional PRE schemes in this paper, and refer to them simply as PRE schemes.

Proxy Re-Encryption, Updatable Encryption and Encryption with Corruption. There are numerous notions related to encryption schemes, including proxy re-encryption [BBS98], updatable encryption [BLMR13], and public-key encryption with multi-user and adaptive key corruption setting [BHJ+15]. In fact, these notions are closely related. An updatable encryption scheme can be seen as a special case of proxy re-encryption scheme by treating the update key as the re-encryption key from epoch i to $i+1$. Furthermore, if we disallow the adversary to query re-encryption or update token, then both proxy re-encryption and updatable encryption schemes can be seen as public-key encryption schemes in a multi-user setting with adaptive key corruption, where the adversary can corrupt users' secret keys adaptively.

Security Model. The security of proxy re-encryption (PRE) schemes is less well understood compared to that of public-key encryption schemes. Since its introduction in [BBS98], various security models have been proposed, including the chosen plaintext attack (CPA) model [BBS98], the honest re-encryption attack (HRA) model [Coh19], and the chosen ciphertext attack (CCA) model [CH07]. Among these, the HRA model, a relaxed variant of the CPA model, has gained prominence as the most widely accepted security framework for PRE schemes. In this model, the adversary is allowed to request the re-encryption of any ciphertext (except the challenge ciphertext) generated by the encryption oracle. While CCA security is a stronger notion, HRA-secure PRE schemes have garnered significant attention in the literature [AFGH05,ABH09,HRsV07], particularly in lattice-based constructions [Gen09,ABPW13,FL19].

Adaptive Security. However, all the above security models (CPA, HRA, CCA) are not fully adaptive, meaning that the adversary is required to pre-select a subset of users that will be corrupted. This requirement greatly facilitates the construction of PRE scheme. Otherwise, if there are $\mathsf{N_U}$ users, the challenger needs to guess which user will be corrupted by the adversary, this adds an extra exponential security loss $\mathcal{O}(2^{\mathsf{N_U}})$. There are several following works focusing on reducing such unrealistic security loss, including Canetti and Hohenberger [CH07] add adaptive corruption of re-encryption token between uncorrupted parties[6]; Jafargholi et al. [JKK+17] proposed a generic transform using the *graph pebbling game techniques* to achieve adaptive security for some special cases, which is followed by [FKKP19] to give an adaptively-secure uni-directional PRE scheme with polynomial security loss with restriction on proxy re-encrypt structure.

We ask the following question:

Can we construct efficient adaptively {HRA, CCA}-secure bi-directional PRE schemes with tight security?

1.1 Our Contribution

In this paper, we affirmatively answer the above question by proposing four adaptively secure proxy re-encryption (PRE) schemes. Prior to this work, all known adaptively secure PRE schemes lacked tight security. We compare our solution with existing schemes in Table 1.

Our contributions are both practical and theoretical.

- **Efficient Constructions.** We propose four novel PRE schemes.
  - We introduce PRE_1 and PRE_2, the most efficient single- and multi-challenge adaptively HRA-secure (aHRA) schemes to date.
  - We present PRE_3 and PRE_4, the first single- and multi-challenge adaptively CCA-secure (aCCA) schemes.

  All four schemes feature tight security reductions, with PRE_4 achieving an almost-tight reduction. In the main body, we focus on the constructions and security proofs of PRE_1 and PRE_2. Due to space constraints and the similarity between HRA and CCA constructions, the detailed constructions and security proofs of PRE_3 and PRE_4 are deferred to the appendix.
- **New Techniques.** We develop new proof techniques and cryptographic tools that are of independent interest.
  - To achieve adaptive security without a non-tight reduction, we introduce a novel proof strategy that confines the guessing argument to a transition between two perfectly indistinguishable hybrid games. We formalize this technique as a selective-to-adaptive (S2A) core lemma (Lemma 3), which can be a general tool for proving tight adaptive security for other cryptographic primitives.

[6] As they have mentioned in their paper, they still need the adversary to selectively choose the subset of corrupted users before the protocol starts

- To reconcile the inherent malleability of PRE with CCA security, we introduce a new type of non-interactive zero-knowledge (NIZK) proof system. Termed language-malleable special simulation-sound NIZK, this system permits a specific, controlled form of malleability required for re-encryption while maintaining simulation soundness against all other forms of tampering. We present the high-level ideas and security definitions in the main body, with detailed constructions and proofs provided in the appendix.

Scheme	Hop	Dir.	Security	Loss	Ct Size	Assumption	PRE?
[BBS98]	multi	Bi	s-MC-CPA	loose	$2\mathbb{G}$	DDH	✓
[CH07, Π_{RO}, Π]	multi	Bi	s-SC-CCA	loose	$\ell + \sigma + 4\mathbb{G}$	DBDH	✓
[Coh19, 5.4]	single	Uni	s-MC-HRA	loose	$2\mathbb{G}$	DDH	✓
[FKKP19]	–	–	a-CPA/HRA	loose	–	–	✓
[HLG23]	–	–	a-MC-CCA	tight	$18\mathbb{G}$	MDDH	×
[LCB+25]	single	Uni	a-MC-HRA	tight	$3\mathbb{G}'$	Composite	✓
PRE_1	multi	Bi	a-SC-HRA	tight	$3\mathbb{G}$	MDDH	✓
PRE_2	multi	Bi	a-MC-HRA	tight	$3\mathbb{G}$	MDDH	✓
PRE_3	multi	Bi	a-SC-CCA	tight	$\ell + \sigma + 24\mathbb{G}$	MDDH	✓
PRE_4	multi	Bi	a-MC-CCA	tight[7]	$\ell + \sigma + 51\mathbb{G}$	MDDH	✓

Table 1. Comparison of our schemes with existing ones. Security format: [corruption]-[challenge]-[security notion], where corruption is s (selective) or a (adaptive), challenge is SC (single-challenge) or MC (multi-challenge), and security notion is CPA, HRA, or CCA. $\mathbb{G}$ represents group elements from $\mathbb{G}_1$ and $\mathbb{G}_2$, $\mathbb{G}'$ from composite-order groups, σ is the signature size, ℓ is the verification key length, and '–' means not specified.

1.2 Related Work

Proxy re-encryption was first introduced in [BBS98] and has since been studied under various security models. Early schemes [AFGH05,CH07] achieved CPA or CCA security but with loose reductions and selective corruption models. The HRA model [Coh19] provides a practical security notion between CPA and CCA, which has been widely adopted in lattice-based constructions [Gen09,ABPW13,FL19].

Achieving tight adaptive security is challenging due to the impossibility result in [BJLS16], which shows that tightly secure adaptive PKE schemes cannot have secret keys that are both unique for each public key and efficiently re-randomizable. Previous adaptive PRE schemes [CH07,FKKP19] either require selective user corruption or suffer from polynomial security loss. Hash proof systems [CS02] provide a way to bypass this barrier by allowing exponentially many secret keys per public key that are not efficiently re-randomizable.

[7] The security reduction of PRE_4 is almost tight with a security loss of $\mathcal{O}(\log Q)$ due to the OR proof technique, where Q is the number of challenge queries.

Recent works [LLP20,HLG23] achieved tight multi-challenge security for standard PKE using random oracles or complex proof systems. For PRE schemes, the work in [LCB+25] is the first to achieve tight adaptive multi-challenge HRA security for unidirectional single-hop schemes in composite-order groups under the subgroup decision assumption SD3 (similar to the SD1 and SD2 assumptions from [LW10]). Our work achieves both HRA and CCA security for bidirectional multi-hop PRE in prime-order groups under the more standard MDDH assumption, with the first tight adaptive multi-challenge CCA-secure schemes and improved HRA efficiency.

Existing tag-based simulation-sound NIZKs [AJO+19] are incompatible with PRE because their strong simulation soundness prevents re-encryption of challenge ciphertexts. We introduce language-malleable NIZKs that allow controlled malleability for re-encryption while maintaining security against other attacks.

1.3 Technique Overview

In this section, we provide an overview of the techniques employed in our schemes, emphasizing the advancements made over previous approaches. We give an abstract way to understand the underlying principles in our actual constructions. To avoid several structure specifications, we give a high-level description of our techniques using the concrete MDDH as an example.

Starting Point: From Adaptively Secure PKE to PRE. We begin by observing that, in the absence of re-encryption and token generation queries, an adaptively secure PRE scheme is equivalent to a multi-user public-key encryption (PKE) scheme secure against adaptive key corruptions. However, constructing such a PKE scheme with a tight security reduction must circumvent the impossibility result of Bader et al. [BJLS16]. Their result precludes tightly secure adaptively-secure PKE schemes where secret keys are unique for each public key or are efficiently re-randomizable.

To overcome this barrier, we employ hash proof systems (HPS) [CS02]. In an HPS-based construction, each public key corresponds to an exponential number of secret keys, which are not efficiently re-randomizable. The ciphertext structure,

$$\left[\mathbf{c}_0\right] = \left[\mathbf{A}\mathsf{w}\right], \qquad \left[\mathbf{c}_1\right] = \left[\mathbf{x}^\intercal \mathsf{w} + \mathsf{m}\right],$$

where the key pair is $(\left[\mathbf{x}^\intercal\right] = \left[\mathbf{k}^\intercal \mathbf{A}\right], \mathbf{k})$, naturally exhibits the key homomorphism required for PRE. This structure allows us to bypass the impossibility result of [BJLS16] and serves as the foundation for our first scheme, PRE_1, which is single-challenge and tightly aHRA-secure.

A Tight Selective-to-Adaptive Security Reduction. A key challenge in proving tight adaptive security is to simulate oracle queries, in particular for key corruptions and re-encryption tokens, without resorting to a guessing argument that incurs a security loss. Our core technical insight is a novel proof strategy

that confines the guessing argument to a single transition between two hybrid games that are otherwise perfectly indistinguishable.

Specifically, we leverage the structure of hash proof systems. For any honestly generated language public key $[\mathbf{A}]$, the space of corresponding secret keys is defined by the **left kernel** $\ker(\mathbf{A})$, which is information-theoretical independent of the public key. Moreover, shifting all user secret keys by a random element from $\ker(\mathbf{A})$ is perfectly undetectable by the adversary. We apply this key-shifting technique in a single hybrid step to re-randomize the challenge ciphertext while leaving all other values unchanged. This allows us to bound the advantage of an adaptive adversary by that of a selective one within this specific transition, thereby avoiding any security loss. We formalize this technique as a selective-to-adaptive (S2A) core lemma (Lemma 3), which we believe is a general tool for proving tight adaptive security for other cryptographic primitives.

From Single-Challenge to Multi-Challenge To achieve multi-challenge security, we present an algebraic interpretation of the multi-key extraction techniques from [HLG23]. Intuitively, when the left kernel of $\mathbf{A}$ has dimension at least k, the MDDH assumption implies that shifting all secret keys by a uniformly random element of $\ker(\mathbf{A})$ is computationally indistinguishable (with a tight reduction) from shifting them by a uniformly random vector in the entire space.[8] This k-dimensional randomness from the kernel space precisely matches the dimension of the underlying language $\mathcal{L}_{\mathsf{lpk}}$[9], which also has dimension k.

The key insight is to leverage the *language linearity* property of our subset membership language. Starting from a language $\mathcal{L}_{\mathsf{lpk}}$ over dimension ℓ, we can construct an isomorphic language $\mathcal{L}_{\mathsf{lpk}'}$ over dimension $\ell + n$ (where $n = 1$ for aHRA schemes and $n = 2$ for aCCA schemes) through a linear transformation matrix $\mathbf{M} = (\mathbf{I}_\ell, \mathbf{r}_1, \ldots, \mathbf{r}_n)^\intercal \in \mathbb{F}^{(\ell+n)\times\ell}$, where each $\mathbf{r}_i$ is sampled uniformly from the orthogonal complement of $\mathcal{L}_{\mathsf{lpk}}$. This transformation preserves the language structure: $\mathbf{M}$ maps $\mathcal{L}_{\mathsf{lpk}}$ isomorphically to $\mathcal{L}_{\mathsf{lpk}'}$, and both languages maintain the same dimension k. Crucially, the hard subset membership (HSM) assumption for the expanded language $\mathcal{L}_{\mathsf{lpk}'}$ provides additional randomness that can be used to simultaneously randomize multiple challenge ciphertexts. By exploiting this expanded randomness space while maintaining tight security through the language isomorphism, we successfully extend the single-challenge aHRA-secure scheme PRE_1 to the multi-challenge aHRA-secure scheme PRE_2 with preserved tight security.

aCCA Security For proxy re-encryption schemes, transitioning from aHRA to aCCA is more challenging compared to standard encryption schemes. To address this, we revisit the construction of CCA-secure encryption schemes based on hash proof systems and develop a novel proof system tailored specifically for this purpose.

[8] For a formal argument under the $\mathsf{Q_C}$-MDDH assumption, one must also incorporate the ct_0 component.

[9] The formal definition of the subset membership language is given in Section 4.1

Simulation-Sound Proofs. In our aHRA-secure encryption schemes, the security relies on the perfect hiding of information in the left kernel $\ker(\mathbf{A})$ from the adversary. For the multi-challenge setting, the language linearity property provides additional randomness from the expanded language dimension, which together with the kernel space information enables simultaneous protection of multiple challenge ciphertexts. However, this condition holds true only in the absence of a decryption oracle. If a decryption oracle is available, the adversary could query the decryption of $\left[\mathsf{ct}_0\right]_1 \notin \mathsf{Span}(\mathbf{A})$, thereby leaking information about the $\ker(\mathbf{A})$ element used in the secret key $\mathbf{k}$. To address this issue, we incorporate zero-knowledge proofs for linear span languages.

One common challenge in using the zero-knowledge proof to achieve CCA security is that, in the security proof of hash proof systems, the challenger needs to intentionally put $\mathsf{ct_0}$ of the challenge ciphertext outside the $\mathsf{Span}(\mathbf{A})$, then use the hidden randomness in $\ker(\mathbf{A})$ to hide the plaintext message. With a zero-knowledge proof for linear span, the adversary then needs to simulate the proof for false statement. Therefore, we need the simulation soundness property of the underlying zero-knowledge proof to guarantee that the adversary cannot prove false statements even after seeing the false proof in the challenge ciphertext.

A similar issue arises in existing multi-challenge tightly secure encryption schemes [LLP20,HLG23]. These schemes address the problem either by employing the random oracle model or by utilizing existing tag-based simulation-sound quasi-adaptive zero-knowledge proofs [AJO+19]. However, in the context of constructing PRE schemes, the key homomorphism property must be preserved, making the random oracle approach unsuitable. Additionally, the existing tag-based simulation-sound quasi-adaptive zero-knowledge proofs [AJO+19] lack the key homomorphism property. This limitation is inherent, as these proofs are designed to achieve strong simulation soundness, which prevents adversaries from generating any new false proofs. In contrast, for PRE schemes, the re-encryption algorithm must allow the generation of certain new false proofs using the re-encryption token, to re-encrypt the challenge ciphertext using the re-encryption token. To address this, we relax the strong simulation-soundness requirement by introducing a notion of special simulation soundness. We then construct new tag-based language-malleable NIZK proofs for linear span languages that satisfy this special simulation-soundness property. Specifically, we propose two such proofs, $\mathbf{\Pi}_{\mathsf{OT}}$ (one-time) and $\mathbf{\Pi}_{\mathsf{U}}$ (unbounded), which are subsequently employed in the CCA-secure PRE schemes PRE_3 and PRE_4. As $\mathbf{\Pi}_{\mathsf{OT}}$ and $\mathbf{\Pi}_{\mathsf{U}}$ are the first NIZK proof systems to combine language malleability with special simulation soundness, we believe they are of independent interest. The intuition behind the construction $\mathbf{\Pi}_{\mathsf{OT}}$ and $\mathbf{\Pi}_{\mathsf{U}}$, is that instead of directly proving the statement $\mathsf{x} \in \mathcal{L}$, we prove the following statement

$$\{\mathsf{x}, \mathbf{C_s} \mid \mathsf{x} \in \mathcal{L} \vee (\mathbf{C_s} = \mathsf{Commit}(\mathsf{ck}, \sigma) \wedge \mathsf{Ver}(\mathsf{svk}, \mathsf{x}, \mathsf{x}') = 1 \wedge \mathsf{x}' = \mathbf{T}(\mathsf{x}))\},$$

where $\mathbf{T}$ is an allowed transformation of the statement. In the context of PRE schemes, $\mathbf{T}$ corresponds to the re-encryption operation. The commitment scheme ensures that the adversary cannot adaptively choose the transformation after

seeing the proof of the challenge ciphertext. The signature scheme ensures that the adversary cannot generate new valid transformations without knowing the signing key. This design allows us to achieve language malleability while maintaining special simulation soundness.

aCCA-secure PRE Schemes. We present two distinct constructions of PRE schemes: PRE_3 with single-challenge security and PRE_4 with multi-challenge security. For clarity, we focus on explaining the construction of PRE_3 (single-challenge CCA) from PRE_1 (single-challenge aHRA), as the construction of PRE_4 (multi-challenge CCA) from PRE_2 (multi-challenge aHRA) follows a similar approach.

The ciphertext structure of PRE_1 can be represented as:

$$\left[\mathbf{c}_0\right] = \left[\mathbf{A}\mathsf{w}\right], \qquad \left[\mathbf{c}_1\right] = \left[\mathbf{x}^\intercal \mathsf{w} + \mathsf{m}\right],$$

where $\left[\mathbf{A}\right]$ and $\left[\mathbf{x}\right]$ form the public key, and $\mathbf{k}$ is the secret key satisfying $\mathbf{x}^\intercal = \mathbf{k}^\intercal \mathbf{A}$. To achieve aCCA security, we must ensure that the plaintext message m cannot be modified. However, proxy re-encryption requires changing $\left[\mathbf{x}\right]$, which affects $\left[\mathbf{c}_1\right]$. This creates a conflict between aCCA security and proxy re-encryption functionality. To resolve this conflict, we modify the ciphertext structure as follows:

$$\left[\mathbf{c}_0\right] = \left[\mathbf{A}\mathsf{w}\right], \qquad \left[\mathbf{c}_1\right] = \left[(\mathbf{k}_i^\intercal \mathbf{A} + \mathbf{x}^\intercal)\mathsf{w}\right], \qquad \left[\mathbf{c}_2\right] = \left[\mathbf{x}^\intercal \mathsf{w} + \mathsf{m}\right],$$

where $\mathbf{k}_i$ is the secret key of user i with public key $\left[\mathbf{k}_i^\intercal \mathbf{A}\right]$, and $\left[\mathbf{A}\right], \left[\mathbf{x}\right]$ are common public parameters.

In this refined structure, $\left[\mathbf{A}\right]$ and $\left[\mathbf{x}\right]$ remain invariant across all users, while only $\mathbf{k}_i$ varies between users. This design enables proxy re-encryption to modify solely $\left[\mathbf{c}_1\right]$. To guarantee non-malleability, we incorporate a signature scheme for $\left[\mathbf{c}_0\right]$ and $\left[\mathbf{c}_2\right]$, along with a NIZK proof to verify that $\left[\mathbf{c}_0\right] \in \mathsf{Span}(\mathbf{A})$ and $\left[\mathbf{c}_1\right]$ is correctly formed with respect to the same witness w. Given that $\left[\mathbf{c}_0\right]$ uniquely determines w, this NIZK proof ensures the non-malleability of $\left[\mathbf{c}_1\right]$ as well.

2 Notations

Group Presentation. We denote group elements as $\left[x\right]_1 \in \mathbb{G}_1$, $\left[x\right]_2 \in \mathbb{G}_2$ and $\left[x\right]_\mathsf{T} \in \mathbb{G}_\mathsf{T}$. Matrices are represented by uppercase letters (e.g., $\mathbf{A}$), vectors by bold lowercase letters (e.g., $\mathbf{w}$), and scalars by regular lowercase letters (e.g., x). The pairing operation is denoted as $\left[\mathbf{A}\right]_1 \bullet \left[\mathbf{B}\right]_2 = \left[\mathbf{AB}\right]_\mathsf{T}$.
Pseudo-codes. We use the notation $x \leftarrow\!\!\$\, \mathsf{U}(\mathbb{Z}_\mathsf{p})$ to indicate that x is sampled uniformly from the set $\mathbb{Z}_\mathsf{p}$, and $\ker(\mathbf{A})$ denotes the left kernel of a matrix $\mathbf{A}$. The symbol **check** denotes an operation that checks the validity of a statement: if the statement is true, execution continues; otherwise, it aborts and returns 0.
Game-based Proofs. The notation pr_i denotes the probability that the adversary wins in $\mathbf{Game}_i$. We denote by 1^λ the unary presentation of the security parameter λ.

3 Proxy Re-Encryption

We follow the classical definition of (bi-directional multi-hop) proxy re-encryption from [CH07]. To better illustrate our security notions, we then define corruption graph which updates dynamically as the adversary makes queries. We also define the security notions of aHRA and aCCA for our proxy re-encryption scheme.

Definition 1 (Proxy Re-Encryption). *A (bi-directional multi-hop) proxy re-encryption scheme* PRE *consists of six* PPT *algorithms* $\mathsf{PRE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{RKGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{REnc})$ *with the following syntax:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$: *Takes the security parameter* λ *as input and returns the public parameter* pp.
- $\mathsf{KGen}(\mathsf{pp}) \to (\mathsf{pk}, \mathsf{sk})$: *Takes the public parameter* pp *as input and returns a public key* pk *and a secret key* sk.
- $\mathsf{RKGen}(\mathsf{sk}_i, \mathsf{sk}_j) \to \mathsf{d}_{i,j}$: *Takes the secret keys of users* i *and* j *as input and returns a re-encryption token* $\mathsf{d}_{i,j}$.
- $\mathsf{Enc}(\mathsf{pk}, \mathsf{m}) \to \mathsf{ct}$: *Takes a public key* pk *and a message* m *as input and returns a ciphertext* ct.
- $\mathsf{Dec}(\mathsf{sk}, \mathsf{ct}) \to \mathsf{m}$: *Takes a secret key* sk *and a ciphertext* ct *as input and returns the decrypted message* m.
- $\mathsf{REnc}(\mathsf{d}_{i,j}, \mathsf{pk}_i, \mathsf{pk}_j, \mathsf{ct}) \to \mathsf{ct}'$: *Takes a re-encryption token* $\mathsf{d}_{i,j}$*, public keys* pk_i*,* pk_j*, and a ciphertext* ct *as input, and returns a new ciphertext* ct'.

Additionally, we require the following correctness property:

Correctness: *We require the multi-hop correctness of the* PRE *scheme. Namely, it requires that after any* L *rounds of re-encryption, the ciphertext can be decrypted to the original plaintext.*
More formally, for any security parameter λ*, any message* m *and any integer* L*, we have*

$$\Pr\left[\mathsf{Dec}(\mathsf{sk}_L, \mathsf{ct}_L) = \mathsf{m} \;\middle|\; \begin{array}{l} \mathsf{pp} \xleftarrow{\$} \mathsf{Setup}(1^\lambda) \\ (\mathsf{pk}_1, \mathsf{sk}_1) \xleftarrow{\$} \mathsf{KGen}(\mathsf{pp}) \\ \mathsf{ct}_1 \xleftarrow{\$} \mathsf{Enc}(\mathsf{pk}_1, \mathsf{m}) \\ \textbf{for } i = 2 \textbf{ to } L \\ \quad (\mathsf{pk}_i, \mathsf{sk}_i) \xleftarrow{\$} \mathsf{KGen}(\mathsf{pp}) \\ \quad \mathsf{d}_{i-1,i} \xleftarrow{\$} \mathsf{RKGen}(\mathsf{sk}_{i-1}, \mathsf{sk}_i) \\ \quad \mathsf{ct}_i \xleftarrow{\$} \mathsf{REnc}(\mathsf{d}_{i-1,i}, \mathsf{pk}_{i-1}, \mathsf{pk}_i, \mathsf{ct}_{i-1}) \end{array}\right] = 1.$$

3.1 Corruption Model

Following Canetti and Hohenberger [CH07], we exclude trivial attacks by maintaining a corruption graph $\mathsf{G} = (\mathsf{V}, \mathsf{E})$ built by the procedure makeG. The vertex set V is partitioned into honest users $\mathsf{V_H}$ and corrupted users $\mathsf{V_C}$. An undirected edge $e_{i,j}$ is added if (i) the adversary can derive one user's secret key from the other via a corrupted re-encryption token, or (ii) a challenge ciphertext has been re-encrypted between users i and j. The graph is updated after each oracle query. We formalize this below.

Definition 2 (Corruption Graph). *A corruption graph is a tuple* $\mathsf{G} = (\mathsf{V}, \mathsf{E})$ *with the following syntax:*

- $\mathsf{V} = \mathsf{V_H} \cup \mathsf{V_C}$ *is the set of vertices, partitioned into honest vertices* $\mathsf{V_H}$ *and corrupted vertices* $\mathsf{V_C}$*.*
- $\mathsf{E} \subseteq \mathsf{V} \times \mathsf{V}$ *is the set of edges, representing the relationships between users.*
- $\mathsf{V_{CL}} \subseteq \mathsf{V_H}$ *is the set of challenge vertices, which includes all vertices that have challenge ciphertexts and all vertices that are connected to challenge vertices.*

The corruption graph is constructed dynamically based on the adversary's queries, where edges are added to represent that

- *the adversary can derive secret keys of connected users using corrupted tokens.*
- *or the adversary re-encrypted challenge ciphertexts to other users.*

The graph is undirected, meaning if there is an edge between two vertices, they can derive each other's secret keys. Moreover, $\mathsf{V_C}$ *is the set of corrupted vertices, which includes all vertices that have been corrupted secret keys and all vertices that are connected to corrupted vertices.*

The initial corruption graph consists only of the honest vertices $\mathsf{V} = \mathsf{V_H}$ and $\mathsf{E} = \emptyset$, and it evolves as the adversary makes queries to the oracles. Each query may reveal new relationships between users, leading to the addition of edges in the graph and the partition of vertices V into $\mathsf{V_H}$ and $\mathsf{V_C}$. We update the corruption graph with the following rules:

- If $\mathcal{A}$ queries $\mathcal{O}_{\mathsf{CorrK}}(i)$ with $i \in \mathsf{V_H}$, then move i from $\mathsf{V_H}$ to $\mathsf{V_C}$ and add all vertices j connected to i to $\mathsf{V_C}$.[10]
- If $\mathcal{A}$ queries the token corruption query $\mathcal{O}_{\mathsf{CorrT}}(i, j)$,
  1. add edge $e_{i,j}$ to E,
  2. if $i \in \mathsf{V_C}$ or $j \in \mathsf{V_C}$, then move both i and j into $\mathsf{V_C}$.
- If $\mathcal{A}$ queries the re-encryption oracle $\mathcal{O}_{\mathsf{ReEnc}}(i, j, \mathsf{ct})$ with a challenge ciphertext $(\mathsf{ct}, i) \in \mathcal{L}_{\mathsf{C}}$, add edge $e_{i,j}$. [11]

We use the algorithm makeG that takes all internal states and produces the corruption graph $\mathsf{G} = (\mathsf{V}, \mathsf{E})$ as output.

3.2 Security Model

Using the corruption graph, we define the security notions of aHRA and aCCA for our proxy re-encryption scheme. The security definitions are based on the winning probability of the adversary while adhering to the constraints imposed by the corruption graph.

[10] The syntax $\mathcal{O}_*(*)$ denotes the oracle query made by the adversary which we define in Figure 2

[11] We emphasize that we only add edge $e_{i,j}$ for re-encryption of challenge ciphertexts, not for arbitrary ciphertexts. This makes a difference between CPA-type security and HRA-type security.

Trivial Attacks We need to first exclude the trivial attacks with respect to aHRA and aCCA security. This includes attacks where the adversary can easily win the game without any significant effort. We formalize the trivial attacks as a predicate $\mathsf{NonTrivial}(\mathcal{L}_{\mathsf{CorrT}}, \mathcal{L}_{\mathsf{CorrK}}, \mathcal{L}_{\mathsf{C}})$ that checks whether the adversary's queries lead to a non-trivial attack, where $\mathcal{L}_{\mathsf{CorrT}}$ records all token corruption queries, $\mathcal{L}_{\mathsf{CorrK}}$ records all key corruption queries, and $\mathcal{L}_{\mathsf{C}}$ records all challenge ciphertexts and its re-encryptions. The predicate is defined in Figure 1.

Alg $\mathsf{NonTrivial}(\mathcal{L}_{\mathsf{CorrT}}, \mathcal{L}_{\mathsf{CorrK}}, \mathcal{L}_{\mathsf{C}})$:

01 if $\exists (ct_{i^\star}, i^\star) \in \mathcal{L}_{\mathsf{C}} : \mathcal{O}_{\mathsf{Dec}}(i^\star, ct_{i^\star})$ is queried // CCA
02 **return** 0 // CCA
03 $(\mathsf{V}, \mathsf{E}) := \mathsf{makeG}(\mathcal{L}_{\mathsf{CorrK}}, \mathcal{L}_{\mathsf{CorrT}})$
04 **parse** $(\mathsf{V}_{\mathsf{H}}, \mathsf{V}_{\mathsf{C}}) =: \mathsf{V}$
05 **if** $\exists (ct_{i^\star}, i^\star) \in \mathcal{L}_{\mathsf{C}} : i^\star \in \mathsf{V}_{\mathsf{C}}$
06 **return** 0
07 **return** 1

Fig. 1. Description of NonTrivial algorithm, which checks whether there is a trivial attack for the adversary against aHRA/aCCA security.

(Un)linkability and Link Function Unlinkability informally requires that an adversary cannot distinguish whether a ciphertext under a target key was obtained by re-encrypting some ciphertext of the same message under another key, or by an independent fresh encryption of that message. We formalize this notion below.

Definition 3 (Unlinkability). *A proxy re-encryption scheme is said to be unlinkable if for any ciphertext* ct_i*, the distribution of a re-encryption ciphertext* ct_j *of* ct_i *with a fresh encryption of the same message is (computational, statistical or perfect) indistinguishable. Formally, we have*

$$\{\mathsf{ct}_j \xleftarrow{\$} \mathsf{REnc}(\mathsf{d}_{i,j}, \mathsf{pk}_i, \mathsf{pk}_j, \mathsf{Enc}(\mathsf{pk}_i, \mathsf{m}))\} \approx_{\mathsf{X}} \{\mathsf{ct}_j \xleftarrow{\$} \mathsf{Enc}(\mathsf{pk}_j, \mathsf{m})\},$$

with $\mathsf{X} \in \{$*computational, statistical, perfect*$\}$.

In the aCCA experiment the adversary additionally obtains a decryption oracle, so we must prevent trivial wins in which a challenge ciphertext (or any of its re-encryptions) is submitted for decryption. To enforce this we use a publicly computable predicate $\mathsf{link}(\mathsf{ct}_i, \mathsf{ct}_j)$ that outputs 1 if ct_j is a (possibly multi-hop) re-encryption of ct_i, and 0 otherwise. This predicate is only required for the aCCA game, since in the aHRA game the adversary has no decryption oracle, so identifying such links yields no immediate advantage.

The availability of a public predicate $\mathsf{link}(\cdot, \cdot)$ rules out perfect unlinkability, since an adversary can always test whether two ciphertexts are (multi-hop) related

by evaluating $\mathsf{link}(\mathsf{ct}_i, \mathsf{ct}_j)$. (Computational or statistical unlinkability may still be meaningful.) In particular, an aCCA-secure scheme with a public linking predicate cannot achieve perfect unlinkability. Otherwise, even an unbounded challenger could not distinguish a re-encryption of a challenge ciphertext from a fresh encryption, yet an adversary could re-encrypt a challenge ciphertext to another honest user and submit the result to the decryption oracle to break aCCA security. This attack is admissible because the adversary obtains re-encryption tokens between honest users, and the perfect unlinkability implies that the challenger has no possible way to distinguish the re-encryption from a fresh encryption.

aHRA and aCCA Security We define the adaptive security of our proxy re-encryption scheme in the multi-challenge setting, where the adversary can issue a polynomial number of challenge queries. The adversary is allowed to corrupt users and re-encrypt challenge ciphertexts, but it must adhere to the constraints imposed by the corruption graph.

Definition 4 (Security). *We define the security experiment* $\mathsf{Exp}^{\mathsf{X},\mathsf{Q_C}}_{\mathsf{PRE},\mathcal{A}}(\lambda)$ *with* $\mathsf{X} \in \{\mathsf{aHRA}, \mathsf{aCCA}\}$ *in Figure 2. A* PRE *scheme is* X*-secure if for every security parameter* λ*, every* PPT *adversary* $\mathcal{A}$*, and any polynomial number of challenge queries* $\mathsf{Q_C} = \mathsf{poly}(\lambda)$*, the adversary's advantage defined as follows is negligible. Formally, we have*

$$\mathsf{Adv}^{\mathsf{X},\mathsf{Q_C}}_{\mathsf{PRE},\mathcal{A}}(\lambda) = \left| \Pr\left[\mathsf{Exp}^{\mathsf{X},\mathsf{Q_C}}_{\mathsf{PRE},\mathcal{A}}(\lambda) = 1\right] - \frac{1}{2} \right| = \mathsf{negl}(\lambda).$$

We define the X *security in the multi-user multi-challenge setting with corruptions, the single-challenge setting* ($\mathsf{SC\text{-}X}$) *is a special case of* X*-security when* $\mathsf{Q_C} = 1$.

4 Hash Proof System

We need the hash proof system with additional homomorphic properties and universality to construct our proxy re-encryption scheme. Before giving the formal definition of the hash proof system that we need, we first introduce the underlying subset membership language distribution.

4.1 Subset Membership Language Distribution

In this section, we define the subset membership language distribution SMP that is used in the construction of the hash proof system. The language distribution is defined as follows:

Definition 5 (Subset Membership Language). *A subset membership language* $\mathcal{L}$ *consists of four* PPT *algorithms* $(\mathsf{Setup}, \mathsf{Sample}_{\mathcal{L}}, \mathsf{Sample}_{\mathcal{X}}, \mathsf{CheckSMP}_{\mathcal{L}})$ *with the following syntax:*

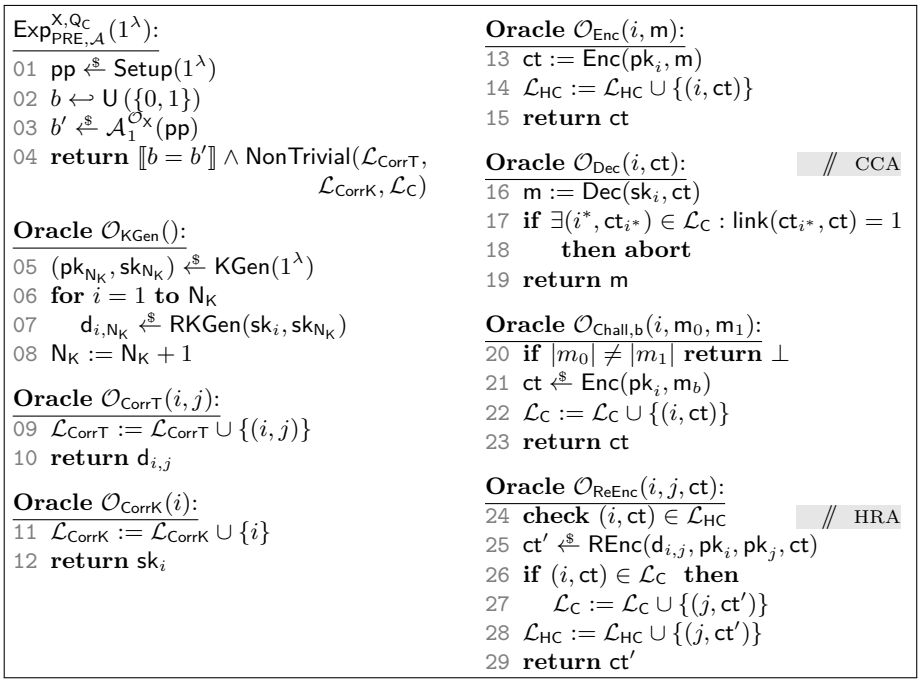

Fig. 2. Security experiment $\mathsf{Exp}^{\mathsf{X},\mathsf{Q_C}}_{\mathsf{PRE},\mathcal{A}}(1^\lambda)$ in the multi-challenge, multi-user setting. The adversary may adaptively issue any polynomial number of oracle queries. For the aHRA security, where $\mathsf{X} = \mathsf{aHRA}$ we provide $\mathcal{O}_{\mathsf{aHRA}} = (\mathcal{O}_{\mathsf{KGen}}, \mathcal{O}_{\mathsf{CorrT}}, \mathcal{O}_{\mathsf{CorrK}}, \mathcal{O}_{\mathsf{Enc}}, \mathcal{O}_{\mathsf{Chall,b}}, \mathcal{O}_{\mathsf{ReEnc}})$. For the aCCA security, where $\mathsf{X} = \mathsf{aCCA}$ we additionally give the decryption oracle, i.e., $\mathcal{O}_{\mathsf{aCCA}} = (\mathcal{O}_{\mathsf{KGen}}, \mathcal{O}_{\mathsf{CorrT}}, \mathcal{O}_{\mathsf{CorrK}}, \mathcal{O}_{\mathsf{Enc}}, \mathcal{O}_{\mathsf{Dec}}, \mathcal{O}_{\mathsf{Chall,b}}, \mathcal{O}_{\mathsf{ReEnc}})$. The parameter $\mathsf{Q_C}$ denotes the number of $\mathcal{O}_{\mathsf{Chall,b}}$ queries. The challenger implicitly maintains all key pairs $(\mathsf{pk}_i, \mathsf{sk}_i)$ and all re-encryption tokens $\mathsf{d}_{i,j}$. Since the scheme is bi-directional, $\mathsf{d}_{i,j}$ can be derived from $\mathsf{d}_{j,i}$.

- $\mathsf{Setup}(1^\lambda, \ell) \to (\mathsf{lpk}, \mathsf{td})$*: Takes a dimension* ℓ *and a security parameter* λ *as input, and outputs a public key* lpk *and a trapdoor* td*. We denote by* $\mathcal{L}_{\mathsf{lpk}} \subseteq \mathcal{X}$ *the* **NP***-language defined by* lpk*, and a relation* $\mathcal{R}_{\mathsf{lpk}} \subseteq \mathcal{L}_{\mathsf{lpk}} \times \mathcal{W}$*, where* $\mathcal{X}$ *is a space associated with* ℓ *and* $\mathcal{W}$ *is the witness space. The relation* $\mathcal{R}_{\mathsf{lpk}}$ *specifies the valid statement-witness pairs for* $\mathcal{L}_{\mathsf{lpk}}$*.*
- $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}) \to (\mathsf{x}, \mathsf{w})$*: Takes the public key* lpk *as input, and outputs a uniformly random statement* $\mathsf{x} \hookleftarrow \mathsf{U}(\mathcal{L}_{\mathsf{lpk}})$ *and a witness* w *such that* $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\mathsf{lpk}}$*.*
- $\mathsf{Sample}_{\mathcal{X}}(1^\lambda) \to \mathsf{x}$*: Takes the security parameter* 1^λ *as input, and outputs a uniformly random statement* $\mathsf{x} \hookleftarrow \mathsf{U}(\mathcal{X})$ *from the ambient space.*
- $\mathsf{CheckSMP}_{\mathcal{L}}(\mathsf{lpk}, \mathsf{td}, \mathsf{x}) \to \{0,1\}$*: Takes the public key* lpk*, trapdoor* td*, and a statement* x *as input, and outputs* 0 *or* 1 *depending on whether* x *is in the language* $\mathcal{L}_{\mathsf{lpk}}$ *defined by* lpk*.*

We require the following properties:

Correctness: *For all security parameter* λ*, all* $(\mathsf{lpk}, \mathsf{td}) \xleftarrow{\$} \mathsf{Setup}(1^\lambda, \ell)$ *generated by the setup algorithm, all statements* $\mathsf{x} \in \mathcal{X}$*, we have* $\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}}$ *is equivalent to* $\mathsf{CheckSMP}_{\mathcal{L}}(\mathsf{lpk}, \mathsf{td}, \mathsf{x}) = 1$*. Namely,*

$$\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}} \iff \mathsf{CheckSMP}_{\mathcal{L}}(\mathsf{lpk}, \mathsf{td}, \mathsf{x}) = 1.$$

Hard Subset Membership: *For all security parameter* λ*, for all* $(\mathsf{lpk}, \mathsf{td}) \xleftarrow{\$} \mathsf{Setup}(1^\lambda, \ell)$ *generated by the setup algorithm, the distribution of statements* x *sampled from* $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk})$ *is computationally indistinguishable from the distribution of statements sampled from* $\mathsf{Sample}_{\mathcal{X}}(1^\lambda)$*. Formally, for any* PPT *adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathsf{HSM}}_{\mathsf{SMP},\mathcal{A}}(1^\lambda) = \left| \Pr\left[b' = b \;\middle|\; \begin{array}{l} (\mathsf{x}_0, \mathsf{w}) \xleftarrow{\$} \mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}) \\ \mathsf{x}_1 \xleftarrow{\$} \mathsf{Sample}_{\mathcal{X}}(1^\lambda) \\ b \xleftarrow{\$} \{0,1\};\ b' \xleftarrow{\$} \mathcal{A}(\mathsf{x}_b) \end{array} \right] - \frac{1}{2} \right| = \mathsf{negl}(\lambda).$$

Linearly Homomorphism: *For all security parameter* λ*, for all* $(\mathsf{lpk}, \mathsf{td}) \xleftarrow{\$} \mathsf{Setup}(1^\lambda, \ell)$ *generated by the setup algorithm, the statement set* $\mathcal{X}$ *forms a vector space with respect to a finite field* $\mathbb{F}$*, and the language* $\mathcal{L}_{\mathsf{lpk}} \subset \mathcal{X}$ *is a subspace of* $\mathcal{X}$*. Moreover, we require that for all* $\alpha \in \mathbb{F}$*, and* $\mathsf{x}_1, \mathsf{x}_2 \in \mathcal{L}_{\mathsf{lpk}}$*, the following holds:*

$$(\mathsf{x}_1, \mathsf{w}_1) \in \mathcal{R}_{\mathsf{lpk}} \wedge (\mathsf{x}_2, \mathsf{w}_2) \in \mathcal{R}_{\mathsf{lpk}} \implies (\alpha \mathsf{x}_1 + \mathsf{x}_2, \alpha \mathsf{w}_1 + \mathsf{w}_2) \in \mathcal{R}_{\mathsf{lpk}}.$$

Language linearity: *For all security parameter* λ*, for any two language public keys generated as* $(\mathsf{lpk}, \mathsf{td}) \xleftarrow{\$} \mathsf{Setup}(1^\lambda, \ell)$ *and* $(\mathsf{lpk}', \mathsf{td}') \xleftarrow{\$} \mathsf{Setup}(1^\lambda, \ell + n)$*, let* $\mathcal{X}$ *and* $\mathcal{X}'$ *be the statement spaces associated with dimensions* ℓ *and* $\ell + n$ *respectively, and let* $\mathcal{L}_{\mathsf{lpk}} \subseteq \mathcal{X}$ *and* $\mathcal{L}_{\mathsf{lpk}'} \subseteq \mathcal{X}'$ *be the corresponding languages. We require that* $\dim(\mathcal{L}_{\mathsf{lpk}}) = \dim(\mathcal{L}_{\mathsf{lpk}'}) = k$ *where* $k = \dim(\mathcal{W})$*, and there exists a linear transformation matrix* $\mathbf{M} = (\mathbf{I}_\ell, \mathbf{r}_1, \ldots, \mathbf{r}_n)^\intercal \in \mathbb{F}^{(\ell+n)\times\ell}$*, where each* $\mathbf{r}_i \in \mathbb{F}^\ell$ *is sampled uniformly from the orthogonal complement of* $\mathcal{L}_{\mathsf{lpk}}$ *in* $\mathcal{X}$*, such that* $\mathbf{M}$ *defines an isomorphism* $\mathbf{M} : \mathcal{L}_{\mathsf{lpk}} \to \mathcal{L}_{\mathsf{lpk}'}$ *between the two languages. That is,* $\mathbf{M}(\mathcal{L}_{\mathsf{lpk}}) = \mathcal{L}_{\mathsf{lpk}'}$ *and for all* $\mathsf{x} \in \mathcal{X}$*, we have*

$$(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\mathsf{lpk}} \iff (\mathbf{M} \cdot \mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\mathsf{lpk}'}.$$

4.2 Vector Space Hash Proof System

Let SMP be the subset membership language distribution of Definition 5, and let $\mathcal{L}_{\mathsf{lpk}}$ denote the language indexed by public key lpk. We now define the associated hash proof system UHPS for $\mathcal{L}_{\mathsf{lpk}}$.

Definition 6 (Vector Space Hash Proof System). *A vector space hash proof system consists of four* PPT *algorithms* $\mathsf{UHPS} = (\mathsf{Setup}, \mathsf{PKGen}, \mathsf{PEval}, \mathsf{SEval})$ *with the following syntax:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{hk}$*: Takes a security parameter* λ *as input, and outputs a hash key* hk*.*

- $\mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}}) \to \mathsf{projk}$*: Takes a hash key* hk *and a language* $\mathcal{L}_{\mathsf{lpk}}$ *as input, and outputs a projection key* projk*.*
- $\mathsf{PEval}(\mathsf{projk}, \mathsf{x}, \mathsf{w}) \to \mathsf{hv}$*: Takes a projection key* projk*, a statement* x*, and a witness* w *for* x*, and outputs a hash value* hv*.*
- $\mathsf{SEval}(\mathsf{hk}, \mathsf{x}) \to \mathsf{hv}$*: Takes a hash key* hk *and a statement* x *as input, and outputs a hash value* hv*.*

Note that $\mathsf{lpk} \xleftarrow{\$} \mathsf{SMP.Setup}(1^\lambda, \ell)$ *is the public parameter output by the setup algorithm, which defines the language* $\mathcal{L}_{\mathsf{lpk}}$*. We denote by* $\mathcal{R}_{\mathsf{lpk}}$ *the relation associated with* $\mathcal{L}_{\mathsf{lpk}}$*, which specifies the valid statement-witness pairs for* $\mathcal{L}_{\mathsf{lpk}}$*.*

We require the following properties:

Correctness: *For all security parameter* λ*, all* $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*, and all* (x, w) *such that* w *is a valid witness for* x*, it holds that*

$$\Pr[\mathsf{PEval}(\mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}}), \mathsf{x}, \mathsf{w}) = \mathsf{SEval}(\mathsf{hk}, \mathsf{x})] = 1.$$

Vector Space Structure: *Let* $\mathbb{F}$ *denote a finite field. The sets of hash keys* $\mathcal{HK}$*, projection keys* $\mathcal{PK}_{\mathsf{lpk}}$ *(which determined by* lpk*), statements* $\mathcal{X}$*, and witnesses* $\mathcal{W}$ *each form an* $\mathbb{F}$*-vector space. The hash value space* $\mathcal{HV} = \mathbb{F}$*. Also, we require that* $\dim(\mathcal{X}) = \dim(\mathcal{HK})$

Homomorphism: *Due to the vector space structure, we require the following homomorphism*

- Projection-Homomorphism: *The projection keys satisfy the homomorphism property: For all security parameter* λ*, all* $\alpha \in \mathbb{F}$*, all* $\mathsf{hk}_1 \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$ *and* $\mathsf{hk}_2 \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*, it holds that*

$$\mathsf{PKGen}(\alpha \cdot \mathsf{hk}_1 + \mathsf{hk}_2, \mathcal{L}_{\mathsf{lpk}}) = \alpha \cdot \mathsf{PKGen}(\mathsf{hk}_1, \mathcal{L}_{\mathsf{lpk}}) + \mathsf{PKGen}(\mathsf{hk}_2, \mathcal{L}_{\mathsf{lpk}}).$$

- Message-Homomorphism: *For all security parameter* λ*, all* $\alpha \in \mathbb{F}$*, all* $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*, for all statements* $\mathsf{x}_1, \mathsf{x}_2 \in \mathcal{X}$*, it holds that*

$$\begin{aligned}&\mathsf{PEval}(\mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}}), \alpha \cdot \mathsf{x}_1 + \mathsf{x}_2, \alpha \cdot \mathsf{w}_1 + \mathsf{w}_2)\\ &= \alpha \cdot \mathsf{PEval}(\mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}}), \mathsf{x}_1, \mathsf{w}_1) + \mathsf{PEval}(\mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}}), \mathsf{x}_2, \mathsf{w}_2).\end{aligned}$$

- Key-Homomorphism: *For all security parameter* λ*, all* $\alpha \in \mathbb{F}$*, all* $\mathsf{hk}_1 \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$ *and* $\mathsf{hk}_2 \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*, for all statement* $\mathsf{x} \in \mathcal{X}$*, it holds that*

$$\mathsf{SEval}(\alpha \cdot \mathsf{hk}_1 + \mathsf{hk}_2, \mathsf{x}) = \alpha \cdot \mathsf{SEval}(\mathsf{hk}_1, \mathsf{x}) + \mathsf{SEval}(\mathsf{hk}_2, \mathsf{x}).$$

Key Collision: *Given the language parameter* lpk*, for any two different sampling of* $\mathsf{hk}_1 \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$ *and* $\mathsf{hk}_2 \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*, the probability that they produce the same projection key is negligible. Formally, we have*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{UHPS},\mathcal{A}}(1^\lambda) &:= \Pr\left[\mathsf{projk}_1 = \mathsf{projk}_2 \;\middle|\; \begin{array}{l}\mathsf{hk}_1, \mathsf{hk}_2 \xleftarrow{\$} \mathsf{Setup}(1^\lambda)\\ \mathsf{projk}_1 = \mathsf{PKGen}(\mathsf{hk}_1, \mathcal{L}_{\mathsf{lpk}})\\ \mathsf{projk}_2 = \mathsf{PKGen}(\mathsf{hk}_2, \mathcal{L}_{\mathsf{lpk}})\end{array}\right]\\ &= \mathsf{negl}(\lambda).\end{aligned}$$

(ε, d)**-Universality:** *For any security parameter* λ*, let* $\mathsf{lpk} \xleftarrow{\$} \mathsf{SMP.Setup}(1^\lambda, \ell)$*,* $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*, and* $\mathsf{projk} := \mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}})$*. Given the projection key* projk*, the set of hash keys consistent with* projk *is defined as*

$$\mathcal{HK}' := \{\mathsf{hk}' \in \mathcal{HK} \mid \mathsf{PKGen}(\mathsf{hk}', \mathcal{L}_{\mathsf{lpk}}) = \mathsf{projk}\}.$$

We require that

- $\mathcal{HK}' \subseteq \mathcal{HK}$ *forms an affine* $\mathbb{F}$*-vector space of dimension* d*, we denote by* $\mathcal{HK}''$ *the vector space corresponded to the affine vector space* $\mathcal{HK}'$*.*
- *Let* x *be a uniformly random statement from* $\mathcal{X}$*, then the probability that* $\mathsf{SEval}(\mathsf{hk}'', \mathsf{x}) = 0$ *for all* $\mathsf{hk}'' \in \mathcal{HK}''$ *is* ε*. Formally, we have*

$$\mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{A}}(1^\lambda) := \Pr\left[\forall \mathsf{hk}'' \in \mathcal{HK}'' : \mathsf{SEval}(\mathsf{hk}'', \mathsf{x}) = 0 \;\middle|\; \mathsf{x} \hookleftarrow \mathsf{U}(\mathcal{X})\right] \leq \varepsilon.$$

We note that for any $\mathsf{hk} \in \mathcal{HK}''$ *and any* $\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}}$*, we have* $\mathsf{SEval}(\mathsf{hk}, \mathsf{x}) = 0$*.* [12]

d**-MinEntropy** *Following the definition of* (ε, d)*-universality, let* $\mathcal{HK}''$ *be the* d*-dimensional vector space corresponding to the affine space* $\mathcal{HK}'$*. For any set of statements* $\{\mathsf{x}_1, \ldots, \mathsf{x}_n\} \subseteq \mathcal{X}$ *(where* $n \geq d$*) such that the linear span of their projections onto the dual space of* $\mathcal{HK}''$ *has dimension exactly* d*, i.e.,*

$$\dim(\mathit{span}\{\langle \mathsf{x}_i, \mathsf{hk}'' \rangle : \mathsf{hk}'' \in \mathcal{HK}'', i = 1, \ldots, n\}) = d,$$

the distribution of hash values

$$(\mathsf{SEval}(\mathsf{hk}'', \mathsf{x}_1), \ldots, \mathsf{SEval}(\mathsf{hk}'', \mathsf{x}_n)), \quad \mathsf{hk}'' \xleftarrow{\$} \mathsf{U}\left(\mathcal{HK}''\right)$$

has min-entropy exactly $d \log_2 |\mathbb{F}|$*, due to the fact that only* d *out of the* n *hash values are truly independent. Specifically, for any* $\mathbf{y} \in \mathbb{F}^n$*, we have*

$$\Pr\left[(\mathsf{SEval}(\mathsf{hk}'', \mathsf{x}_1), \ldots, \mathsf{SEval}(\mathsf{hk}'', \mathsf{x}_n)) = \mathbf{y}\right] \leq |\mathbb{F}|^{-d}.$$

5 aHRA-secure PRE with Tight Security

In this section, we present tightly aHRA-secure proxy re-encryption (PRE) schemes. Our constructions are based on the key-homomorphic properties of hash proof systems (HPS). Specifically, we adapt the HPS from the Cramer-Shoup cryptosystem [CS02] to build a PRE scheme. A limitation of this HPS is its single-use nature for elements outside the language, which inherently restricts the resulting PRE scheme to single-challenge security. To achieve multi-challenge security, we incorporate a ciphertext randomization technique from [HLG23].

This approach yields two constructions: a single-challenge scheme, PRE_1, and a multi-challenge scheme, PRE_2. Both can be described within a unified framework, differing only in the underlying HPS. For PRE_1, we use a $(\varepsilon, 1)$-universal VSHPS. For PRE_2, we employ a (ε, k)-universal VSHPS over a linear language of dimension k.

[12] The $\mathcal{HK}''$ defines the secret keys that project to the zero projection key over the language $\mathcal{L}_{\mathsf{lpk}}$. Thus, by perfect correctness and the key-homomorphism property, for any $\mathsf{hk} \in \mathcal{HK}''$ and any $\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}}$, we have $\mathsf{SEval}(\mathsf{hk}, \mathsf{x}) = 0$.

5.1 Generic Construction from a Vector Space HPS

We begin with a generic construction of an aHRA-secure PRE scheme from a vector space hash proof system (VSHPS). The construction is detailed in Figure 3.

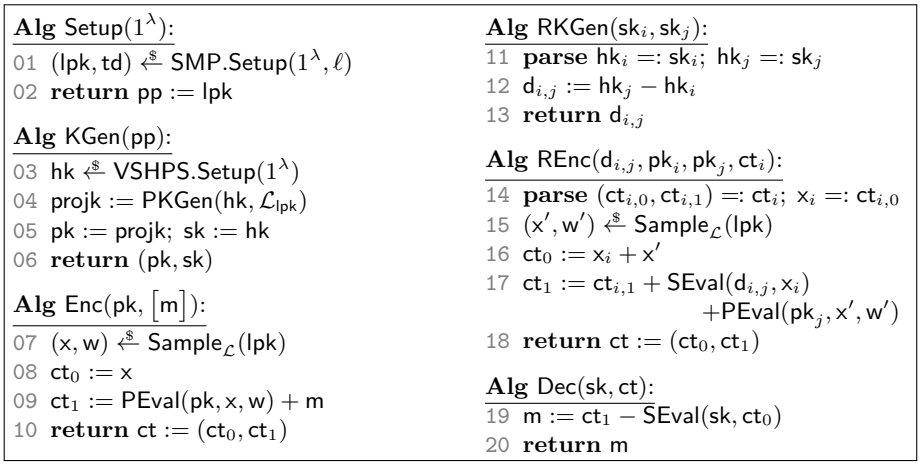

Fig. 3. Construction of aHRA-secure proxy re-encryption PRE with tight security from vector space hash proof system (VSHPS).

The correctness of the construction in Figure 3 is established by verifying the decryption of both original and re-encrypted ciphertexts. This relies on the correctness and linear properties of the underlying vector space hash proof system (VSHPS).

Correctness of Decryption. For a fresh ciphertext $\mathsf{ct} := \mathsf{Enc}(\mathsf{pk}, \mathsf{m})$, where $\mathsf{ct} = (\mathsf{x}, \mathsf{PEval}(\mathsf{pk}, \mathsf{x}, \mathsf{w}) + \mathsf{m})$, decryption with the corresponding secret key sk works as follows:

$$\begin{aligned}\mathsf{Dec}(\mathsf{sk}, \mathsf{ct}) &= \mathsf{ct}_1 - \mathsf{SEval}(\mathsf{sk}, \mathsf{ct}_0) \\ &= (\mathsf{PEval}(\mathsf{pk}, \mathsf{x}, \mathsf{w}) + \mathsf{m}) - \mathsf{SEval}(\mathsf{sk}, \mathsf{x}) = \mathsf{m}.\end{aligned}$$

The final step holds due to the correctness property of the VSHPS, which guarantees that $\mathsf{PEval}(\mathsf{pk}, \mathsf{x}, \mathsf{w}) = \mathsf{SEval}(\mathsf{sk}, \mathsf{x})$ for any statement x in the language.

Correctness of Re-encryption. Let ct_i be a ciphertext under key pk_i, and let $\mathsf{ct}_j := \mathsf{REnc}(\mathsf{d}_{i,j}, \mathsf{pk}_i, \mathsf{pk}_j, \mathsf{ct}_i)$ be the re-encrypted ciphertext. Decrypting ct_j

with secret key sk_j yields:

$$\begin{aligned}
\mathsf{Dec}(\mathsf{sk}_j, \mathsf{ct}_j) &= \mathsf{ct}_{j,1} - \mathsf{SEval}(\mathsf{sk}_j, \mathsf{ct}_{j,0}) \\
&= \big(\mathsf{ct}_{i,1} + \mathsf{SEval}(\mathsf{d}_{i,j}, \mathsf{x}_i) + \mathsf{PEval}(\mathsf{pk}_j, \mathsf{x}', \mathsf{w}')\big) - \mathsf{SEval}(\mathsf{sk}_j, \mathsf{x}_i + \mathsf{x}') \\
&= \big(\mathsf{PEval}(\mathsf{pk}_i, \mathsf{x}_i, \mathsf{w}_i) + \mathsf{m} + \mathsf{SEval}(\mathsf{hk}_j - \mathsf{hk}_i, \mathsf{x}_i) \\
&\qquad + \mathsf{PEval}(\mathsf{pk}_j, \mathsf{x}', \mathsf{w}')\big) - \big(\mathsf{SEval}(\mathsf{hk}_j, \mathsf{x}_i) + \mathsf{SEval}(\mathsf{hk}_j, \mathsf{x}')\big) \\
&= \big(\mathsf{SEval}(\mathsf{hk}_i, \mathsf{x}_i) + \mathsf{m} + \mathsf{SEval}(\mathsf{hk}_j, \mathsf{x}_i) - \mathsf{SEval}(\mathsf{hk}_i, \mathsf{x}_i) \\
&\qquad + \mathsf{SEval}(\mathsf{hk}_j, \mathsf{x}')\big) - \mathsf{SEval}(\mathsf{hk}_j, \mathsf{x}_i) - \mathsf{SEval}(\mathsf{hk}_j, \mathsf{x}') \\
&= \mathsf{m}.
\end{aligned}$$

This demonstrates that the original message is recovered after re-encryption and subsequent decryption. The derivation uses the linearity of SEval, the definition of the re-encryption key $\mathsf{d}_{i,j} = \mathsf{hk}_j - \mathsf{hk}_i$, and the correctness property of the VSHPS. In Section 5.2 and Section 5.3, we prove that the construction from Figure 3 achieves tight aHRA security in both single- and multi-challenge settings, depending on the universality of the underlying VSHPS. Specifically:

- An $(\varepsilon, 1)$-universal VSHPS yields a scheme with tight single-challenge aHRA security.
- An (ε, d)-universal VSHPS over a k-dimensional linear language, where $d \geq k$, yields a scheme with tight multi-challenge aHRA security.

5.2 Single-challenge aHRA security

Theorem 1. *Let* VSHPS *be a* $(\varepsilon, 1)$*-universal vector space hash proof system, the proxy re-encryption scheme* PRE *described in Figure 3 is* $\mathsf{SC\text{-}aHRA}$ *secure. Formally, we have:*

$$\mathsf{Adv}^{\mathsf{SC\text{-}aHRA}}_{\mathsf{PRE},\mathcal{A}}(\lambda) \leq \binom{\mathsf{N_U}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{HSM}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).$$

Proof. We prove the above theorem via the following hybrid security games $\mathbf{G}_0, \ldots, \mathbf{G}_5$. The detailed security games are presented in Figure 7.

Game $\mathbf{G}_0$: This is the original $\mathsf{SC\text{-}aHRA}$ security game. By definition, the adversary's success probability is

$$\mathsf{pr}_0 := \left| \Pr\left[\mathsf{Exp}^{\mathsf{SC\text{-}aHRA}}_{\mathsf{PRE},\mathcal{A}}(1^\lambda) = 1\right] - \frac{1}{2} \right| = 1.$$

Game $\mathbf{G}_1$: This game is identical to $\mathbf{G}_0$, except that the challenger aborts if a collision occurs among the generated public keys. Let $\mathsf{Bad}_{\mathsf{Col}}$ be the event that there exist distinct $i, j \in [\mathsf{N_U}]$ such that $\mathsf{pk}_i = \mathsf{pk}_j$. If $\mathsf{Bad}_{\mathsf{Col}}$ occurs, the game aborts.

The difference in the adversary's success probability between these two games is bounded by the probability of the event $\mathsf{Bad}_{\mathsf{Col}}$. Using a standard union bound argument, we have:

$$|\mathsf{pr}_0 - \mathsf{pr}_1| \leq \Pr[\mathsf{Bad}_{\mathsf{Col}}] \leq \binom{\mathsf{N}_{\mathsf{U}}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_2$: The only difference between $\mathbf{G}_2$ and $\mathbf{G}_1$ is that, the challenge ciphertext is encrypted using the hash key hk instead of using the public key projk. We generate the challenge ciphertext as

$$(\mathsf{ct}_0, \mathsf{w}) \xleftarrow{\$} \mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}) \qquad \mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}, \mathsf{ct}_0) + \mathsf{m}_b$$

since $\mathsf{ct}_0 \in \mathcal{L}_{\mathsf{lpk}}$ with witness w, by the perfect correctness of HPS, the challenge ciphertext in $\mathbf{G}_2$ and $\mathbf{G}_1$ is identical. Therefore, we have

$$\mathsf{pr}_2 = \mathsf{pr}_1.$$

Game $\mathbf{G}_3$: In this game, we change the way the challenge ciphertext is generated. Instead of sampling ct_0 from the language using $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk})$, we sample it from the statement set $\mathsf{ct}_0 \xleftarrow{\$} \mathsf{Sample}_{\mathcal{X}}(1^\lambda)$. Since the bad event $\mathsf{Bad}_{\mathsf{Col}}$ does not occur, the adversary cannot simply check if $\mathsf{ct}_0 \in \mathcal{L}_{\mathsf{lpk}}$ by checking if $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_j, \mathsf{ct}_0) = 0$ where i, j are any two collision users. [13] By the hard subset membership property of the language, we have

$$|\mathsf{pr}_3 - \mathsf{pr}_2| \leq \mathsf{Adv}^{\mathsf{HSM}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_4$: In $\mathbf{G}_4$, we introduce a bad event $\mathsf{Bad}_{\mathsf{Univ}}$ that captures the failure of the universal hash property. Specifically, we define

$$\mathsf{Bad}_{\mathsf{Univ}} := \{\forall \mathsf{hk}'' \in \mathcal{HK}'' : \mathsf{SEval}(\mathsf{hk}'', \mathsf{x}) = 0 \mid \mathsf{ct}_0 \xleftarrow{\$} \mathsf{U}(\mathcal{X})\}.$$

We recall that $\mathcal{HK}' \subset \mathcal{HK}$ is the affine vector space, such that we can change hk to any element $\mathsf{hk}' \in \mathcal{HK}'$ without changing its projection over $\mathcal{L}_{\mathsf{lpk}}$. $\mathcal{HK}''$ is the vector space that corresponds to the affine vector space $\mathcal{HK}'$. Using the universality of VSHPS, we have

$$|\mathsf{pr}_4 - \mathsf{pr}_3| \leq \Pr[\mathsf{Bad}_{\mathsf{Univ}}] \leq \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_5$: In this game, we modify the generation of the challenge ciphertext. We sample $v \hookleftarrow \mathsf{U}(\mathbb{F})$ at the beginning of the experiment and compute the

[13] If there is a collision between two public keys pk_i and pk_j, then the adversary can simply gets $\mathsf{hk}_i - \mathsf{hk}_j \in \mathcal{HK}''$ by querying the oracle $\mathcal{O}_{\mathsf{CorrT}}(i, j)$, and check if $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_j, \mathsf{ct}_0) = 0$. If yes, then $\mathsf{ct}_0 \in \mathcal{L}_{\mathsf{lpk}}$, otherwise $\mathsf{ct}_0 \notin \mathcal{L}_{\mathsf{lpk}}$.

challenge ciphertext as $\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}_i + v \cdot \mathsf{h}, \mathsf{ct}_0) + \mathsf{m}_b = \mathsf{SEval}(\mathsf{hk}_i, \mathsf{ct}_0) + \mathsf{m}_b + v \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0)$, where i indexes the challenge user and h is a basis of the $\mathcal{HK}''$ with dimension $d = 1$ over the linear language $\mathcal{L}_{\mathsf{lpk}}$.

We first give the value of $|\mathsf{pr}_5 - \mathsf{pr}_4|$, then we explain why the above modification is valid in Lemma 1. We have

$$|\mathsf{pr}_5 - \mathsf{pr}_4| = 0$$

In $\mathbf{G}_5$, since the game does not abort, we are guaranteed that $\mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) \neq 0$. As v is chosen uniformly at random from $\mathbb{F}$, the term $v \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0)$ acts as a one-time pad, perfectly masking the message m_b. Consequently, the adversary's advantage is zero, and its success probability is exactly $1/2$. Thus, we have

$$\mathsf{pr}_5 = \frac{1}{2}.$$

Taking union bound over all the hybrid games, we complete the proof. □

Lemma 1. *Games $\boldsymbol{G}_4$ and $\boldsymbol{G}_5$ are perfectly indistinguishable. That is, for any* PPT *adaptive adversary $\mathcal{A}$, we have*

$$\left|\Pr\left[\boldsymbol{G}_5^{\mathcal{A}} \Rightarrow 1\right] - \Pr\left[\boldsymbol{G}_4^{\mathcal{A}} \Rightarrow 1\right]\right| = 0,$$

where $\Pr\left[\boldsymbol{G}_i^{\mathcal{A}} \Rightarrow 1\right]$ denotes the probability that adversary $\mathcal{A}$ outputs 1 in game $\boldsymbol{G}_i$.

Proof. We prove the lemma via a reduction from adaptive to selective security. The argument is structured in two parts, formalized in Lemma 2 and Lemma 3.

Lemma 2 establishes that no PPT selective adversary can distinguish between $\mathbf{G}_4$ and $\mathbf{G}_5$ with any advantage. Subsequently, by Lemma 3, we can extend the result to any PPT adaptive adversary $\mathcal{A}$ that no PPT adaptive adversary $\mathcal{A}$ can distinguish between $\mathbf{G}_4$ and $\mathbf{G}_5$ with any advantage which completes the proof. □

Lemma 2. *For any* PPT *selective adversary $\mathcal{B}$, which by definition needs to commit to all its oracle queries before the game begins, we have:*

$$\left|\Pr\left[\boldsymbol{G}_5^{\mathcal{B}} \Rightarrow 1\right] - \Pr\left[\boldsymbol{G}_4^{\mathcal{B}} \Rightarrow 1\right]\right| = 0.$$

Proof. We prove the lemma by introducing three intermediate games, $\mathbf{G}_{4.1}$, $\mathbf{G}_{4.2}$ and $\mathbf{G}_{4.3}$, and show that for any selective adversary $\mathcal{B}$, the following chain of perfect indistinguishability holds: $\mathbf{G}_4 \approx_p \mathbf{G}_{4.1} \approx_p \mathbf{G}_{4.2} \approx_p \mathbf{G}_{4.3} \approx_p \mathbf{G}_5$. This implies that $\mathbf{G}_4$ and $\mathbf{G}_5$ are perfectly indistinguishable for $\mathcal{B}$.

The simulation strategy for these games, detailed in Figure 8, leverages the selective nature of $\mathcal{B}$. Since all oracle queries are known beforehand, the challenger can simulate the environment without knowing most secret keys corresponding to challenge queries. Without loss of generality, we re-index users such that all

challenge queries in $\mathcal{L}_{\mathsf{C}}$ are for users $i \in \{1, \ldots, |\mathsf{V}_{\mathsf{CL}}|\}$.

Game $\mathbf{G}_{4.1}$: This game, detailed in Figure 8, is identical to $\mathbf{G}_4$ but uses a different simulation strategy. The challenger generates a secret key hk_1 and its corresponding public key projk_1. For other challenged users $i \in \{2, \ldots, |\mathsf{V}_{\mathsf{CL}}|\}$, public keys are defined as $\mathsf{projk}_i := \mathsf{projk}_1 + \mathsf{PKGen}(\mathsf{d}_i, \mathcal{L}_{\mathsf{lpk}})$ for randomly sampled $\mathsf{d}_i \in \mathcal{HK}$. Keys for non-challenged users $j > |\mathsf{V}_{\mathsf{CL}}|$ are generated normally. This setup allows for a perfect simulation of all oracle queries. Re-encryption and corruption queries involving only challenged users can be answered using the relative tokens, while those involving non-challenged users can be answered using their known secret keys. The only secret key needed for the challenge part of the simulation is hk_1, used to generate the challenge ciphertext. The distribution of all public keys and re-encryption keys is identical to that in $\mathbf{G}_4$, so the games are perfectly indistinguishable. Thus, we have

$$\mathsf{pr}_{4.1} = \mathsf{pr}_4.$$

Game $\mathbf{G}_{4.2}$: This game modifies the generation of hk_1. Let $\mathsf{h} \in \mathcal{HK}''$ be a hash key such that $\mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) \neq 0$ for the challenge statement ct_0. Such an h is guaranteed to exist by the $(\varepsilon, 1)$-universality of the VSHPS. The challenger now computes $\mathsf{hk}_1 := \mathsf{hk}'_1 + v \cdot \mathsf{h}$, where $\mathsf{hk}'_1 \xleftarrow{\$} \mathcal{HK}$ and $v \hookleftarrow \mathsf{U}(\mathbb{F})$.

Since $\mathcal{HK}$ is a vector space over $\mathbb{F}$, the distribution of hk_1 remains uniform over $\mathcal{HK}$. Furthermore, by the definition of $\mathcal{HK}''$, we have $\mathsf{PKGen}(\mathsf{hk}_1, \mathcal{L}_{\mathsf{lpk}}) = \mathsf{PKGen}(\mathsf{hk}'_1, \mathcal{L}_{\mathsf{lpk}})$. Thus, the adversary's view is identical to that in $\mathbf{G}_{4.1}$, and we have

$$\mathsf{pr}_{4.2} = \mathsf{pr}_{4.1}.$$

Game $\mathbf{G}_{4.3}$: This game modifies the generation of the projection key projk_1 by using the hash key hk'_1 instead of hk_1. Moreover, we answer all the oracle queries except $\mathcal{O}_{\mathsf{Chall}}$ using hk'_1 instead of hk_1.

By the definition of $\mathcal{HK}''$, we have $\mathsf{PKGen}(\mathsf{hk}_1, \mathcal{L}_{\mathsf{lpk}}) = \mathsf{PKGen}(\mathsf{hk}'_1, \mathcal{L}_{\mathsf{lpk}})$, so the public key projk_1 remains unchanged. Since the adversary cannot query $\mathcal{O}_{\mathsf{CorrK}}(i)$ for user $i \in [|\mathsf{V}_{\mathsf{CL}}|]$, $\mathcal{O}_{\mathsf{CorrK}}$ leaks no information about hk_1 or hk'_1. When answering $\mathcal{O}_{\mathsf{CorrT}}(i, j)$ queries for challenged users $i, j \in [|\mathsf{V}_{\mathsf{CL}}|]$, we return $\mathsf{d}_{i,j} = \mathsf{d}_j - \mathsf{d}_i$[14], which is independent of both hk_1 and hk'_1. For non-challenged users $i, j > |\mathsf{V}_{\mathsf{CL}}|$, we answer $\mathcal{O}_{\mathsf{CorrT}}(i, j)$ queries using their known secret keys, which are also independent of both hk_1 and hk'_1.

As for $\mathcal{O}_{\mathsf{REnc}}(i, j)$: if $i, j \in [|\mathsf{V}_{\mathsf{CL}}|]$, we compute the re-encryption ciphertexts using $\mathsf{d}_{i,j} = \mathsf{d}_j - \mathsf{d}_i$, which is independent of both hk_1 and hk'_1; if $i \in [|\mathsf{V}_{\mathsf{CL}}|]$ but $j > |\mathsf{V}_{\mathsf{CL}}|]$, since $(\mathsf{ct}_0, \mathsf{ct}_1) \in \mathcal{L}_{\mathsf{HC}}$, the only component $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_1, \mathsf{ct}_0)$ that includes hk_1 is equal to $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}'_1, \mathsf{ct}_0)$.[15] If $i, j > |\mathsf{V}_{\mathsf{CL}}|$, we compute the

[14] Assumes that $\mathsf{d}_1 = 0$

[15] $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_1, \mathsf{ct}_0) = \mathsf{SEval}(\mathsf{hk}_i - (\mathsf{hk}'_1 + v \cdot \mathsf{h}), \mathsf{ct}_0) = \mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}'_1, \mathsf{ct}_0) - v \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0)$. Since $(\mathsf{ct}_0, \mathsf{ct}_1) \in \mathcal{L}_{\mathsf{HC}}$, we have $\mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) = 0$, thus $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}'_1, \mathsf{ct}_0) = \mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_1, \mathsf{ct}_0)$.

ciphertexts using their secret keys, which are also independent of both hk_1 and hk'_1.

Therefore, the adversary's view in $\mathbf{G}_{4.2}$ and $\mathbf{G}_{4.3}$ is identical, leading to

$$\mathsf{pr}_{4.3} = \mathsf{pr}_{4.2}.$$

In $\mathbf{G}_{4.3}$, the challenge ciphertext component ct_1 is computed as: $\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}'_1 + v \cdot \mathsf{h}, \mathsf{ct}_0) + \mathsf{m}_b$. The challenge ciphertext in $\mathbf{G}_5$ is computed as $\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}_1 + v \cdot \mathsf{h}, \mathsf{ct}_0) + \mathsf{m}_b$, where $\mathsf{hk}_1 \xleftarrow{\$} \mathcal{HK}$. Since hk'_1 and hk_1 are both uniformly random in $\mathcal{HK}$ and v is uniformly random in $\mathbb{F}$, the expressions $\mathsf{hk}'_1 + v \cdot \mathsf{h}$ and $\mathsf{hk}_1 + v \cdot \mathsf{h}$ follow the same distribution. Thus, $\mathbf{G}_{4.3}$ and $\mathbf{G}_5$ have identically distributed challenge ciphertexts.

The only difference between $\mathbf{G}_{4.3}$ and $\mathbf{G}_5$ lies in the generation of secret keys for users $i \in \{2, \cdots, |\mathsf{V}_{\mathsf{CL}}|\}$. In $\mathbf{G}_{4.3}$, these keys are implicitly defined as $\mathsf{hk}_i = \mathsf{hk}_1 + \mathsf{d}_i$, while in $\mathbf{G}_5$, they are sampled uniformly from $\mathcal{HK}$. However, since d_i is uniformly random in $\mathcal{HK}$ and independent of hk_1, the distribution of $\mathsf{hk}_i = \mathsf{hk}_1 + \mathsf{d}_i$ is uniform over $\mathcal{HK}$ (by the vector space structure of $\mathcal{HK}$).

Thus, the secret keys in both games are identically distributed, we have

$$\mathsf{pr}_5 = \mathsf{pr}_{4.3}.$$

Combining the above results, we complete the proof. □

Lemma 3 (S2A - Core Lemma). *For any two games $\boldsymbol{G}_1$ and $\boldsymbol{G}_2$, if for any* PPT *selective adversary $\mathcal{B}$, we have*

$$\left|\Pr\left[\boldsymbol{G}_2^{\mathcal{B}} \Rightarrow 1\right] - \Pr\left[\boldsymbol{G}_1^{\mathcal{B}} \Rightarrow 1\right]\right| = 0,$$

then for any PPT *adaptive adversary $\mathcal{A}$, we also have*

$$\left|\Pr\left[\boldsymbol{G}_2^{\mathcal{A}} \Rightarrow 1\right] - \Pr\left[\boldsymbol{G}_1^{\mathcal{A}} \Rightarrow 1\right]\right| = 0.$$

Proof. We prove the lemma by contradiction. Assume, for the sake of contradiction, that there exists a PPT adaptive adversary $\mathcal{A}$ that distinguishes between $\mathbf{G}_1$ and $\mathbf{G}_2$ with a non-zero advantage $\varepsilon > 0$. We show that this implies the existence of a PPT selective adversary $\mathcal{B}$ that also distinguishes between the games with a non-zero advantage. This contradicts the premise of the lemma, which states that no such selective adversary exists.

The core of the argument is a standard reduction that converts the adaptive adversary $\mathcal{A}$ into a selective adversary $\mathcal{B}$. The reduction relies on a query-guessing technique.

Let Q be a polynomial bound on the number of oracle queries made by $\mathcal{A}$. The selective adversary $\mathcal{B}$ operates as follows:

1. Before the game begins, $\mathcal{B}$ generates a guess $\mathcal{Q}^\star$ for the entire sequence of oracle queries that $\mathcal{A}$ will make. Since $\mathcal{A}$ is a PPT algorithm, the number of possible query sequences is at most exponential, and $\mathcal{B}$ can guess the correct sequence with a non-zero, though possibly negligible, probability $\gamma > 0$.

2. $\mathcal{B}$ commits to the guessed query sequence $\mathcal{Q}^\star$ as required for a selective adversary.
3. $\mathcal{B}$ then runs $\mathcal{A}$ as a subroutine. When $\mathcal{A}$ makes an oracle query, $\mathcal{B}$ checks if it matches the next query in the pre-committed sequence $\mathcal{Q}^\star$.
   - If the query matches, $\mathcal{B}$ forwards it to its own challenger and returns the response to $\mathcal{A}$.
   - If the query does not match, or if $\mathcal{A}$ makes more queries than predicted, $\mathcal{B}$ aborts the simulation and outputs a random bit.
4. When $\mathcal{A}$ terminates and outputs a bit b', $\mathcal{B}$ outputs the same bit b'.

The adversary $\mathcal{B}$ is a valid PPT selective adversary. If its initial guess $\mathcal{Q}^\star$ is correct (an event that occurs with probability γ), then $\mathcal{B}$ provides a perfect simulation of the environment for $\mathcal{A}$. In this scenario, $\mathcal{A}$'s distinguishing advantage is fully preserved. The overall distinguishing advantage of $\mathcal{B}$ is therefore:

$$\begin{aligned}\left|\Pr\left[\mathbf{G}_2^{\mathcal{B}} \Rightarrow 1\right] - \Pr\left[\mathbf{G}_1^{\mathcal{B}} \Rightarrow 1\right]\right| &= \gamma \cdot \left|\Pr\left[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1\right] - \Pr\left[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1\right]\right| \\ &= \gamma \cdot \varepsilon.\end{aligned}$$

Since both γ and ε are non-zero, their product is also non-zero. This means $\mathcal{B}$ can distinguish between $\mathbf{G}_1$ and $\mathbf{G}_2$ with a non-zero advantage, which contradicts our initial assumption. Therefore, no such adaptive adversary $\mathcal{A}$ can exist, and the lemma holds. □

5.3 Multi-challenge aHRA security

Theorem 2. *Let* VSHPS *be a* (ε, d)*-universal vector space hash proof system over a linear language of dimension* k *with dimension* $\ell \geq 2k$ *ambient vector space* $\mathcal{X}$*, the proxy re-encryption scheme* PRE *described in Figure 3 is* aHRA *secure in the multi-challenge setting. Specifically, for any* PPT *adversary* $\mathcal{A}$ *making at most* $\mathsf{Q_C}$ *challenge queries, there exists a* PPT *algorithm* $\mathcal{B}$ *such that* $\mathbf{T}(\mathcal{B}) \approx \mathbf{T}(\mathcal{A})$*, and we have:*

$$\mathsf{Adv}^{\mathsf{aCCA},\mathsf{Q_C}}_{\mathsf{PRE}_2,\mathcal{A}}(\lambda) \leq \binom{\mathsf{N_U}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + 3 \cdot \mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).$$

Proof. The proof proceeds via a sequence of hybrid games, $\mathbf{G}_0, \ldots, \mathbf{G}_9$. The transitions from $\mathbf{G}_0$ to $\mathbf{G}_5$ mirror those in the proof of Theorem 1, but are adapted for the multi-challenge setting. The key differences are as follows:

- The transition from $\mathbf{G}_2$ to $\mathbf{G}_3$ relies on the $\mathsf{Q_C}$-Hard Subset Membership ($\mathsf{Q_C}$-HSM) property. All $\mathsf{Q_C}$ challenge statements are switched from being sampled in the language via $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk})$ to being sampled uniformly from the ambient space via $\mathsf{Sample}_{\mathcal{X}}(1^\lambda)$. This introduces a difference of at most $\mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda)$.
- In $\mathbf{G}_4$, the bad event $\mathsf{Bad}_{\mathsf{Univ}}$ is defined with respect to the (ε, d)-universality of the VSHPS. The difference is bounded by $|\mathsf{pr}_4 - \mathsf{pr}_3| \leq \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda)$.

– In $\mathbf{G}_5$, the challenge ciphertexts are randomized. Let $\mathbf{H} \in \mathbb{F}^{d\times\ell}$ be a basis of the $\mathcal{HK}''$ over linear language $\mathcal{L}_{\mathsf{lpk}}$. We sample a random vector $\mathbf{v} \in \mathbb{F}^d$ and compute all the challenge ciphertexts as $\mathsf{ct}_1^j := \mathsf{SEval}(\mathsf{hk}_i + \mathbf{v}^\intercal \cdot \mathbf{H}, \mathsf{ct}_0^j) + \mathsf{m}_b^j$ for $j \in [\mathsf{Q_C}]$ and $i \in [\mathsf{N_U}]$ indexing the challenge users.[16] The proof of this step is similar to the single-challenge case, but requires a more detailed explanation.
  - In $\mathbf{G}_{4.1}$, we index the first challenge user as 1 and generate its secret key hk_1 normally. We add a relative token d_i for each other challenge user i,[17] so we can still simulate all the oracle queries perfectly without knowing any other user's secret key in list $\mathsf{V_{CL}}$.
  - In $\mathbf{G}_{4.2}$ and $\mathbf{G}_{4.3}$, we do the same modification as in the single-challenge case, except that we now sample a random vector $\mathbf{v} \in \mathbb{F}^d$ and compute $\mathsf{hk}_1 := \mathsf{hk}'_1 + \mathbf{v}^\intercal \cdot \mathbf{H}$, where $\mathsf{hk}'_1 \xleftarrow{\$} \mathcal{HK}$. We note that since all the challenge users share the same hash key hk_1, the random vector $\mathbf{v}^\intercal\mathbf{H}$ is also shared among all challenge ciphertexts.

  As in the single-challenge case, this change is purely conceptual, so $\mathsf{pr}_5 = \mathsf{pr}_4$.

The subsequent games, $\mathbf{G}_6$ through $\mathbf{G}_9$, are detailed in Figure 9 and complete the reduction.

Game $\mathbf{G}_6$: In this game, we change the distribution of the statement component ct_0 for all challenge ciphertexts. Instead of sampling ct_0 uniformly from the ambient space via $\mathsf{Sample}_{\mathcal{X}}(1^\lambda)$, we sample it from the language $\mathcal{L}_{\mathsf{lpk}_1}$ via $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}_1)$, where $\mathsf{lpk}_1 \xleftarrow{\$} \mathsf{SMP.Setup}(1^\lambda, \ell)$ is new language public key. The difference between $\mathbf{G}_5$ and $\mathbf{G}_6$ is bounded by the $\mathsf{Q_C}$-HSM property of the language.

$$|\mathsf{pr}_6 - \mathsf{pr}_5| \le \mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_7$: This game is a conceptual rewrite of $\mathbf{G}_6$. We first express the challenge ciphertext generation in matrix form, which does not change the adversary's view. Then we rewrite the term $\mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H}, \mathsf{x})$ as $\mathbf{R}^\intercal \cdot \mathsf{x}$ where $\mathbf{R}^\intercal := \mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H}, \mathbf{u}_1) | \cdots | \mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H}, \mathbf{u}_\ell)$ is a row vector in $\mathbb{F}^{1\times\ell}$ and $\mathbf{u}_1, \ldots, \mathbf{u}_\ell$ is the standard basis of $\mathbb{F}^\ell$. Since SEval is linear in its second argument, we have $\mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H}, \mathsf{x}) = \mathbf{R}^\intercal \cdot \mathsf{x}$. Finally, we modify the vector $\mathbf{R}$ to be uniformly random in a subspace of dimension d of $\mathbb{F}^{\ell\times 1}$, where d is the dimension of the hash key space $\mathcal{HK}''$. This change is valid because $\mathbf{v} \in \mathbb{F}^d$ is uniformly random, $\mathbf{H} \in \mathbb{F}^{d\times\ell}$ has full row rank d, and the vectors $\mathbf{u}_1, \ldots, \mathbf{u}_\ell$ are linearly independent. By the d-MinEntropy property of VSHPS, $\mathbf{R}$ has full min-entropy $d \cdot \log|\mathbb{F}|$. Thus we can rewrite $\mathbf{R}$ as $\mathbf{r}^\intercal\mathbf{M} + \mathbf{R}_0$ for some fixed $\mathbf{R}_0 \in \mathbb{F}^{\ell\times 1}$, where $\mathbf{r} \xleftarrow{\$} \mathsf{U}\left(\mathbb{F}^d\right)$ and $\mathbf{M} \in \mathbb{F}^{d\times\ell}$ is a full row rank matrix, the change is purely conceptual.

Thus, the games are perfectly indistinguishable.

$$\mathsf{pr}_7 = \mathsf{pr}_6.$$

[16] We note that there are at most $\mathsf{Q_C}$ challenge ciphertexts which are distributed among $\mathsf{N_U}$ users and the exact allocation of ciphertexts to users is not fixed.

[17] The adversary may not query a $\mathcal{O}_{\mathsf{CorrT}}$ between two challenge users, which means the graph of $\mathsf{V_{CL}}$ is not connected.

Game $\mathbf{G}_8$: This game reinterprets the structure of the challenge ciphertext. In $\mathbf{G}_7$, the vector added to the core encryption is $\begin{pmatrix} \mathsf{x} \\ \mathbf{r}^\intercal\mathbf{M}\cdot\mathsf{x}\end{pmatrix} = \begin{pmatrix}\mathbf{I}_\ell \\ \mathbf{r}^\intercal\mathbf{M}\end{pmatrix}\cdot\mathsf{x}$. Since $\mathbf{I}_\ell$ is the $\ell\times\ell$ identity matrix, $\mathbf{r}$ is a random vector in $\mathbb{F}^d$ with $d \geq k = \dim(\mathcal{L}_{\mathsf{lpk}})$ and $\mathbf{M}\in\mathbb{F}^{d\times\ell}$ is a full row rank matrix, the matrix $\begin{pmatrix}\mathbf{I}_\ell \\ \mathbf{r}^\intercal\mathbf{M}\end{pmatrix}$ is an isomorphism from $\mathcal{L}_{\mathsf{lpk}_1}$ to a new linear space of dimension k over an ambient space of dimension $\ell+1$. We denote this new linear space as $\mathcal{L}_{\mathsf{lpk}'_1}$, and rewrite the challenge ciphertexts as $\begin{pmatrix}\mathsf{ct}_0\\ \mathsf{ct}_1\end{pmatrix} := \begin{pmatrix}\mathbf{0}\\ \mathsf{SEval}(\mathsf{hk}_i,\bar{\mathsf{x}}') + \mathsf{m}_b + \mathbf{R}_0^\intercal\cdot\bar{\mathsf{x}}'\end{pmatrix} + \mathsf{x}'$, where $(\mathsf{x}',\mathsf{w}) \xleftarrow{\$} \mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}'_1)$ and $\bar{\mathsf{x}}'$ is the first ℓ components of x'.

This change is purely conceptual, thus, we have

$$\mathsf{pr}_8 = \mathsf{pr}_7.$$

Game $\mathbf{G}_9$: In this final game, we change how x' is generated for all challenge ciphertexts. Instead of sampling x' from the language $\mathcal{L}_{\mathsf{lpk}'_1}$ via $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}'_1)$, we sample it uniformly at random from the ambient space $\mathbb{F}^{\ell+1}$ via $\mathsf{Sample}_{\mathcal{X}}(1^\lambda)$. The indistinguishability between $\mathbf{G}_8$ and $\mathbf{G}_9$ is guaranteed by the $\mathsf{Q_C}$-HSM property of the language $\mathcal{L}_{\mathsf{lpk}'}$. [18]

$$|\mathsf{pr}_9 - \mathsf{pr}_8| \leq \mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

In $\mathbf{G}_9$, the challenge ciphertext is computed as $\begin{pmatrix}\mathsf{ct}_0\\ \mathsf{ct}_1\end{pmatrix} := \begin{pmatrix}\mathbf{0}\\ \mathsf{SEval}(\mathsf{hk}_i,\bar{\mathsf{x}}') + \mathsf{m}_b\end{pmatrix} + \mathsf{x}'$, where x' is a uniformly random vector in $\mathbb{F}^{\ell+1}$. This vector acts as a one-time pad, making the resulting ciphertext $(\mathsf{ct}_0,\mathsf{ct}_1)$ uniformly random and statistically independent of the challenge bit b. Consequently, the adversary's advantage is zero.

$$\mathsf{pr}_9 = \frac{1}{2}.$$

Combining the above results, we complete the proof. □

5.4 Instantiation with MDDH assumption

We give the instantiation of the subset membership language and the vector space hash proof system based on the MDDH assumption in bilinear groups.

[18] The language dimension remains the same, but the ambient space dimension differs (from $\mathbb{F}^\ell$ to $\mathbb{F}^{\ell+1}$), which causes a negligible difference in the HSM advantage. For simplicity, we uniformly denote it by $\mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda)$.

Subset Membership Language Let $\mathbb{G}$ be a prime order group of order p, and let g be a generator of $\mathbb{G}$. We define a language public key as $\mathbf{A} \in \mathbb{G}^{\ell \times k}$, where k is an arbitrary integer and $\ell \geq 2k$ varies dependent on the properties we need. More specifically, for (ε, d)-universal VSHPS instantiation, we will need $d = \ell - k \geq k$. We give the full description of subset membership language in Figure 4, where $(\mathbb{G}, \mathsf{p}, \mathsf{g}) \xleftarrow{\$} \mathsf{GGen}(1^\lambda)$ generates a DDH hard group. The

$\underline{\textbf{Alg } \mathsf{Setup}(1^\lambda):}$
01 $(\mathbb{G}, \mathsf{p}, \mathsf{g}) \xleftarrow{\$} \mathsf{GGen}(1^\lambda)$
02 $\mathbf{A} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{\ell \times k}\right)$
03 $\mathbf{K} \xleftarrow{\$} \mathsf{Base}(\ker(\mathbf{A})) \in \mathbb{Z}_\mathsf{p}^{\ell \times (\ell - k)}$
04 **return** $(\mathsf{lpk} := \left[\mathbf{A}\right]_1, \mathsf{td} := \mathbf{K})$

$\underline{\textbf{Alg } \mathsf{Sample}_\mathcal{X}(1^\lambda):}$
05 $\mathsf{x} := \mathbf{x} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{\ell}\right)$
06 **return** x

$\underline{\textbf{Alg } \mathsf{Sample}_\mathcal{L}(\mathsf{lpk}):}$
07 $\mathsf{w} := \mathbf{r} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k}\right)$
08 $\mathsf{x} := \left[\mathbf{x}\right]_1 = \left[\mathbf{A}\right]_1 \mathbf{r}$
09 **return** (x, w)

$\underline{\textbf{Alg } \mathsf{CheckSMP}_\mathcal{L}(\mathsf{lpk}, \mathsf{td}, \mathsf{x}):}$
10 **parse** $(\left[\mathbf{x}\right]_1, \mathbf{K}) =: (\mathsf{x}, \mathsf{td})$
11 **return** $[\![\mathbf{K}^\intercal \left[\mathbf{x}\right]_1 = \mathbf{0}]\!]$

Fig. 4. Instantiation of the subset membership language $\mathcal{L}_\mathsf{lpk}$ based on the MDDH assumption in bilinear groups. Here, $\left[\cdot\right]_1$ denotes the group action that maps elements from $\mathbb{Z}_\mathsf{p}$ to $\mathbb{G}$. $\mathsf{Base}(\ker \mathbf{A})$ denotes the probabilistic algorithm that returns a basis for the kernel of the matrix $\mathbf{A}$.

properties of this construction are established as follows.

Correctness The correctness of the membership check follows from the definition of the language and the trapdoor. For any statement $\mathsf{x} = \left[\mathbf{A}\right]_1 \mathbf{r}$ in the language $\mathcal{L}_\mathsf{lpk}$, we have $\mathbf{K}^\intercal \mathsf{x} = \mathbf{K}^\intercal(\left[\mathbf{A}\right]_1 \mathbf{r}) = \mathbf{0}$, since $\mathbf{K}$ is a basis for the left kernel of $\mathbf{A}$.

Hard Subset Membership The indistinguishability of a random statement $\mathsf{x} \in \mathbb{G}^\ell$ from a statement in the language $\mathcal{L}_\mathsf{lpk}$ is guaranteed by the Matrix Decisional Diffie-Hellman (MDDH) assumption. Moreover, due to the random self-reducibility [EHK+13], we have

$$\mathsf{Adv}^{\mathsf{MDDH},\mathsf{Q_C}}_{\mathbb{G},\mathcal{A}}(1^\lambda) = \mathsf{Adv}^{\mathsf{MDDH}}_{\mathbb{G},\mathcal{B}}(1^\lambda) + \mathsf{negl}(\lambda).$$

Linear Homomorphism The language is a k-dimensional vector space over $\mathbb{Z}_p$, and thus is linearly homomorphic by construction.

Language linearity This property is established as follows: Given two language public keys $\mathsf{lpk} = \left[\mathbf{A}\right]_1$ and $\mathsf{lpk}' = \left[\mathbf{A}'\right]_1$, where $\mathbf{A} \in \mathbb{Z}_\mathsf{p}^{\ell \times k}$ and $\mathbf{A}' \in \mathbb{Z}_\mathsf{p}^{(\ell+n) \times k}$ are both full-rank matrices, the languages $\mathcal{L}_\mathsf{lpk} = \{\left[\mathbf{A}\right]_1 \mathbf{w} : \mathbf{w} \in \mathbb{Z}_\mathsf{p}^k\}$ and $\mathcal{L}_{\mathsf{lpk}'} = \{\left[\mathbf{A}'\right]_1 \mathbf{w} : \mathbf{w} \in \mathbb{Z}_\mathsf{p}^k\}$ are isomorphic as k-dimensional vector spaces. The linear transformation matrix $\mathbf{M} = (\mathbf{I}_\ell, \mathbf{r}_1, \ldots, \mathbf{r}_n)^\intercal \in \mathbb{F}^{(\ell+n) \times \ell}$, where each $\mathbf{r}_i \in \mathbb{Z}_\mathsf{p}^\ell$ is sampled uniformly from the orthogonal complement of $\mathrm{span}(\mathbf{A})$ in $\mathbb{Z}_\mathsf{p}^\ell$, defines the isomorphism $\mathbf{M} : \mathcal{L}_\mathsf{lpk} \to \mathcal{L}_{\mathsf{lpk}'}$. Specifically, for

any $\mathsf{x} = [\mathbf{A}]_1 \mathbf{w} \in \mathcal{L}_{\mathsf{lpk}}$, we have

$$\mathbf{M} \cdot \mathsf{x} = \begin{pmatrix} \mathbf{I}_\ell \\ \mathbf{r}_1^\intercal \\ \vdots \\ \mathbf{r}_n^\intercal \end{pmatrix} \cdot [\mathbf{A}]_1 \mathbf{w} = \begin{pmatrix} [\mathbf{A}]_1 \\ \mathbf{r}_1^\intercal [\mathbf{A}]_1 \\ \vdots \\ \mathbf{r}_n^\intercal [\mathbf{A}]_1 \end{pmatrix} \mathbf{w} = [\mathbf{A}']_1 \mathbf{w} \in \mathcal{L}_{\mathsf{lpk}'}$$

where $\mathbf{A}' = (\mathbf{A}^\intercal, \mathbf{A}^\intercal \mathbf{r}_1, \ldots, \mathbf{A}^\intercal \mathbf{r}_n)^\intercal \in \mathbb{Z}_\mathsf{p}^{(\ell+n)\times k}$ maintains full rank k, ensuring that $\mathbf{M}(\mathcal{L}_{\mathsf{lpk}}) = \mathcal{L}_{\mathsf{lpk}'}$ and the relation is preserved: $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\mathsf{lpk}} \iff (\mathbf{M} \cdot \mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\mathsf{lpk}'}$.

Vector Space Hash Proof System With respect to the subset membership language defined in Figure 4, we give the instantiation of the vector space hash proof system in Figure 5 based on the MDDH assumption. The properties of this

Alg $\mathsf{Setup}(1^\lambda)$:
01 $\mathsf{hk} := \mathbf{k} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^\ell\right)$
02 **return** hk

Alg $\mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}})$:
03 **parse** $\mathbf{k} =: \mathsf{hk}$
04 $\mathsf{projk} := \mathbf{k}^\intercal [\mathbf{A}]_1$
05 **return** projk

Alg $\mathsf{PEval}(\mathsf{projk}, \mathsf{x}, \mathsf{w})$:
06 **parse** $\mathbf{k}^\intercal [\mathbf{A}]_1 =: \mathsf{projk}$; $\mathbf{w} =: \mathsf{w}$
07 **return** $\mathsf{hv} := \left(\mathbf{k}^\intercal [\mathbf{A}]_1\right) \cdot \mathbf{w}$

Alg $\mathsf{SEval}(\mathsf{hk}, \mathsf{x})$:
08 **parse** $\mathbf{k} =: \mathsf{hk}$; $\mathbf{x} =: \mathsf{x}$
09 **return** $\mathsf{hv} := \mathbf{k}^\intercal \mathbf{x}$

Fig. 5. Construction of VSHPS with respect to the subset membership language $\mathcal{L}_{\mathsf{lpk}} = \mathsf{Span}(\mathbf{A})$.

construction are established as follows.

Correctness The correctness follows from the definitions of PEval and SEval. For any statement $\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}}$ with witness w, we have $\mathsf{PEval}(\mathsf{projk}, \mathsf{x}, \mathsf{w}) = \left(\mathbf{k}^\intercal [\mathbf{A}]_1\right) \cdot \mathsf{w} = \mathbf{k}^\intercal \cdot \left([\mathbf{A}]_1 \mathsf{w}\right) = \mathsf{SEval}(\mathsf{hk}, \mathsf{x})$.

Vector Space Structure Let $\mathbb{F} = \mathbb{Z}_\mathsf{p}$, the vector space structure is preserved in the projection, as the projection of a linear combination of vectors is the same linear combination of their projections.

Homomorphism The projection operation is a homomorphism, meaning it respects the linear structure of the space.

Key Collision Resistance Given two distinct secret keys $\mathbf{k}, \mathbf{k}' \in \mathbb{Z}_\mathsf{p}^\ell$, the probability that $\mathbf{k}^\intercal [\mathbf{A}]_1 = \mathbf{k}'^\intercal [\mathbf{A}]_1$ is negligible. This would require $\mathbf{k} - \mathbf{k}'$ to lie in the left kernel of $[\mathbf{A}]_1$. Since $\mathbf{k} - \mathbf{k}'$ is non-zero and uniformly random, the probability is at most $1/|\mathbb{F}|^k$, which is negligible.

(ε, d)-Universality The construction is (ε, d)-universal given a projection key $\mathsf{projk} := \mathbf{k}^\intercal [\mathbf{A}]_1$, since all public evaluations are of the form $\mathbf{k}^\intercal [\mathbf{A}]_1 \cdot \mathbf{r}$. Therefore, $\mathbf{k} + \mathsf{Span}(\ker([\mathbf{A}]_1))$ forms an affine vector space of dimension $d = \ell - k$ consistent with all public evaluations.

d-**MinEntropy** We have $\mathsf{SEval}(\mathsf{hk}'', \mathsf{x}) = \mathsf{hk}'' \cdot \mathsf{x}$ where $\mathsf{hk}'' \xleftarrow{\$} \mathsf{U}\left(\mathcal{HK}''\right)$ and $\mathcal{HK}''$ is d-dimensional. For any n statements $\{\mathsf{x}_1, \ldots, \mathsf{x}_n\}$ (where $n \geq d$) with exactly d linearly independent among them, the hash values $(\mathsf{hk}'' \cdot \mathsf{x}_1, \ldots, \mathsf{hk}'' \cdot \mathsf{x}_n)$ have min-entropy $d \log_2 |\mathbb{Z}_\mathsf{p}|$ because the d-dimensional hk'' can extract d independent values through inner products.

6 CCA-secure PRE with (almost) tight security

In this section, we extend the aHRA-secure proxy re-encryption scheme from Figure 3 to achieve CCA-security with a tight security reduction.

6.1 General construction of aCCA construction with (almost) tight security

Our construction builds upon the aHRA-secure proxy re-encryption scheme from Section 5. In that scheme, a ciphertext $\mathsf{ct} = (\mathsf{ct}_0, \mathsf{ct}_1)$ consists of a statement ct_0 for a hard subset membership problem and a component $\mathsf{ct}_1 = \mathsf{PEval}(\mathsf{projk}_i, \mathsf{ct}_0, \mathsf{w}) + \mathsf{m}$ that binds the message m to an evaluation of ct_0 within a hash proof system.

The first obstacle in attaining CCA2-security is that it forbids any ciphertext malleability. A standard remedy, going back to [CHK03], is to append a one-time signature to the ciphertext. One might be tempted to sign the component ct_1, since it binds the message. However, re-encryption necessarily modifies ct_1, thereby invalidating the signature. This conflict precludes a direct extension of the aHRA-secure scheme to a aCCA-secure construction.

To resolve this issue, we restructure the ciphertext by introducing a third component, ct_2. We redefine ct_1 to be solely the hash proof system evaluation, i.e., $\mathsf{ct}_1 = \mathsf{PEval}(\mathsf{projk}_i, \mathsf{ct}_0, \mathsf{w})$, and move the message to ct_2, setting $\mathsf{ct}_2 = \mathsf{PEval}(\mathsf{projk}, \mathsf{ct}_0, \mathsf{w}) + \mathsf{m}$, where projk is a public parameter common to all users. Consequently, ct_2 remains invariant under re-encryption, which only affects ct_1. This design allows us to sign the pair $(\mathsf{ct}_0, \mathsf{ct}_2)$ to ensure integrity, providing the non-malleability required for aCCA-security while preserving the re-encryption functionality.

Furthermore, inspired by the Cramer-Shoup cryptosystem, we must ensure that the ciphertext components ct_0 and ct_1 are well-formed. Specifically, we need to prove that ct_1 is a valid evaluation of ct_0 under the hash proof system. We achieve this by incorporating a simulation-sound tag-based language-malleable NIZK proof system (tLM-NIZK). This system proves the relation between ct_0 and ct_1 using a tag derived from the one-time signature's verification key, i.e., $\mathsf{t} = \mathsf{H}(\mathsf{ovk})$. The final ciphertext is thus $\mathsf{ct} = (\mathsf{ct}_0, \mathsf{ct}_1, \mathsf{ct}_2, \mathsf{ovk}, \sigma, \pi)$.

Remark 1. It is important to note that our NIZK proof system cannot directly prove that ct_1 is a valid evaluation of ct_0 under the hash proof system in general, since the hash function may not possess the necessary linear structure required by the Groth-Sahai proof system [GS08]. Fortunately, the specific hash proof system

instantiated from the MDDH assumption that we employ does exhibit such linear structure, making our construction viable. Whether there exists a NIZK proof system capable of proving this relation for arbitrary hash proof systems remains an open question.

Our construction combines the following cryptographic primitives:

- A language distribution SMP for a subset membership problem.
- A vector space hash proof system VSHPS.
- A one-time signature scheme OTS.
- A special simulation-sound tag-based language-malleable NIZK proof system $\mathsf{tLM\text{-}NIZK}$ for the language sampled by SMP.

We give our construction in Figure 6.

Alg $\mathsf{Setup}(1^\lambda)$:
01 $(\mathsf{lpk}, \mathsf{td}) \xleftarrow{\$} \mathsf{SMP.Setup}(1^\lambda, \ell)$
02 $\mathsf{projk} \hookleftarrow \mathsf{U}(\mathcal{PK}_{\mathsf{lpk}})$
03 $\mathsf{crs} \xleftarrow{\$} \mathbf{\Pi}.\mathsf{Setup}(1^\lambda)$
04 **return** $\mathsf{pp} := (\mathsf{lpk}, \mathsf{projk}, \mathsf{crs})$

Alg $\mathsf{KGen}(\mathsf{pp})$:
05 $\mathsf{sk}_i := \mathsf{hk}'_i \xleftarrow{\$} \mathsf{VSHPS.Setup}(1^\lambda)$
06 $\mathsf{projk}'_i := \mathsf{PKGen}(\mathsf{hk}'_i, \mathcal{L}_{\mathsf{lpk}})$
07 $\mathsf{pk}_i := \mathsf{projk}_i = \mathsf{projk} + \mathsf{projk}'_i$
08 **return** $(\mathsf{pk}_i, \mathsf{sk}_i)$

Alg $\mathsf{RKGen}(\mathsf{sk}_i, \mathsf{sk}_j)$:
09 **parse** $\mathsf{sk}_i := \mathsf{hk}'_i$; $\mathsf{sk}_j := \mathsf{hk}'_j$
10 $\mathsf{d}_{i,j} := \mathsf{hk}'_j - \mathsf{hk}'_i$
11 **return** $\mathsf{d}_{i,j}$

Alg $\mathsf{Enc}(\mathsf{pk}_i, \mathsf{m})$:
12 $(\mathsf{x}, \mathsf{w}) \xleftarrow{\$} \mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk})$
13 $\mathsf{ct}_0 := \mathsf{x}$
14 $\mathsf{ct}_1 := \mathsf{PEval}(\mathsf{projk}_i, \mathsf{x}, \mathsf{w})$
15 $\mathsf{ct}_2 := \mathsf{PEval}(\mathsf{projk}, \mathsf{x}, \mathsf{w}) + \mathsf{m}$
16 $(\mathsf{ovk}, \mathsf{osk}) \xleftarrow{\$} \mathsf{OTS.KGen}(1^\lambda)$
17 $\mathsf{t} = \mathsf{H}(\mathsf{ovk})$
18 $\sigma \xleftarrow{\$} \mathsf{OTS.Sig}(\mathsf{osk}, (\mathsf{ct}_0, \mathsf{ct}_2))$
19 $\pi \xleftarrow{\$} \mathbf{\Pi}.\mathsf{Prove}(\mathsf{crs}, \mathsf{t}, (\mathsf{ct}_0, \mathsf{ct}_1), \mathsf{w})$
20 **return** $\mathsf{ct} := (\mathsf{ct}_0, \mathsf{ct}_1, \mathsf{ct}_2, \mathsf{ovk}, \sigma, \pi)$

Alg $\mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$:
21 **parse** $\mathsf{sk}_i := \mathsf{hk}'_i$; $\mathsf{t} = \mathsf{H}(\mathsf{ovk})$
22 **check** $\mathbf{\Pi}.\mathsf{Ver}(\mathsf{crs}, \mathsf{t}, (\mathsf{ct}_0, \mathsf{ct}_1), \pi) = 1$
23 **check** $\mathsf{OTS.Ver}(\mathsf{ovk}, (\mathsf{ct}_0, \mathsf{ct}_2), \sigma) = 1$
24 **return** $\mathsf{m} := \mathsf{ct}_2 - \mathsf{ct}_1 + \mathsf{SEval}(\mathsf{sk}_i, \mathsf{ct}_0)$

Alg $\mathsf{REnc}(\mathsf{d}_{i,j}, \mathsf{pk}_i, \mathsf{pk}_j, \mathsf{ct})$:
25 **parse** $(\mathsf{ct}_0, \mathsf{ct}_1, \mathsf{ct}_2, \mathsf{ovk}, \sigma, \pi) := \mathsf{ct}$
26 $\mathsf{t} = \mathsf{H}(\mathsf{ovk})$
27 **check** $\mathbf{\Pi}.\mathsf{Ver}(\mathsf{crs}, \mathsf{t}, (\mathsf{ct}_0, \mathsf{ct}_1), \pi) = 1$
28 $\mathsf{ct}'_1 := \mathsf{ct}_1 + \mathsf{SEval}(\mathsf{d}_{i,j}, \mathsf{ct}_0)$
29 $\pi' \xleftarrow{\$} \mathbf{\Pi}.\mathsf{Trans}(\mathsf{crs}, \mathsf{lpk}_i, \mathsf{t}, \mathsf{ct}_0, \pi, \mathsf{d}_{i,j})$
30 **return** $\mathsf{ct} := (\mathsf{ct}_0, \mathsf{ct}'_1, \mathsf{ct}_2, \sigma, \pi')$

Fig. 6. General construction of aCCA-secure PRE scheme $\mathsf{PRE}_{\mathsf{aCCA}}$ with (almost) tight security. $\mathbf{\Pi}$ is a tLM-NIZK of the statement $\exists \mathsf{w} : \mathsf{x} \in_{\mathsf{w}} \mathcal{L}_{\mathsf{lpk}} \wedge \mathsf{ct}_1 = \mathsf{PEval}(\mathsf{projk}_i, \mathsf{x}, \mathsf{w})$.

Correctness. The correctness of the above construction follows from the correctness of the underlying VSHPS, OTS, and tLM-NIZK proof system.

$$\begin{aligned}
&\mathsf{ct}_2 - \mathsf{ct}_1 + \mathsf{SEval}(\mathsf{sk}_i, \mathsf{ct}_0) \\
=&\mathsf{PEval}(\mathsf{projk}, \mathsf{x}, \mathsf{w}) + \mathsf{m} - \mathsf{PEval}(\mathsf{projk}_i, \mathsf{x}, \mathsf{w}) + \mathsf{SEval}(\mathsf{sk}_i, \mathsf{ct}_0) \\
=&\mathsf{PEval}(\mathsf{projk}, \mathsf{x}, \mathsf{w}) + \mathsf{m} - \mathsf{PEval}(\mathsf{projk} + \mathsf{projk}'_i, \mathsf{x}, \mathsf{w}) + \mathsf{SEval}(\mathsf{sk}_i, \mathsf{ct}_0) + \mathsf{m} \\
=&\mathsf{m} - \mathsf{PEval}(\mathsf{projk}'_i, \mathsf{x}, \mathsf{w}) + \mathsf{SEval}(\mathsf{hk}'_i, \mathsf{x}) = \mathsf{m}.
\end{aligned}$$

6.2 (Almost) Tight aCCA security

The security proof for our aCCA-secure scheme builds upon the proof for the aHRA-secure scheme from Section 5. However, handling the decryption and re-encryption oracles requires additional arguments.

Decryption Oracle Queries We now detail the handling of decryption oracle queries. The core of the argument relies on the unforgeability of the one-time signature scheme and the special simulation-soundness of the tLM-NIZK proof system. When the oracle receives a ciphertext $\mathsf{ct} = (\mathsf{ct}_0, \mathsf{ct}_1, \mathsf{ct}_2, \mathsf{ovk}, \sigma, \pi)$ for decryption under a key pk_i, we distinguish between the following cases. We denote components of the challenge ciphertext with a superscript star ($\star$).

The query uses the challenge verification key $\mathsf{ovk}^\star$ but is not a re-encryption of the challenge ciphertext. This case covers queries where $\mathsf{ovk} = \mathsf{ovk}^\star$ but either $\sigma \neq \sigma^\star$ or $(\mathsf{ct}_0, \mathsf{ct}_2) \neq (\mathsf{ct}_0^\star, \mathsf{ct}_2^\star)$. For the decryption query to be valid, σ must be a valid signature on $(\mathsf{ct}_0, \mathsf{ct}_2)$ under $\mathsf{ovk}^\star$. This constitutes a forgery against the one-time signature scheme, an event that occurs with only negligible probability due to its unforgeability.

A collision has been found for ovk. This corresponds to the case where $\mathsf{ovk} \neq \mathsf{ovk}^\star$, but $\mathsf{t} = \mathsf{H}(\mathsf{ovk}) = \mathsf{H}(\mathsf{ovk}^\star) = \mathsf{t}^\star$. If this event occurs, the adversary has successfully found a collision for the hash function.

The query uses a new verification key. This is the case where $\mathsf{ovk} \neq \mathsf{ovk}^\star$. The challenger has not simulated a proof for the tag $\mathsf{t} = \mathsf{H}(\mathsf{ovk})$. By the special simulation-soundness of the tLM-NIZK system, an adversary cannot produce a valid proof π for a false statement. Thus, if the proof verifies, it must be that $\exists \mathsf{w} : \mathsf{x} \in_{\mathsf{w}} \mathcal{L}_{\mathsf{lpk}} \wedge \mathsf{ct}_1 = \mathsf{PEval}(\mathsf{projk}_i, \mathsf{x}, \mathsf{w})$. The information revealed by such a query does not compromise the (ε, d)-universality of the hash proof system, as the set of possible hash keys consistent with all public information and oracle queries remains an affine subspace of dimension at least d.

The query is a re-encryption of the challenge ciphertext. This corresponds to the case where $\mathsf{ovk} = \mathsf{ovk}^\star$, $\sigma = \sigma^\star$, and $(\mathsf{ct}_0, \mathsf{ct}_2) = (\mathsf{ct}_0^\star, \mathsf{ct}_2^\star)$. Here, the query ciphertext ct differs from the challenge ciphertext $\mathsf{ct}^\star$ only in the components ct_1 and π. For the proof π to be valid, the special simulation-soundness of the tLM-NIZK implies that ct must be a valid re-encryption of some ciphertext. Since the other components match the challenge, ct must be a re-encryption of $\mathsf{ct}^\star$. Such queries are disallowed by the definition of CCA-security, so the oracle can reject them.

Re-Encryption Oracle Queries The primary difference in handling re-encryption oracle queries compared to the aHRA-secure scheme arises when the adversary requests the re-encryption of a self-generated ciphertext from an uncorrupted user to a corrupted user. In other scenarios, the simulation can rely on the re-encryption key, which is available since re-encryption key is either for two uncorrupted users or for two corrupted users.

To handle the challenging case, we add a check to the algorithm REnc which ensures the components ct_0 and ct_1 is well-formed, i.e., $\mathbf{\Pi}.\mathsf{Ver}(\mathsf{crs}, \mathsf{t}, (\mathsf{ct}_0, \mathsf{ct}_1), \pi) = 1$. This ensures that only well-formed ciphertexts—those for which there exists a witness w such that $\mathsf{x} \in_{\mathsf{w}} \mathcal{L}_{\mathsf{lpk}}$ and $\mathsf{ct}_1 = \mathsf{PEval}(\mathsf{projk}_i, \mathsf{x}, \mathsf{w})$—can be re-encrypted. If this check fails, the oracle rejects the query. In this case, all the information the adversary can learn from $\mathcal{O}_{\mathsf{ReEnc}}$ is through the evaluation of $\mathsf{SEval}(\mathsf{d}_{i,j}, \mathsf{ct}_0)$ for users i and j, which leak no information except the public keys. [19]

Summarizing the above intuitions, we have the following theorem.

Theorem 3 (aCCA security). *Let* $\mathsf{PRE}_{\mathsf{aCCA}}$ *be the proxy re-encryption scheme from Figure 6. Assume that*

- SMP *is a hard subset membership language distribution.*
- *The vector space hash proof system* VSHPS *is* (ε, d)*-universal for negligible* ε *with* $d \geq 1$ *for single-challenge* aCCA *or* $d \geq k$ *for multi-challenge* aCCA*.*
- *The one-time signature scheme* OTS *is strongly unforgeable.*
- *The* tLM-NIZK *proof system* $\mathbf{\Pi}$ *is a (one-time/unbounded) extractable special simulation-sound tag-based language-malleable* NIZK *proof system.*

Then, $\mathsf{PRE}_{\mathsf{aCCA}}$ *is* aCCA*-secure. We will prove the theorem in G and H.*

[19] We will prove this result in the security proof of aCCA-security PRE.

References

ABH09. Giuseppe Ateniese, Karyn Benson, and Susan Hohenberger. Key-private proxy re-encryption. In Marc Fischlin, editor, *CT-RSA 2009*, volume 5473 of *LNCS*, pages 279–294. Springer, Berlin, Heidelberg, April 2009. 2

ABPW13. Yoshinori Aono, Xavier Boyen, Le Trieu Phong, and Lihua Wang. Key-private proxy re-encryption under LWE. In Goutam Paul and Serge Vaudenay, editors, *INDOCRYPT 2013*, volume 8250 of *LNCS*, pages 1–18. Springer, Cham, December 2013. 2, 4

AFGH05. Giuseppe Ateniese, Kevin Fu, Matthew Green, and Susan Hohenberger. Improved proxy re-encryption schemes with applications to secure distributed storage. In *NDSS 2005*. The Internet Society, February 2005. 2, 4

AJO+19. Masayuki Abe, Charanjit S. Jutla, Miyako Ohkubo, Jiaxin Pan, Arnab Roy, and Yuyu Wang. Shorter QA-NIZK and SPS with tighter security. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 669–699. Springer, Cham, December 2019. 5, 7, 40, 43

BBS98. Matt Blaze, Gerrit Bleumer, and Martin Strauss. Divertible protocols and atomic proxy cryptography. In Kaisa Nyberg, editor, *EUROCRYPT'98*, volume 1403 of *LNCS*, pages 127–144. Springer, Berlin, Heidelberg, May / June 1998. 2, 4

BHJ+15. Christoph Bader, Dennis Hofheinz, Tibor Jager, Eike Kiltz, and Yong Li. Tightly-secure authenticated key exchange. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part I*, volume 9014 of *LNCS*, pages 629–658. Springer, Berlin, Heidelberg, March 2015. 2

BJLS16. Christoph Bader, Tibor Jager, Yong Li, and Sven Schäge. On the impossibility of tight cryptographic reductions. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 273–304. Springer, Berlin, Heidelberg, May 2016. 4, 5

BLMR13. Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 410–428. Springer, Berlin, Heidelberg, August 2013. 2

CH07. Ran Canetti and Susan Hohenberger. Chosen-ciphertext secure proxy re-encryption. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007*, pages 185–194. ACM Press, October 2007. 2, 3, 4, 9

CHK03. Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 255–271. Springer, Berlin, Heidelberg, May 2003. 28

Coh19. Aloni Cohen. What about bob? The inadequacy of CPA security for proxy reencryption. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part II*, volume 11443 of *LNCS*, pages 287–316. Springer, Cham, April 2019. 2, 4

CS02. Ronald Cramer and Victor Shoup. Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption. In Lars R. Knudsen, editor, *EUROCRYPT 2002*, volume 2332 of *LNCS*, pages 45–64. Springer, Berlin, Heidelberg, April / May 2002. 4, 5, 16

EHK+13. Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 129–147. Springer, Berlin, Heidelberg, August 2013. 26, 34

EHK+17. Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Luis Villar. An algebraic framework for Diffie-Hellman assumptions. *Journal of Cryptology*, 30(1):242–288, January 2017. 34

FKKP19. Georg Fuchsbauer, Chethan Kamath, Karen Klein, and Krzysztof Pietrzak. Adaptively secure proxy re-encryption. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part II*, volume 11443 of *LNCS*, pages 317–346. Springer, Cham, April 2019. 3, 4

FL19. Xiong Fan and Feng-Hao Liu. Proxy re-encryption and re-signatures from lattices. In Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, editors, *ACNS 2019*, volume 11464 of *LNCS*, pages 363–382. Springer, Cham, June 2019. 2, 4

Gen09. Craig Gentry. *A fully homomorphic encryption scheme*. PhD thesis, Stanford University, USA, 2009. 2, 4

GHKP18. Romain Gay, Dennis Hofheinz, Lisa Kohl, and Jiaxin Pan. More efficient (almost) tightly secure structure-preserving signatures. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 230–258. Springer, Cham, April / May 2018. 51

GS08. Jens Groth and Amit Sahai. Efficient non-interactive proof systems for bilinear groups. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 415–432. Springer, Berlin, Heidelberg, April 2008. 28, 36, 37, 44, 47, 53

HLG23. Shuai Han, Shengli Liu, and Dawu Gu. Almost tight multi-user security under adaptive corruptions & leakages in the standard model. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 132–162. Springer, Cham, April 2023. 4, 5, 6, 7, 16

HRsV07. Susan Hohenberger, Guy N. Rothblum, abhi shelat, and Vinod Vaikuntanathan. Securely obfuscating re-encryption. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 233–252. Springer, Berlin, Heidelberg, February 2007. 2

JKK+17. Zahra Jafargholi, Chethan Kamath, Karen Klein, Ilan Komargodski, Krzysztof Pietrzak, and Daniel Wichs. Be adaptive, avoid overcommitting. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 133–163. Springer, Cham, August 2017. 3

KPW15. Eike Kiltz, Jiaxin Pan, and Hoeteck Wee. Structure-preserving signatures from standard assumptions, revisited. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 275–295. Springer, Berlin, Heidelberg, August 2015. 47, 48

LCB+25. Yunhao Ling, Jie Chen, Zijian Bao, Man Ho Au, Luping Wang, and Haifeng Qian. Tightly, adaptively secure proxy re-encryption in multi-challenge setting. Springer-Verlag, 2025. 4, 5

LLP20. Youngkyung Lee, Dong Hoon Lee, and Jong Hwan Park. Tightly CCA-secure encryption scheme in a multi-user setting with corruptions. *DCC*, 88(11):2433–2452, 2020. 5, 7

LV08. Benoît Libert and Damien Vergnaud. Unidirectional chosen-ciphertext secure proxy re-encryption. In Ronald Cramer, editor, *PKC 2008*, volume 4939 of *LNCS*, pages 360–379. Springer, Berlin, Heidelberg, March 2008. 2

LW10. Allison B. Lewko and Brent Waters. New techniques for dual system encryption and fully secure HIBE with short ciphertexts. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 455–479. Springer, Berlin, Heidelberg, February 2010. 5

Sah99. Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *40th FOCS*, pages 543–553. IEEE Computer Society Press, October 1999. 40

A Assumptions

We first describe the Matrix Diffie-Hellman assumption [EHK+17].

Definition 7 ((ℓ,k)-Full Rank Matrix Distribution). *Let $(\ell,k)\in\mathbb{N}$ be integers with $\ell>k$. We define the (ℓ,k)-full rank matrix distribution $\mathcal{D}_{\ell,k}$ uniformly outputs $\mathbb{Z}_{\mathsf{p}}^{\ell\times k}$ matrices, where the first k rows forms a full-rank matrix in $\mathbb{Z}_{\mathsf{p}}^{k\times k}$.*

For simplicity of notation, we denote the distribution by $\mathcal{D}_k=\mathcal{D}_{k+1,k}$ when $\ell=k+1$.

For a group $\mathbb{G}$ of prime order p, we give the MDDH assumption over $\mathbb{G}$. Note that in pairing groups $(\mathbb{G}_1,\mathbb{G}_2,\mathbb{G}_\mathsf{T})$, variants of MDDH assumption can be defined over $\mathbb{G}_1,\mathbb{G}_2$ respectively.

Definition 8 ($\mathcal{D}_{\ell,k}$-MDDH Assumption). *The $\mathcal{D}_{\ell,k}$-MDDH assumption holds over $\mathbb{G}$, if for all PPT adversary $\mathcal{A}$, the following advantage $\mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G},\mathcal{A}}(\lambda)$ is negligible.*

$$\mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G},\mathcal{A}}(1^\lambda):=\left|\Pr\left[\mathcal{A}(\left[\mathbf{A}\right]_1,\left[\mathbf{u}\right]_1)=1\right]-\Pr\left[\mathcal{A}(\left[\mathbf{A}\right]_1,\left[\mathbf{Aw}\right]_1)=1\right]\right|.$$

where $\mathbf{A}\leftarrow\mathcal{D}_{\ell,k}$, $\mathbf{w}\leftarrow\mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{k}\right)$, and $\mathbf{u}\leftarrow\mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{\ell}\right)$.

Similarly, we define the $\mathcal{U}_{\ell,k}$-MDDH assumption over $\mathbb{G}$ as same as $\mathcal{D}_{\ell,k}$-MDDH assumption except that $\mathbf{A}\leftarrow\mathcal{U}_{\ell,k}$. Then we give two lemmas which show the relationship between $\mathcal{D}$-MDDH and $\mathcal{U}$-MDDH assumptions.

For an integer$Q\geq 1$, we define the Q-$\mathcal{D}$-MDDH problem over $\mathbb{G}$ as follows.

Definition 9 (Q-$\mathcal{D}_{\ell,k}$-MDDH). *The Q-$\mathcal{D}_{\ell,k}$-MDDH assumption holds over $\mathbb{G}$, if for all PPT adversary $\mathcal{A}$, the following advantage $\mathsf{Adv}^{Q\text{-}\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G},\mathcal{A}}(\lambda)$ is negligible.*

$$\mathsf{Adv}^{Q\text{-}\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G},\mathcal{A}}(\lambda):=\left|\Pr\left[\mathcal{A}(\left[\mathbf{A}\right]_1,\left[\mathbf{AW}\right]_1)=1\right]-\Pr\left[\mathcal{A}(\left[\mathbf{A}\right]_1,\left[\mathbf{U}\right]_1)=1\right]\right|.$$

where $\mathbf{A}\leftarrow\mathcal{D}_{\ell,k}$, $\mathbf{W}\leftarrow\mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{k\times Q}\right)$, and $\mathbf{U}\leftarrow\mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{\ell\times Q}\right)$.

$\mathcal{D}_{\ell,k}$-MDDH problem is random self-reducible [EHK+13], i.e., Q-$\mathcal{D}_{\ell,k}$-MDDH and $\mathcal{D}_{\ell,k}$-MDDH problems can be tightly reduced to each other. Especially, if the distribution is change to $\mathcal{U}_{\ell,k}$, the reduction is even tighter.

Lemma 4 (Random Self-Reducibility of MDDH [EHK+13]). *Let $Q\geq\ell-k$, for any PPT adversary $\mathcal{A}$, there exists a PPT adversary $\mathcal{B}$ such that for all security parameter λ, the following holds.*

$$\mathsf{Adv}^{Q\text{-}\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G},\mathcal{A}}(\lambda)\leq(\ell-k)\cdot\mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G},\mathcal{B}}(\lambda)+\tfrac{1}{p-1}.$$

Let $Q\geq\ell-k$, for any PPT adversary $\mathcal{A}$, there exists a PPT adversary $\mathcal{B}$ such that for all security parameter λ, the following holds.

$$\mathsf{Adv}^{Q\text{-}\mathsf{MDDH}}_{\mathcal{U}_{\ell,k},\mathbb{G},\mathcal{A}}(\lambda)\leq\mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{U}_{\ell,k},\mathbb{G},\mathcal{B}}(\lambda)+\tfrac{1}{p-1}.$$

Definition 10 (Signature Scheme). *A signature scheme* $\Sigma = (\mathsf{KGen}, \mathsf{Sig}, \mathsf{Ver})$ *with the following syntax.*

- $\mathsf{KGen}(1^\lambda) \to (\mathsf{ssk}, \mathsf{svk})$*: The key generation algorithm takes as input a security parameter* λ *and outputs a signing key* ssk *and a verification key* svk*.*
- $\mathsf{Sig}(\mathsf{ssk}, \mathsf{m}) \to \sigma$*: The signing algorithm takes as input a signing key* ssk *and a message* m*, and outputs a signature* σ*.*
- $\mathsf{Ver}(\mathsf{svk}, \mathsf{m}, \sigma) \to \{0,1\}$*: The verification algorithm takes as input a verification key* svk*, a message* m*, and a signature* σ*, and outputs* 1 *if the signature is valid, and* 0 *otherwise.*

We require the following properties:

One-Time Unforgeable: *For any* PPT *adversary* $\mathcal{A}$*, the probability of successfully forging a signature on a new message after seeing one valid signature is negligible. Formally, for any* $\mathcal{A}$*, we have:*

$$\Pr\left[\mathsf{Ver}(\mathsf{svk}, \mathsf{m}', \sigma') = 1 \wedge \mathsf{m}' \neq \mathsf{m} \;\middle|\; \begin{array}{l}(\mathsf{svk}, \mathsf{ssk}) := \mathsf{KeyGen}(1^\lambda),\\ \mathsf{m} := \mathcal{M}, \sigma := \mathsf{Sig}(\mathsf{ssk}, \mathsf{m}),\\ (\mathsf{m}', \sigma') := \mathcal{A}(\mathsf{svk}, \mathsf{m}, \sigma)\end{array}\right] \leq \mathsf{negl}(\lambda).$$

where $\mathcal{M}$ *is the message space and* $\mathsf{negl}(\lambda)$ *denotes a negligible function in* λ*.*

Unforgeability against Chosen Message Attacks: *For any* PPT *adversary* $\mathcal{A}$*, the probability of successfully forging a signature on a new message after querying the signing oracle is negligible. Formally, for any* $\mathcal{A}$*, we have:*

$$\Pr\left[\begin{array}{l}\mathsf{Ver}(\mathsf{svk}, \mathsf{m}', \sigma') = 1\\ \wedge\ \mathsf{m}' \notin \{m_1, \ldots, m_q\}\end{array} \;\middle|\; \begin{array}{l}(\mathsf{svk}, \mathsf{ssk}) := \mathsf{KeyGen}(1^\lambda),\\ (\mathsf{m}_1, \sigma_1), \ldots, (\mathsf{m}_q, \sigma_q)\\ \quad := \mathcal{O}_{\mathsf{Sig}(\mathsf{ssk}, \cdot)}(\mathcal{A}(\mathsf{svk})),\\ (\mathsf{m}', \sigma') := \mathcal{A}^{\mathcal{O}_{\mathsf{Sig}(\mathsf{ssk}, \cdot)}}(\mathsf{svk})\end{array}\right] \leq \mathsf{negl}(\lambda).$$

where $\mathcal{O}_{\mathsf{Sig}}(\mathsf{ssk}, \cdot)$ *is the signing oracle,* $\mathcal{M}$ *is the message space, and* $\mathsf{negl}(\lambda)$ *denotes a negligible function in* λ*.*

Definition 11 (Commitment Scheme). *A commitment scheme* Com *is a tuple* $\mathsf{Com} = (\mathsf{Setup}, \mathsf{Com},)$ *with the following syntax.*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{ck}$*: Takes the security parameter as input, and outputs* ck*.*
- $\mathsf{Com}(\mathsf{ck}, \mathsf{m}; \mathsf{open}) \to \mathsf{c}$*: Takes a commitment key* ck *and a message* m*, and an opening value* open*, and outputs a commitment* c*.*

We require the following properties:

Hiding: *For any security parameter* λ*, any two messages* m_0 *and* m_1*, the commitments to these messages are indistinguishable. Formally, for any* PPT *adversary* $\mathcal{A}$*, we have:*

$$\left|\Pr\left[b = b' \;\middle|\; \begin{array}{l}\mathsf{ck} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)\\ (\mathsf{m}_0, \mathsf{m}_1) \xleftarrow{\$} \mathcal{A}(\mathsf{ck})\\ b \hookleftarrow \{0,1\};\ \mathsf{open} \hookleftarrow \mathsf{U}(\mathcal{O})\\ \mathsf{c} := \mathsf{Com}(\mathsf{ck}, \mathsf{m}_b; \mathsf{open})\\ b' := \mathcal{A}(\mathsf{ck}, \mathsf{c})\end{array}\right] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda).$$

Binding: *For any security parameter* 1^λ*, it is infeasible to find two different messages* m_0 *and* m_1 *and corresponding opening values* open_0 *and* open_1 *such that they produce the same commitment. Formally, for any probabilistic polynomial-time adversary* $\mathcal{A}$*, we have:*

$$\Pr\left[\begin{array}{l|l} \mathsf{m}_0 \neq \mathsf{m}_1 \wedge & \mathsf{ck} \xleftarrow{\$} \mathsf{Setup}(1^\lambda) \\ \mathsf{Commit}(\mathsf{ck}, \mathsf{m}_0; \mathsf{open}_0) & (\mathsf{c}, \mathsf{m}_0, \mathsf{m}_1, \mathsf{open}_0, \mathsf{open}_1) := \mathcal{A}(\mathsf{pp}) \\ = \mathsf{Commit}(\mathsf{ck}, \mathsf{m}_1; \mathsf{open}_1) & \end{array}\right] \leq \mathsf{negl}(\lambda).$$

where $\mathsf{negl}(\lambda)$ *denotes a negligible function in* λ*.*

B Groth-Sahai Non-interactive Zero-Knowledge Proofs

Groth and Sahai [GS08] introduced a unified methodology for constructing non-interactive witness-indistinguishable (NIWI) and non-interactive zero-knowledge (NIZK) proofs over bilinear groups $\mathcal{PG}$, capable of proving satisfiability for systems of mixed-type equations. The equations could be written in a unified way as:

$$\sum_{j=1}^{n} f(a_j, y_j) + \sum_{i=1}^{m} f(x_i, b_i) + \sum_{i=1}^{m}\sum_{j=1}^{n} f(x_i, \gamma_{ij} y_j) = t$$

where A_1,A_2,A_T are $\mathbb{Z}_\mathsf{p}$-modules, $\mathbf{x} \in A_1^m$, $\mathbf{y} \in A_2^n$ are the variables, $\mathbf{a} \in A_1^n$, $\mathbf{b} \in A_2^m$, $\Gamma = (\gamma_{ij}) \in \mathbb{Z}_\mathsf{p}^{m \times n}$, $t \in A_T$ are the constants and $f : A_1 \times A_2 \to A_T$ is bilinear map.

In order to simplify notation, we define:

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} f(x_i, y_i).$$

The equations can now be written as:

$$\mathbf{a} \cdot \mathbf{y} + \mathbf{x} \cdot \mathbf{b} + \mathbf{x} \cdot \Gamma \mathbf{y} = t.$$

Because A_1, A_2, A_T are $\mathbb{Z}_\mathsf{p}$-modules, equations are of one of these types:

- Pairing product equations: with $A_1 = \mathbb{G}_1$,$A_2 = \mathbb{G}_2$,$A_T = \mathbb{G}_\mathsf{T}$, $f(x, y) = x \bullet y$, the equation is $(\mathcal{A} \cdot \mathcal{Y})(\mathcal{X} \cdot \mathcal{B})(\mathcal{X} \cdot \Gamma\mathcal{Y}) = t_T$.
- Multi-scalar multiplication in $\mathbb{G}_1$: with $A_1 = \mathbb{G}_1$,$A_2 = \mathbb{Z}_\mathsf{p}$,$A_T = \mathbb{G}_1$, $f(\mathcal{X}, y) = y\mathcal{X}$, the equation is $\mathcal{A} \cdot \mathbf{y} + \mathcal{X} \cdot \mathbf{b} + \mathcal{X} \cdot \Gamma\mathbf{y} = \mathcal{T}_1$.
- Multi-scalar multiplication in $\mathbb{G}_2$: with $A_1 = \mathbb{Z}_\mathsf{p}$,$A_2 = \mathbb{G}_2$,$A_T = \mathbb{G}_2$, $f(x, \mathcal{Y}) = x\mathcal{Y}$, the equation is $\mathbf{a} \cdot \mathcal{Y} + \mathbf{x} \cdot \mathcal{B} + \mathbf{x} \cdot \mathcal{Y}\Gamma = \mathcal{T}_2$.
- Quadratic equation in $\mathbb{Z}_\mathsf{p}$: with $A_1 = \mathbb{Z}_\mathsf{p}$,$A_2 = \mathbb{Z}_\mathsf{p}$,$A_T = \mathbb{Z}_\mathsf{p}$, $f(x, y) = xy \bmod p$, the equation is $\mathbf{a} \cdot \mathbf{y} + \mathbf{x} \cdot \mathbf{b} + \mathbf{x} \cdot \Gamma\mathbf{y} = t$.

In summary, the GS proof system allows constructing NIWI and NIZK proofs for satisfiability of a set of equations of the type above, which means that there is a choice of variables (witness) satisfying all equations simultaneously.

To construct the proof systems, Groth and Sahai gave a commitment to each element of the witness and "plug in" the commitments in place of the variables since the commitments themselves also "behave" like the values being committed to [GS08]. With some additional information, i.e., the proof and the properties of the commitment, they construct the proof system successfully. We then give the construction of the GS proof system used in this paper.

Commitments. The commitment scheme uses public parameters consisting of a matrix $\mathbf{D} \hookleftarrow \mathcal{D}_k$ and vectors $\mathbf{z}, \mathbf{t} \in \mathbb{Z}_{\mathsf{p}}^{k+1}$. The commitment key is set as $\mathbf{U} := [\mathbf{D} \mid \mathbf{z}]$. We consider two parameter regimes:

- **Perfectly Binding Mode:** For scalar vectors: $\mathbf{z} \notin \mathsf{Span}(\mathbf{D})$. For group vectors: $\mathbf{z} \in \mathsf{Span}(\mathbf{D})$ and $\mathbf{t} \notin \mathsf{Span}(\mathbf{D})$.
- **Perfectly Hiding Mode:** For scalar vectors: $\mathbf{z} \in \mathsf{Span}(\mathbf{D})$. For group vectors: $\mathbf{z} \notin \mathsf{Span}(\mathbf{D})$ and $\mathbf{t} \in \mathsf{Span}(\mathbf{D})$.

The two parameter regimes are computationally indistinguishable based on the $\mathcal{D}_k$-MDDH assumption.

Scalar Commitments: To commit to a scalar vector $\mathbf{w} \in \mathbb{Z}_{\mathsf{p}}^n$ with randomness $\mathbf{R} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{k \times n}\right)$, we compute:

$$\left[\mathbf{C}_{\mathbf{w}}\right] := \left[\mathbf{D}\right] \mathbf{R} + \left[\mathbf{z}\right] \mathbf{w}^{\intercal}$$

Group Element Commitments: To commit to a group vector $\left[\mathbf{w}\right] \in \mathbb{G}^n$ with randomness $\mathbf{R} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{(k+1) \times n}\right)$, we compute:

$$\left[\mathbf{C}_{\mathbf{w}}\right] := \left[\mathbf{U}\right] \mathbf{R} + \mathbf{t} \left[\mathbf{w}^{\intercal}\right]$$

Dual-Mode Properties and Mathematical Foundation: The commitment scheme exhibits dual-mode behavior essential for zero-knowledge proofs and simulation soundness. In the perfectly binding mode, commitments uniquely determine the committed values, ensuring soundness by preventing adversaries from generating false proofs. In the perfectly hiding mode, commitments statistically hide the committed values, enabling zero-knowledge simulation where the simulator can create indistinguishable fake proofs without knowing the witness.

The mathematical foundation relies on the linear algebraic structure of the commitment scheme. For **scalar commitments**, in the binding mode when $\mathbf{z} \notin \mathsf{Span}(\mathbf{D})$, the commitment matrix $[\mathbf{D} \mid \mathbf{z}]$ has full rank, making the commitment function injective—each commitment value corresponds to a unique message. In the hiding mode when $\mathbf{z} \in \mathsf{Span}(\mathbf{D})$, there exists a vector $\mathbf{r}$ such that $\mathbf{z} = \mathbf{D}\mathbf{r}$, allowing the commitment $\left[\mathbf{D}\right] \mathbf{R} + \left[\mathbf{z}\right] \mathbf{w}^{\intercal} = \left[\mathbf{D}\right] (\mathbf{R} + \mathbf{r}\mathbf{w}^{\intercal})$ to be perfectly randomized by choosing appropriate $\mathbf{R}$, thus statistically hiding the message $\mathbf{w}$.

For **group element commitments**, the dual-mode property operates through a complementary mechanism: in the binding mode, we have $\mathbf{z} \in \mathsf{Span}(\mathbf{D})$ and $\mathbf{t} \notin \mathsf{Span}(\mathbf{D})$, ensuring that the commitment $\left[\mathbf{U}\right] \mathbf{R} + \mathbf{t} \left[\mathbf{w}^{\intercal}\right]$ uniquely determines $\left[\mathbf{w}\right]$ since $\mathbf{t}$ provides the necessary linear independence. In the hiding

mode, $\mathbf{z} \notin \mathsf{Span}(\mathbf{D})$ and $\mathbf{t} \in \mathsf{Span}(\mathbf{D})$, where $\mathbf{t} = \mathbf{D}\mathbf{s}$ for some $\mathbf{s}$, allowing the commitment to be rewritten as $\left[\mathbf{U}\right]\mathbf{R} + \mathbf{D}\mathbf{s}\ \left[\mathbf{w}^\intercal\right] = \left[\mathbf{U}\right](\mathbf{R} + [\mathbf{I}_k \mid \mathbf{0}]\mathbf{s}\ \left[\mathbf{w}^\intercal\right])$, which can be perfectly randomized.

The $\mathcal{D}_k$-MDDH assumption ensures that distinguishing between the parameter regimes (whether vectors are in the span or not) is computationally infeasible, enabling the seamless transition between binding and hiding properties and allowing the proof system to achieve both soundness and zero-knowledge simultaneously through a standard hybrid argument.

Notation: For the proof system, we use the commitment schemes defined above. To commit to scalar witnesses $\mathbf{x} \in \mathbb{Z}_\mathsf{p}^m$, we compute $\mathbf{c} = \left[\mathbf{D}\right]\mathbf{R} + \left[\mathbf{z}\right]\mathbf{x}^\intercal$ with randomness $\mathbf{R} \leftarrow\!\!\shortmid \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k\times m}\right)$. To commit to group witnesses $\left[\mathbf{y}\right] \in \mathbb{G}^n$, we compute $\mathbf{d} = \left[\mathbf{U}\right]\mathbf{S} + \mathbf{t}\ \left[\mathbf{y}^\intercal\right]$ with randomness $\mathbf{S} \leftarrow\!\!\shortmid \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times n}\right)$.

We define different types of bilinear operations depending on the input types:

- **Scalar-Commitment Operation:** For scalar $\mathbf{a} \in \mathbb{Z}_\mathsf{p}^n$ and commitment $\mathbf{d} \in \mathbb{G}^{k+1}$:
$$\mathbf{a} \bullet \mathbf{d} = \sum_{i=1}^{n} a_i \cdot d_i$$
where $\cdot$ denotes scalar multiplication in the group.
- **Commitment-Group Operation:** For commitment $\mathbf{c} \in \mathbb{G}^{k+1}$ and group element $\left[\mathbf{b}\right] \in \mathbb{G}^m$:
$$\mathbf{c} \bullet \left[\mathbf{b}\right] = \sum_{i=1}^{k+1}\sum_{j=1}^{m} c_i \bullet b_j$$
where $\bullet$ denotes the bilinear pairing.
- **Commitment-Commitment Operation:** For commitments $\mathbf{c}, \mathbf{d} \in \mathbb{G}^{k+1}$:
$$\mathbf{c} \bullet \mathbf{d} = \sum_{i=1}^{k+1}\sum_{j=1}^{k+1} c_i \bullet d_j$$
where $\bullet$ denotes the bilinear pairing.

Proof To prove the equation $\mathbf{a}\cdot\left[\mathbf{y}\right] + \mathbf{x}\cdot\left[\mathbf{b}\right] + \mathbf{x}\cdot\Gamma\left[\mathbf{y}\right] = \left[t\right]$, the prover proceeds as follows:

1. Commit to the witnesses: Compute commitments $\mathbf{c} = \left[\mathbf{D}\right]\mathbf{R} + \left[\mathbf{z}\right]\mathbf{x}^\intercal$ and $\mathbf{d} = \left[\mathbf{U}\right]\mathbf{S} + \mathbf{t}\ \left[\mathbf{y}^\intercal\right]$ using randomness $\mathbf{R} \leftarrow\!\!\shortmid \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k\times m}\right)$ and $\mathbf{S} \leftarrow\!\!\shortmid \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times n}\right)$.
2. Generate proof elements: Sample $\mathbf{T} \leftarrow\!\!\shortmid \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k\times(k+1)}\right)$ and compute:
$$\pi := \mathbf{R}^\intercal\left[\mathbf{b}\right] + \mathbf{R}^\intercal\Gamma(\left[\mathbf{U}\right]\mathbf{S} + \mathbf{t}\ \left[\mathbf{y}^\intercal\right]) - \mathbf{T}\left[\mathbf{z}\right]$$
$$\theta := \mathbf{S}^\intercal\mathbf{a} + \mathbf{S}^\intercal(\Gamma^\intercal\mathbf{x}) + \mathbf{T}^\intercal\mathbf{t}$$
where the operations are performed component-wise for group elements.
3. Return the proof $(\mathbf{c}, \mathbf{d}, \pi, \theta)$.

Verification Given a proof $(\mathbf{c}, \mathbf{d}, \pi, \theta)$ for the equation $\mathbf{a}\cdot\left[\mathbf{y}\right]+\mathbf{x}\cdot\left[\mathbf{b}\right]+\mathbf{x}\cdot\Gamma\left[\mathbf{y}\right] = \left[t\right]$, the verifier checks whether the following equation holds in the target group $\mathbb{G}_{\mathsf{T}}$:

$$\mathbf{a}\bullet\mathbf{d}+\mathbf{c}\bullet\left[\mathbf{b}\right]+\mathbf{c}\bullet(\Gamma\mathbf{d}) = e(\left[1\right],\left[t\right])+\left[\mathbf{z}\right]\bullet\pi+\theta\bullet\mathbf{t}$$

where $e(\left[1\right],\left[t\right])$ represents the pairing of the generator with $\left[t\right]$. The verification accepts if and only if the above equation holds.

C Security Game Descriptions for aHRA-secure PRE

This appendix provides detailed descriptions of the hybrid security games used in the proofs of Theorem 1 and Theorem 2. These games formalize the sequence of indistinguishable modifications applied to the security experiment to establish tight adaptive security for our proxy re-encryption schemes.

C.1 Hybrid Games for Single-Challenge aHRA Security

The following figure 7 presents the complete description of hybrid games $\mathbf{G}_0$ through $\mathbf{G}_5$ for the single-challenge aHRA security proof. These games progressively transform the original security experiment into a game where the adversary has no advantage.

$\underline{\mathsf{Exp}^{\mathsf{SC\text{-}aHRA}}_{\mathsf{PRE},\mathcal{A}}(1^\lambda)}$:
01 $v \hookleftarrow \mathsf{U}(\mathbb{F})$ // $\mathbf{G}_5$
02 $\mathsf{pp} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$
03 $b \hookleftarrow \mathsf{U}(\{0,1\})$
04 $\mathcal{O}_{\mathsf{aHra}} := (\mathcal{O}_{\mathsf{KGen}}, \mathcal{O}_{\mathsf{CorrT}}, \mathcal{O}_{\mathsf{CorrK}}, \mathcal{O}_{\mathsf{Enc}}, \mathcal{O}_{\mathsf{Chall,b}}, \mathcal{O}_{\mathsf{ReEnc}})$
05 **if** $\mathsf{Bad}_{\mathsf{Col}}$ **then** // $\mathbf{G}_1$-$\mathbf{G}_5$
06 **abort** // $\mathbf{G}_1$-$\mathbf{G}_5$
07 $b' \xleftarrow{\$} \mathcal{A}_1^{\mathcal{O}_{\mathsf{aHra}}}(\mathsf{pp})$
08 **return** $[\![b = b']\!] \wedge$ $\mathsf{NonTrivial}(\mathcal{L}_{\mathsf{CorrT}}, \mathcal{L}_{\mathsf{CorrK}}, , \mathcal{L}_{\mathsf{C}})$

$\underline{\textbf{Alg } \mathcal{O}_{\mathsf{Chall,b}}(i, \mathsf{m}_0, \mathsf{m}_1)}$:
09 $(\mathsf{ct}_0, \mathsf{w}) \xleftarrow{\$} \mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk})$ // $\mathbf{G}_0$-$\mathbf{G}_2$
10 $\mathsf{ct}_0 \xleftarrow{\$} \mathsf{Sample}_{\mathcal{X}}(1^\lambda)$ // $\mathbf{G}_3$-$\mathbf{G}_5$
11 $\mathsf{ct}_1 := \mathsf{PEval}(\mathsf{projk}_i, \mathsf{ct}_0, \mathsf{w}) + \mathsf{m}_b$ // $\mathbf{G}_0$-$\mathbf{G}_1$
12 $\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}_i, \mathsf{ct}_0) + \mathsf{m}_b$ // $\mathbf{G}_2$-$\mathbf{G}_3$
13 **if** $\mathsf{Bad}_{\mathsf{Univ}}$ **then abort** // $\mathbf{G}_4$-$\mathbf{G}_5$
14 $\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}_i, \mathsf{ct}_0) + \mathsf{m}_b + v \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0)$ // $\mathbf{G}_5$
15 $\mathsf{ct} := (\mathsf{ct}_0, \mathsf{ct}_1)$
16 $\mathcal{L}_{\mathsf{C}} := \mathcal{L}_{\mathsf{C}} \cup \{(\mathsf{ct}, i)\}$
17 **return** ct

Fig. 7. Hybrid games for SC-aHRA security. We denote by $\mathsf{h} \in \mathcal{HK}$ the base vector for kernel space corresponds to the dimension k linear language with support in $\mathbb{F}^\ell$. Note that such vector can be generated together with the linear language.

C.2 Hybrid Games for Selective Adversary

The following figure 8 describes the simulation strategy for hybrid games $\mathbf{G}_{4.1}$ through $\mathbf{G}_{4.3}$ in the selective adversary setting. This simulation is crucial for establishing the selective-to-adaptive security reduction via the core lemma (Lemma 3).

Oracle $\mathcal{O}_{\mathsf{KGen}}()$:
01 $(\mathsf{lpk}, \mathsf{td}) \xleftarrow{\$} \mathsf{SMP.Setup}(1^\lambda)$
02 $\mathsf{hk}_1 \xleftarrow{\$} \mathsf{VSHPS.Setup}(1^\lambda)$
03 $\mathsf{hk}'_1 \xleftarrow{\$} \mathsf{VSHPS.Setup}(1^\lambda)$ // $\mathbf{G}_{4.2-4.3}$
04 $v \hookleftarrow \mathsf{U}(\mathbb{F})$ // $\mathbf{G}_{4.2-4.3}$
05 $\mathsf{hk}_1 := \mathsf{hk}'_1 + v \cdot \mathsf{h}$ // $\mathbf{G}_{4.2-4.3}$
06 $\mathsf{projk}_1 \xleftarrow{\$} \mathsf{PKGen}(\mathsf{hk}_1, \mathcal{L}_{\mathsf{lpk}})$ // $\mathbf{G}_{4.1-4.2}$
07 $\mathsf{projk}_1 \xleftarrow{\$} \mathsf{PKGen}(\mathsf{hk}'_1, \mathcal{L}_{\mathsf{lpk}})$ // $\mathbf{G}_{4.3}$
08 **for** $i \in \{2, \ldots, |\mathsf{V_{CL}}|\}$:
09 $\quad \mathsf{d}_i \hookleftarrow \mathsf{U}(\mathcal{HK})$
10 $\quad \mathsf{projk}_i := \mathsf{projk}_1 + \mathsf{PKGen}(\mathsf{d}_i, \mathcal{L}_{\mathsf{lpk}})$
11 $\mathsf{d}_1 = 0$
12 **for** $i \in \{|\mathsf{V_{CL}}| + 1, \ldots, \mathsf{N_K}\}$:
13 $\quad \mathsf{hk}_i \xleftarrow{\$} \mathsf{VSHPS.Setup}(1^\lambda)$
14 $\quad \mathsf{projk}_i := \mathsf{PKGen}(\mathsf{hk}_i, \mathcal{L}_{\mathsf{lpk}})$

Oracle $\mathcal{O}_{\mathsf{CorrT}}(i, j)$:
15 **if** $i, j \in \{1, \ldots, |\mathsf{V_{CL}}|\}$:
16 $\quad$ **return** $\mathsf{d}_{i,j} := \mathsf{d}_j - \mathsf{d}_i$
17 **else**
18 $\quad$ **return** $\mathsf{d}_{i,j} := \mathsf{hk}_j - \mathsf{hk}_i$

Oracle $\mathcal{O}_{\mathsf{CorrK}}(i)$:
19 **if** $i \in \{|\mathsf{V_H}| + 1, \ldots, \mathsf{N_K}\}$:
20 $\quad$ **return** hk_i
21 **return** $\perp$

Fig. 8. Description of the hybrid game $\mathbf{G}_{4.1}$ for a selective adversary. The simulation of the oracles is based on the adversary's queries ($\mathcal{L}_{\mathsf{CorrT}}$, $\mathcal{L}_{\mathsf{CorrK}}$, $\mathcal{L}_{\mathsf{HC}}$, and $\mathcal{L}_{\mathsf{C}}$), which are known before the game begins. Without loss of generality, we re-index keys such that challenge queries in $\mathcal{L}_{\mathsf{C}}$ correspond to the first $|\mathsf{V_{CL}}|$ users. The key generation oracle $\mathcal{O}_{\mathsf{KGen}}$ is executed only once at the start.

C.3 Hybrid Games for Multi-Challenge aHRA Security

The following figure 9 presents the hybrid games $\mathbf{G}_5$ through $\mathbf{G}_9$ for the multi-challenge aHRA security proof. These games extend the single-challenge construction to handle multiple challenge queries while maintaining tight security.

D Language-Malleable Zero-Knowledge Proof with Special Simulation Soundness

We adapt the notion of simulation soundness from Sahai [Sah99] to the tag-based setting. Our definition differs from the existing notions of weak and strong unbounded simulation soundness from [AJO+19]. As noted in [AJO+19], weak unbounded simulation soundness is insufficient for building CCA-secure protocols. This is because it allows an adversary to generate a new proof for the same statement contained in a challenge ciphertext, and then submit this modified ciphertext to a decryption oracle.

Conversely, the stronger notion of unbounded simulation soundness is overly restrictive for our application. This notion forbids an adversary from creating a valid proof for any false statement, thereby precluding any form of proof malleability. Such a strict requirement is incompatible with proxy re-encryption, where the core functionality relies on the ability to transform a proof for an original ciphertext into a new, valid proof for a re-encrypted ciphertext.

To bridge this gap, we introduce the notion of *special simulation soundness*. This notion permits a controlled form of malleability, allowing an adversary to

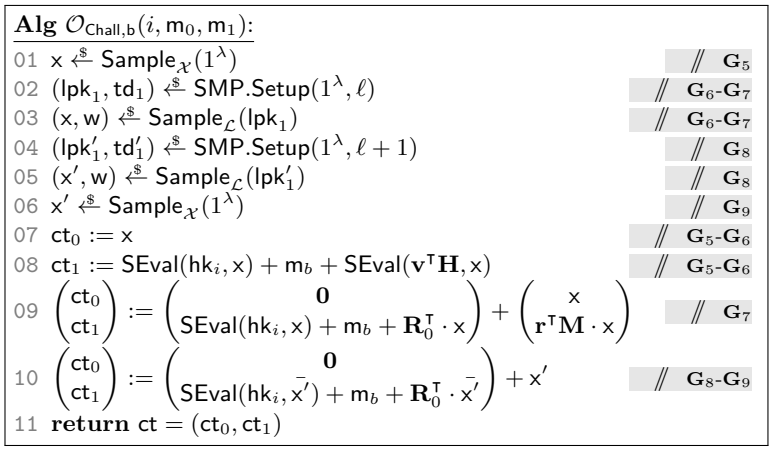

Fig. 9. Hybrid games for aHRA security from $\mathbf{G}_5$ onwards. Let lpk_1 define a linear language of dimension k over an ambient space of dimension ℓ. The original language lpk_0 has dimension k over an ambient space of dimension ℓ.

transform a simulated proof for a given statement into a new, valid proof for a related, but otherwise false, statement. This controlled malleability is formalized through the concept of a trapdoor language distribution, which we define below.

Definition 12 (Membership-Preserving Language Family).

A membership-preserving language family MPL *with respect to a map family* $\mathcal{H} = \{\mathsf{H}_i\} = \{(\mathsf{H}_i^{\mathsf{lpk}}, \mathsf{H}_i^{\mathsf{x}}, \mathsf{H}_i^{\mathsf{w}})\}$ *consists of three* PPT *algorithms* $\mathsf{MPL} = (\mathsf{Setup}, \mathsf{Samp}, \mathsf{Test})$ *with the following syntax:*

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{lpk}, \mathsf{td})$*: Takes a security parameter* λ *as input, and outputs a language public key* lpk *and trapdoor* td*.*
- $\mathsf{Samp}(\mathsf{lpk}) \to (\mathsf{x}, \mathsf{w})$*: Takes a public key* lpk *as input, and outputs a pair* (x, w) *such that* $\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}}$ *and* $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\mathsf{lpk}}$*.*
- $\mathsf{Test}(\mathsf{lpk}, \mathsf{x}, \mathsf{w}, \mathsf{td}) \to \{0, 1\}$*: Takes a public key* lpk*, a statement* x*, a witness* w*, and a trapdoor* td *as input, and outputs a bit indicating whether* $\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}}$*.*

We require the following properties.

Correctness. *For any security parameter* λ*, we have*

$$\Pr\left[\mathsf{Test}(\mathsf{lpk}, \mathsf{x}, \mathsf{w}, \mathsf{td}) = 1 \;\middle|\; \begin{array}{l} (\mathsf{lpk}, \mathsf{td}) \xleftarrow{\$} \mathsf{Setup}(1^\lambda) \\ (\mathsf{x}, \mathsf{w}) \xleftarrow{\$} \mathsf{Samp}(\mathsf{lpk}) \end{array}\right] = 1.$$

Membership-Preserving. *We say that* MPL *is membership-preserving with respect to a function family* $\mathcal{H} = \{\mathsf{H}_i\} = \{(\mathsf{H}_i^{\mathsf{lpk}}, \mathsf{H}_i^{\mathsf{x}}, \mathsf{H}_i^{\mathsf{w}})\}$*, if for any security parameter* λ*, any public key* lpk*, any statement* x*, let* $\mathsf{lpk}' := \mathsf{H}^{\mathsf{lpk}}(\mathsf{lpk})$*,* $\mathsf{x}' = \mathsf{H}^{\mathsf{x}}(\mathsf{x})$ *and* $\mathsf{w}' = \mathsf{H}^{\mathsf{w}}(\mathsf{w})$*, we have*

$$\mathsf{x} \in_{\mathsf{w}} \mathcal{L}_{\mathsf{lpk}} \iff \mathsf{x}' \in_{\mathsf{w}'} \mathcal{L}_{\mathsf{lpk}'}.$$

To formalize the notion of special simulation soundness, we introduce a symmetric link function, $\mathsf{link} : (\mathcal{X}, \mathcal{L})^2 \to \{0,1\}$, which specifies when two statement-language pairs are considered linked. This function is designed to capture the controlled malleability of the proof system. In the context of our proxy re-encryption scheme, for instance, two statements (i.e., ciphertexts) are linked if one can be transformed into the other via re-encryption. Consequently, the special simulation soundness property must permit this form of malleability. That is, the production of a new proof for a statement-language pair linked to a previously simulated one does not constitute a break of soundness. Taking a MPL as an example, we have

$$\begin{aligned}\mathsf{link}(\mathsf{x}, \mathsf{lpk}, \mathsf{x}', \mathsf{lpk}') = 1 \implies &(\mathsf{x}' = \mathsf{H}^{\mathsf{x}}(\mathsf{x}) \wedge \mathsf{lpk}' = \mathsf{H}^{\mathsf{lpk}}(\mathsf{lpk})) \\ &\vee (\mathsf{x} = \mathsf{H}^{\mathsf{x}}(\mathsf{x}') \wedge \mathsf{lpk} = \mathsf{H}^{\mathsf{lpk}}(\mathsf{lpk}')).\end{aligned}$$

Definition 13 (Language Malleable Tag-based NIZK). *A language-malleable tag-based* NIZK *proof system for a membership-preserving language family* MPL *with respect to a function family* $\mathcal{H} = \{\mathsf{H}_i\} = \{(\mathsf{H}_i^{\mathsf{lpk}}, \mathsf{H}_i^{\mathsf{x}}, \mathsf{H}_i^{\mathsf{w}})\}$[20] *consists of four* PPT *algorithms* $\boldsymbol{\Pi} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Ver}, \mathsf{Trans})$ *with the following syntax:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{crs}$*: Takes a security parameter* λ *as input, and outputs a common reference string* crs*.*
- $\mathsf{Prove}(\mathsf{crs}, \mathsf{lpk}, \mathsf{x}, \mathsf{w}) \to (\mathsf{t}, \pi)$*: Takes a* crs*, a language public key* lpk*, a statement* x*, and a witness* w *for the language* $\mathcal{L}_{\mathsf{lpk}}$ *as input, and outputs a tag* t *and a proof* π*.*
- $\mathsf{Ver}(\mathsf{crs}, \mathsf{lpk}, \mathsf{t}, \mathsf{x}, \pi) \to \{0,1\}$*: Takes a* crs*, a language public key* lpk*, a tag* t*, a statement* x*, and a proof* π *as input, and outputs a bit.*
- $\mathsf{Trans}(\mathsf{crs}, \mathsf{lpk}, \mathsf{t}, \mathsf{x}, \pi, \mathsf{H}^{\mathsf{lpk}}) \to (\mathsf{lpk}', \mathsf{x}', \pi')$*: Takes a* crs*, a language public key* lpk*, a tag* t*, a statement-proof pair* (x, π) *for the language* $\mathcal{L}_{\mathsf{lpk}}$*, and a key homomorphism* $\mathsf{H}^{\mathsf{lpk}}$ *as input, and outputs a language public key* lpk' *and a new statement-proof pair* (x', π') *for the language* $\mathcal{L}_{\mathsf{lpk}'} := \mathsf{H}^{\mathsf{lpk}}(\mathcal{L}_{\mathsf{lpk}})$ *with the same tag* t*.*

We require the following properties.

Correctness. *For any security parameter* λ*, any* $i \in [\mathsf{N}_\mathsf{U}]$*, and any statement-witness pair* $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$*, we have:*

$$\Pr\left[\mathsf{Ver}(\mathsf{crs}, \mathcal{L}_{\mathsf{lpk}}, \mathsf{t}, \mathsf{x}, \pi) = 1 \;\middle|\; \begin{array}{l}\mathsf{crs} \xleftarrow{\$} \mathsf{Setup}(1^\lambda) \\ (\mathsf{t}, \pi) \xleftarrow{\$} \mathsf{Prove}(\mathsf{crs}, \mathcal{L}_{\mathsf{lpk}}, \mathsf{x}, \mathsf{w})\end{array}\right] = 1.$$

Zero-Knowledge. *There exists a* PPT *simulator* $\mathcal{S} = (\mathsf{SSetup}, \mathsf{Sim})$ *such that for any* PPT *adversary* $\mathcal{A}$*, the following advantage is negligible,*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{ZK}}_{\boldsymbol{\Pi},\mathcal{A}}(1^\lambda) := \Big|&\Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Prove}}(\mathsf{crs},\mathsf{lpk},\cdot)}(\mathsf{crs}) = 1\right] \\ &- \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Sim}}(\mathsf{crs},\mathsf{lpk},\mathsf{simK},\cdot)}(\mathsf{crs}) = 1\right]\Big|,\end{aligned}$$

[20] We assume that the identity function is included in $\mathcal{H}$.

where $(\mathsf{crs}, \mathsf{simK}) \xleftarrow{\$} \mathsf{SSetup}(1^\lambda)$ *in the second probability, and* $\mathsf{crs} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$ *in the first. The oracles only answer queries for statements* $\mathsf{x} \in \mathcal{L}_{\mathsf{lpk}}$*. Note that* Prove_i *and* Sim_i *can detect invalid statements using* td *for the trapdoor language* $\mathcal{L}_{\mathsf{lpk}}$*.*

Language Malleability. *We require that the proof transformation algorithm can efficiently transform a valid proof* π *for statement* x *in* $\mathcal{L}_{\mathsf{lpk}}$ *into a valid proof* π' *for a new statement* x' *in* $\mathcal{L}_{\mathsf{lpk}'}$ *with* $\mathsf{lpk}' := \mathsf{H}^{\mathsf{lpk}}(\mathsf{lpk})$ *with* $(\mathsf{H}^{\mathsf{lpk}}, \cdot, \cdot) \in \mathcal{H}$*. Formally, for any* $(\mathsf{x}, \mathsf{w}) \in \mathcal{L}_{\mathsf{lpk}}$*, for any* $\mathsf{H}^{\mathsf{lpk}}$ *such that* $(\mathsf{H}^{\mathsf{lpk}}, \cdot, \cdot) \in \mathcal{H}$*, we have,*

$$\Pr\left[\mathsf{Ver}(\mathsf{crs}, \mathsf{lpk}', \mathsf{t}, \mathsf{x}', \pi') = 1 \;\middle|\; \begin{array}{l} \mathsf{crs} \xleftarrow{\$} \mathsf{Setup}(1^\lambda) \\ (\mathsf{t}, \pi) \xleftarrow{\$} \mathsf{Prove}(\mathsf{crs}, \mathsf{lpk}, \mathsf{x}, \mathsf{w}) \\ \mathsf{lpk}' := \mathsf{H}^{\mathsf{lpk}}(\mathsf{lpk}) \\ (\mathsf{x}', \pi') \xleftarrow{\$} \mathsf{Trans}(\mathsf{crs}, \mathsf{lpk}, \mathsf{t}, \mathsf{x}, \pi, \mathsf{H}^{\mathsf{lpk}}) \end{array}\right] = 1.$$

Special Simulation Soundness. *For any* PPT *adversary* $\mathcal{A}$*, there exists a* PPT *simulator* $\mathcal{S} = (\mathsf{SSetup}, \mathsf{Sim})$ *such that the following probability is negligible.*

$$\Pr\left[\begin{array}{c} \mathsf{Ver}(\mathsf{crs}, \mathsf{lpk}^\star, \mathsf{t}^\star, \mathsf{x}^\star, \pi^\star) = 1 \\ \wedge \mathsf{x}^\star \notin \mathcal{L}_{\mathsf{lpk}^\star} \\ \wedge \nexists(\mathsf{x}, \mathsf{lpk}) \in \mathcal{T} : \mathsf{link}(\mathsf{x}, \mathsf{lpk}, \mathsf{x}^\star, \mathsf{lpk}^\star) = 1 \end{array} \;\middle|\; \begin{array}{l} (\mathsf{crs}, \mathsf{simK}) \xleftarrow{\$} \mathsf{SSetup}(1^\lambda) \\ (\mathsf{lpk}^\star, \mathsf{x}^\star, \mathsf{t}^\star, \pi^\star) \\ \quad \xleftarrow{\$} \mathcal{A}^{\mathsf{Sim}(\mathsf{crs}, \mathsf{simK}, \cdot)}(\mathsf{crs}) \end{array}\right],$$

where $\mathcal{T}$ *is the set of tags and language public keys generated via the simulation oracle queries* $\mathcal{A}$*, and* $\mathsf{link}(\mathsf{x}_1, \mathsf{lpk}_1, \mathsf{x}_2, \mathsf{lpk}_2)$ *is the predicate defining whether two statements are linked via the function family* $\mathcal{H}$*.*

Our notion of special simulation soundness permits a controlled form of malleability. Specifically, the proof transformation function, Trans, allows for the derivation of a new, valid proof for a statement that is legitimately related to the original one (e.g., a re-encrypted ciphertext in a PRE scheme), even when starting from a simulated proof. This property positions our notion between existing definitions: it is stronger than weak simulation soundness, which allows arbitrary proof modifications for a given tag, but weaker than strong simulation soundness, which prohibits any malleation of proofs for false statements, regardless of the tag.

Relation to other simulation soundness notions The literature presents two distinct definitions of simulation soundness for tag-based NIZK proof systems. The weak simulation soundness defined in [AJO+19] does not prevent the adversary from generating a new proof for false statements with the same tag used in the simulated proof but prohibits generating a valid proof for any false statement with other tags. Conversely, the strong simulation soundness in [AJO+19] prevents the adversary from generating any new valid proof for false statements. Our notion of special simulation soundness lies between these two definitions. It is stronger than weak simulation soundness because the adversary cannot generate a valid proof for any false statement with tags different from the one in the simulated

proof. However, it is weaker than strong simulation soundness, as it allows the adversary to generate a new valid proof for some false statements with the same tag as the simulated proof. This relaxation is essential in the proxy re-encryption scheme, where the adversary can generate a new valid proof for re-encrypted ciphertexts with known tokens.

D.1 Generic Construction of Special Simulation Sound tLM-NIZK

We give here the language family that we will use in the aCCA proxy re-encryption scheme. In our case, a language public key consists of a matrix $\left[\mathbf{A}_i\right]_1$, we need to prove the linear span language $\mathcal{L}_{\mathbf{A}_i}$ defined as follows:

$$\mathcal{L}_{\mathbf{A}_i} := \{\mathsf{x} = \left[\mathbf{u}\right]_1 \in \mathbb{G}_1^{\ell+1} \mid \exists \mathbf{w} \in \mathbb{Z}_\mathsf{p}^k : \mathsf{x} = \left[\mathbf{A}_i\right]_1 \mathbf{w}\}.$$

We define an extended language $\mathcal{L}_{\mathbf{A}_i}^{\mathsf{svk},\mathsf{ck}}$, we will show that a proof of $\mathcal{L}_{\mathbf{A}_i}^{\mathsf{svk},\mathsf{ck}}$ can be seen as a special simulation-sound tLM-NIZK proof of the language $\mathcal{L}_{\mathbf{A}_i}$. Moreover, if the proof of $\mathcal{L}_{\mathbf{A}_i}^{\mathsf{svk},\mathsf{ck}}$ has language malleability over the choice of $\mathbf{A}_i$, then the corresponding tLM-NIZK proof of $\mathcal{L}_{\mathbf{A}_i}$ also has language malleability over the choice of $\mathbf{A}_i$. The extended language is defined as follows:

$$\mathcal{L}_{\mathbf{A}_i}^{(\mathsf{svk},\mathsf{ck})} := \left\{ (\mathsf{x}, \mathbf{C_s}) \middle| \begin{array}{l} \exists \mathsf{w} \in \mathbb{Z}_\mathsf{p}^k : \mathsf{x} = \left[\mathbf{A}_i\right]_1 \mathsf{w} \\ \\ \vee \exists \mathbf{A}_j, \mathsf{x}', \sigma, \mathsf{open} : \left\{ \begin{array}{l} \mathbf{C_s} = \mathsf{Com.Commit}(\mathsf{ck}, \sigma; \mathsf{open}) \\ \wedge \mathsf{Ver}(\mathsf{svk}, \mathsf{H}(\mathbf{A}_j, \mathsf{x}', \mathsf{t}), \sigma) = 1 \\ \wedge \mathsf{link}(\mathsf{x}, \mathbf{A}_i, \mathsf{x}', \mathbf{A}_j) = 1 \end{array} \right. \end{array} \right\},$$

where $(\mathsf{svk}, \mathsf{ssk}) \xleftarrow{\$} \mathsf{SIG.KGen}(1^\lambda)$ and $\mathsf{ck} \xleftarrow{\$} \mathsf{COM.KGen}(1^\lambda)$.

Remark 2. In our instantiation using the Groth-Sahai proof system [GS08], the use of commitment $\mathbf{C_s}$ in the extended language definition is equivalent to the extractability property of GS08. Specifically, if an adversary can produce a valid proof for $(\mathsf{x}, \mathbf{C_s}) \in \mathcal{L}_{\mathbf{A}_i}^{(\mathsf{svk},\mathsf{ck})}$ where $\mathsf{x} \notin \mathcal{L}_{\mathbf{A}_i}$, then by the extractability of GS08, one can extract witnesses σ, open such that $\mathbf{C_s} = \mathsf{Com.Commit}(\mathsf{ck}, \sigma; \mathsf{open})$ and $\mathsf{Ver}(\mathsf{svk}, \mathsf{H}(\mathbf{A}_j, \mathsf{x}', \mathsf{t}), \sigma) = 1$ for some $\mathbf{A}_j, \mathsf{x}'$ with $\mathsf{link}(\mathsf{x}, \mathbf{A}_i, \mathsf{x}', \mathbf{A}_j) = 1$. The commitment mechanism thus transforms the existential quantification over signatures into a concrete extractable witness, which is essential for the security reduction to work properly in the GS framework.

Theorem 4. *Let* $\mathsf{SIG} = (\mathsf{KGen}, \mathsf{Sig}, \mathsf{Ver})$ *be a signature scheme. Consider a language-malleable* tLM-NIZK *proof system* $\mathbf{\Pi}_{\mathsf{SIG}}$ *for the extended language family* $\{\mathcal{L}_{\mathbf{A}_i}^{\mathsf{svk},\mathsf{ck}}\}_i$ *with respect to the function family* $\mathcal{H}$*. If* $\mathbf{\Pi}_{\mathsf{SIG}}$ *is computationally sound, it can be interpreted as a special simulation-sound* tLM-NIZK *proof system, denoted* $\mathbf{\Pi}$*, for the linear language family* $\{\mathcal{L}_{\mathbf{A}_i}\}_i$*. More formally, for any* PPT *adversary* $\mathcal{A}$*, there exist* PPT *adversaries* $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ *such that:*

$$\mathsf{Adv}_{\mathbf{\Pi},\mathcal{A}}^{\mathsf{ZK}}(1^\lambda) = \mathsf{Adv}_{\mathbf{\Pi}_{\mathsf{SIG}},\mathcal{B}_1}^{\mathsf{ZK}}(1^\lambda) + \mathsf{Adv}_{\mathsf{COM},\mathcal{B}_4}^{\mathsf{Hiding}}(1^\lambda),$$
$$\mathsf{Adv}_{\mathbf{\Pi},\mathcal{A}}^{\mathsf{SSS}}(1^\lambda) \leq \mathsf{Adv}_{\mathbf{\Pi}_{\mathsf{SIG}},\mathcal{B}_2}^{\mathsf{Sound}}(1^\lambda) + \mathsf{Adv}_{\mathsf{SIG},\mathcal{B}_3}^{\mathsf{UF\text{-}CMA}}(1^\lambda) + \mathsf{Adv}_{\mathsf{COM},\mathcal{B}_5}^{\mathsf{Binding}}(1^\lambda).$$

Consequently, if $\mathbf{\Pi}_{\mathsf{SIG}}$ *is computationally sound and* SIG *is* UF-CMA *(resp., one-time* UF-CMA*), then* $\mathbf{\Pi}$ *is special simulation-sound (resp., one-time special simulation-sound).*

Proof. The correctness and language malleability of $\mathbf{\Pi}$ are direct consequences of the corresponding properties of $\mathbf{\Pi}_{\mathsf{SIG}}$, since a proof for the language $\mathcal{L}_{\mathbf{A}_i}$ is a specific instance of a proof for the extended language $\mathcal{L}^{\mathsf{svk},\mathsf{ck}}_{\mathbf{A}_i}$. Specifically, an honest proof for $\mathcal{L}_{\mathbf{A}_i}$ satisfies the first clause in the OR condition defining $\mathcal{L}^{\mathsf{svk},\mathsf{ck}}_{\mathbf{A}_i}$. Therefore, we omit the details for the correctness and language malleability.

We give more details for the zero-knowledge and special simulation-soundness property.

Zero-Knowledge. We construct a simulator for $\mathbf{\Pi}$. The simulation key for $\mathbf{\Pi}$ is the signing key ssk of the signature scheme SIG. To simulate a proof for a statement $\mathsf{x} \in \mathcal{L}_{\mathbf{A}_i}$ with tag t, the simulator first computes a signature σ on the hash value $\mathsf{hv} := \mathsf{H}(\mathbf{A}_i, \mathsf{x}, \mathsf{t})$ using its knowledge of ssk. It then commits to σ to obtain $\mathbf{C_s}$. Since the identity function is in $\mathcal{H}$, any statement is linked to itself, i.e., $\mathsf{link}(\mathbf{A}_i, \mathsf{x}, \mathbf{A}_i, \mathsf{x}) = 1$. Thus, the simulator can use the signature σ as a witness for the second clause of the extended language $\mathcal{L}^{\mathsf{svk},\mathsf{ck}}_{\mathbf{A}_i}$. It then invokes the honest prover of $\mathbf{\Pi}_{\mathsf{SIG}}$ to generate a proof for the extended statement $(\mathsf{x}, \mathbf{C_s}, \bot)$. The indistinguishability of the simulated proof from a real one follows from the zero-knowledge property of $\mathbf{\Pi}_{\mathsf{SIG}}$ and the hiding property of the commitment scheme COM.

Special Simulation Soundness. The special simulation soundness of $\mathbf{\Pi}$ relies on the computational soundness of $\mathbf{\Pi}_{\mathsf{SIG}}$, the unforgeability of the signature scheme SIG, and the binding property of the commitment scheme COM. Suppose an adversary $\mathcal{A}$ breaks the special simulation soundness of $\mathbf{\Pi}$. It outputs a valid proof $(\mathsf{t}^\star, \pi^\star)$ for a false statement $\mathsf{x}^\star \notin \mathcal{L}_{\mathsf{lpk}^\star}$, where the pair $(\mathsf{lpk}^\star, \mathsf{x}^\star)$ is not linked to any pair $(\mathsf{lpk}, \mathsf{x})$ from the simulation oracle queries. Since $\mathsf{x}^\star$ is a false statement, the witness for the proof $\pi^\star$ must satisfy the second clause of the extended language $\mathcal{L}^{\mathsf{svk},\mathsf{ck}}_{\mathsf{lpk}^\star}$. This implies that the proof contains a commitment $\mathbf{C_s}$ to a valid signature σ on $\mathsf{hv}^\star = \mathsf{H}(\mathbf{A}_j, \mathsf{x}', \mathsf{t}^\star)$ for some linked pair $(\mathbf{A}_j, \mathsf{x}')$. Because $(\mathsf{lpk}^\star, \mathsf{x}^\star)$ is not linked to any prior query, the hash value $\mathsf{hv}^\star$ has not been signed by the simulation oracle. This constitutes a forgery against the signature scheme SIG. A formal reduction can be constructed where an adversary breaking the special simulation soundness of $\mathbf{\Pi}$ is used to break either the computational soundness of $\mathbf{\Pi}_{\mathsf{SIG}}$, the unforgeability of SIG, or the binding property of COM.

□

Language Family Specification To construct our aCCA-secure proxy re-encryption schemes, we first specify the language family $\{\mathcal{L}_{\mathbf{A}_i}\}_i$ and the corresponding transformation functions $\mathcal{H}$. Let $\mathsf{N_U}$ be a positive integer. The construction involves a set of matrices $\{\mathbf{A}_i\}_{i \in [\mathsf{N_U}]}$, where each $\mathbf{A}_i \in \mathbb{Z}_{\mathsf{p}}^{(\ell+1)\times k}$. Each matrix $\mathbf{A}_i$ is partitioned into its top ℓ rows, denoted by $\bar{\mathbf{A}}_i \in \mathbb{Z}_{\mathsf{p}}^{\ell \times k}$, and its bottom row, denoted by $\underline{\mathbf{A}}_i \in \mathbb{Z}_{\mathsf{p}}^{1 \times k}$.

All matrices $\mathbf{A}_i$ for $i \in [\mathsf{N_U}]$ share a common sub-matrix $\bar{\mathbf{A}} \in \mathbb{Z}_{\mathsf{p}}^{\ell \times k}$ comprising their top ℓ rows. The bottom row $\underline{\mathbf{A}}_i$ of each matrix $\mathbf{A}_i$ is defined based on a common vector $\mathbf{x} \leftarrow\!\!\shortmid \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{k}\right)$ and a set of unique vectors $\{\bar{\mathbf{k}}_i \leftarrow\!\!\shortmid \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{\ell}\right)\}_{i \in [\mathsf{N_U}]}$. Specifically, for each $i \in [\mathsf{N_U}]$, the bottom row is computed as $\underline{\mathbf{A}}_i := \mathbf{x}^\intercal + \bar{\mathbf{k}}_i^\intercal \bar{\mathbf{A}}$.

With these definitions, we can specify the language family and the transformation functions. For each $i \in [\mathsf{N_U}]$, we define a vector $\mathbf{k}_i := (\bar{\mathbf{k}}_i^\intercal, 1)^\intercal \in \mathbb{Z}_{\mathsf{p}}^{\ell+1}$. The transformation from index i to j is determined by the difference vector $(d_1, \ldots, d_\ell, 0)^\intercal := \mathbf{k}_j - \mathbf{k}_i$. This vector is used to construct a transformation matrix $\boldsymbol{\Delta}_{i,j} \in \mathbb{Z}_{\mathsf{p}}^{(\ell+1)\times(\ell+1)}$ as follows:

$$\boldsymbol{\Delta}_{i,j} := \begin{pmatrix} 1 & 0 & \ldots & 0 & 0 \\ 0 & 1 & \ldots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \ldots & 1 & 0 \\ d_1 & d_2 & \ldots & d_\ell & 1 \end{pmatrix}.$$

By construction, this matrix maps $\mathbf{A}_i$ to $\mathbf{A}_j$, i.e., $\mathbf{A}_j = \boldsymbol{\Delta}_{i,j}\mathbf{A}_i$. The language family and transformation functions are then defined as follows:

- The language $\mathcal{L}_{\mathbf{A}_i}$ is the linear span language defined by

$$\mathcal{L}_{\mathbf{A}_i} := \{\mathsf{x} \in \mathbb{G}_1^{\ell+1} \mid \exists \mathbf{r} \in \mathbb{Z}_{\mathsf{p}}^{k} : \mathsf{x} = \left[\mathbf{A}_i\right]_1 \mathbf{r}\}.$$

- The transformation function family $\mathcal{H} = \{(\mathsf{H}_{i,j}^{\mathsf{lpk}}, \mathsf{H}_{i,j}^{\mathsf{x}}, \mathsf{H}_{i,j}^{\mathsf{w}})\}_{i,j \in [\mathsf{N_U}]}$ consists of:
  - A language public key transformation $\mathsf{H}_{i,j}^{\mathsf{lpk}} : \mathbb{Z}_{\mathsf{p}}^{(\ell+1)\times k} \to \mathbb{Z}_{\mathsf{p}}^{(\ell+1)\times k}$ that maps $\mathbf{A}_i$ to $\mathbf{A}_j$ via left-multiplication by $\boldsymbol{\Delta}_{i,j}$:

$$\mathsf{H}_{i,j}^{\mathsf{lpk}}(\mathbf{A}_i) := \boldsymbol{\Delta}_{i,j}\mathbf{A}_i = \mathbf{A}_j.$$

  - A statement transformation $\mathsf{H}_{i,j}^{\mathsf{x}} : \mathbb{G}_1^{\ell+1} \to \mathbb{G}_1^{\ell+1}$ that maps a statement $\mathsf{x}_i \in \mathcal{L}_{\mathbf{A}_i}$ to a statement $\mathsf{x}_j \in \mathcal{L}_{\mathbf{A}_j}$ via left-multiplication by $\boldsymbol{\Delta}_{i,j}$:

$$\mathsf{H}_{i,j}^{\mathsf{x}}(\mathsf{x}_i) := \boldsymbol{\Delta}_{i,j}\mathsf{x}_i = \mathsf{x}_j.$$

  - A witness transformation $\mathsf{H}_{i,j}^{\mathsf{w}} : \mathbb{Z}_{\mathsf{p}}^{k} \to \mathbb{Z}_{\mathsf{p}}^{k}$ that maps a witness w_i to a witness w_j via identity function:

$$\mathsf{H}_{i,j}^{\mathsf{w}}(\mathsf{w}_i) := \mathsf{w}_i = \mathsf{w}_j.$$

We provide the full instantiation of our one-time and unbounded special simulation-sound $\mathsf{tLM\text{-}NIZK}$ proof system in Section E. The construction is based on a Groth-Sahai proof for the language $\mathcal{L}_{\mathbf{A}_i}^{(\mathsf{svk},\mathsf{ck})}$. The language malleability property is a direct consequence of the linearity inherent in the Groth-Sahai framework.

E Instantiation of Special Simulation Sound tLM-NIZK

To instantiate the generic construction from D.1, we first fix the following public parameters and commitment algorithms.

To commit a vector $\mathbf{w} \in \mathbb{Z}_{\mathsf{p}}^n$ or $[\mathbf{w}] \in \mathbb{G}^n$, we proceed as follows:

Public parameters: Sample $\mathbf{D} \hookleftarrow \mathcal{D}_k$, choose $\mathbf{z}, \mathbf{y} \in \mathbb{Z}_{\mathsf{p}}^{k+1}$, and set $\mathbf{U} := [\mathbf{D}|\mathbf{z}]$.
We consider two parameter regimes:
- Perfectly Binding Mode:
  Scalar vectors: $\mathbf{z} \notin \mathsf{Span}(\mathbf{D})$.
  Group vectors: $\mathbf{z} \in \mathsf{Span}(\mathbf{D})$ and $\mathbf{y} \notin \mathsf{Span}(\mathbf{D})$.
- Perfectly Hiding Mode:
  Scalar vectors: $\mathbf{z} \in \mathsf{Span}(\mathbf{D})$.
  Group vectors: $\mathbf{z} \notin \mathsf{Span}(\mathbf{D})$ and $\mathbf{y} \in \mathsf{Span}(\mathbf{D})$.

Commitments: Given the parameters above, commitments are computed as follows:
- Scalar $\mathbf{w} \in \mathbb{Z}_{\mathsf{p}}^n$: $[\mathbf{C_w}] := [\mathbf{D}]\,\mathbf{R} + [\mathbf{z}]\,\mathbf{w}^\intercal$, where we sample $\mathbf{R} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{k\times n}\right)$.
- Group vector $[\mathbf{w}] \in \mathbb{G}^n$: $[\mathbf{C_w}] := [\mathbf{U}]\,\mathbf{R} + \mathbf{y}\,[\mathbf{w}^\intercal]$, where we sample $\mathbf{R} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{(k+1)\times n}\right)$.

We use subscripts to indicate whether the public parameters are instantiated in $\mathbb{G}_1$ or $\mathbb{G}_2$.

E.1 One-Time Special Simulation Sound tLM-NIZK

To instantiate the one-time special simulation sound tLM-NIZK with tight security, we proceed by the following two steps

- We use the tightly-secure one-time structure preserving signature OTS given in [KPW15] to instantiate the underlying signature scheme.
- We use the Groth-Sahai proof system [GS08] to instantiate the language-malleable tLM-NIZK proof system $\mathbf{\Pi}_{\mathsf{SIG}}$ for the extended language family $\{\mathcal{L}^{\mathsf{svk}}_{\mathbf{A}_i}\}_i$ with respect to the function family $\mathcal{H}$.

We first recall the signature $\mathsf{OTS} = (\mathsf{KGen}, \mathsf{Sig}, \mathsf{Ver})$ in Figure 10. To be compatible with the other part of the tLM-NIZK, we exchange $\mathbb{G}_1$ and $\mathbb{G}_2$ in the original scheme. Due to the symmetric nature of pairing computation, this modification does not affect the correctness and security of the scheme.

Theorem 5 ([KPW15]). *The signature scheme* OTS *in Figure 10 is one-time* UF-CMA. *Concretely, for all* PPT *adversaries* $\mathcal{A}$*, there exists an* PPT *adversary* $\mathcal{B}$ *with* $\mathbf{T}(\mathcal{A}) \approx \mathbf{T}(\mathcal{B})$ *and*

$$\mathsf{Adv}^{\mathsf{OT\text{-}CMA}}_{\mathsf{OTS},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{KerMDH}}_{\mathcal{D}_k,\mathsf{GGen},\mathcal{B}}(\lambda) + 1/q.$$

We translate the statements of $\mathcal{L}^{\mathsf{svk}}_{\mathbf{A}_i}$ as follows:

$\underline{\textbf{Alg } \mathsf{KGen}(1^\lambda)}$:
01 $\mathbf{B} \hookleftarrow \mathcal{D}_k$
02 $\mathbf{K} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(n+1)\times(k+1)}\right)$
03 $\mathbf{C} = \mathbf{K}\mathbf{B}$
04 $\mathsf{svk} := \left(\left[\mathbf{B}^\intercal\right]_1, \left[\mathbf{C}^\intercal\right]_1\right)$
05 $\mathsf{ssk} := \mathbf{K}$
06 **return** $(\mathsf{svk}, \mathsf{ssk})$

$\underline{\textbf{Alg } \mathsf{Sig}(\mathsf{ssk}, \left[\mathsf{m}\right]_2 \in \mathbb{G}_2^n)}$:
07 $\left[\sigma\right]_2 := \left[\left(1 \ \mathsf{m}\right)\mathbf{K}\right]_2^\intercal$
08 **return** $\left[\sigma\right]_2 \in \mathbb{G}_2^{k+1}$

$\underline{\textbf{Alg } \mathsf{Ver}(\mathsf{svk}, \mathsf{m}, \left[\sigma\right]_2)}$:
09 **check** $\left[\mathbf{B}^\intercal\right]_1 \bullet \left[\sigma\right]_2 = \left[\mathbf{C}^\intercal\right]_1 \bullet \left[\left(1 \ \mathsf{m}\right)^\intercal\right]_2$

Fig. 10. Construction of the one-time structure preserving signature scheme $\mathbf{\Sigma}_{\mathsf{OT}} = (\mathsf{KGen}, \mathsf{Sig}, \mathsf{Ver})$ [KPW15].

- $\left[\mathbf{A}_i\right]_1 \bullet \boxed{\left[\mathbf{w}\right]_2} - \left[\mathsf{x}\right]_1 \bullet \left[1\right]_2 = \left[\mathbf{0}\right]_1$[21] // $\mathsf{x} \in \mathcal{L}_{\mathbf{A}_i}$
- $\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1 - \left[\bar{\mathbf{A}}^\intercal\right]_1 \boxed{\bar{\mathbf{k}}^\star} = \left[0\right]_1$ // $\mathsf{link}(\mathbf{A}_i, \mathsf{x}, \mathbf{A}^\star, \mathsf{x}^\star) = 1$
- $\left[\underline{\mathsf{x}}^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1 - \left[\bar{\mathsf{x}}^\intercal\right]_1 \boxed{\bar{\mathbf{k}}^\star} = \left[0\right]_1$ // $\mathsf{link}(\mathbf{A}_i, \mathsf{x}, \mathbf{A}^\star, \mathsf{x}^\star) = 1$
- $\left[\mathbf{B}^\intercal\right]_1 \bullet \boxed{\left[\sigma\right]_2} = \left[\mathbf{C}^\intercal\right]_1 \bullet \begin{bmatrix}1\\ \mathsf{hv}\end{bmatrix}_2$ // $\mathsf{Ver}(\mathsf{svk}, \mathsf{hv}, \sigma) = 1$

where $\mathsf{hv} = \mathsf{H}(\underline{\mathbf{A}}^\star, \underline{\mathsf{x}}^\star, \mathsf{t})$.

Remark 3. In our proxy re-encryption setting, a ciphertext consists of three components $(\mathsf{ct}_0, \mathsf{ct}_1, \mathsf{ct}_2)$, where $(\mathsf{ct}_0, \mathsf{ct}_1)$ together form the statement x used in this proof system, with ct_1 corresponding to the bottom row $\underline{\mathsf{x}}$. A complete link verification should ensure: (1) the transformational relationship of ct_1 (the bottom component), and (2) the invariance of ct_0 and ct_2 between linked ciphertexts.

The hash function and link verification equations above only address the bottom row relationships, corresponding to the ct_1 component. However, in our CCA construction, we employ an external signature scheme to authenticate ct_0 and ct_2, ensuring their invariance across linked ciphertexts. Since the hash value incorporates the tag t, any signature reuse necessarily implies tag reuse, which would be detected by the signature verification in the outer CCA protocol. Therefore, this tLM-NIZK system focuses solely on proving the correct transformation of ct_1, while the complete link property is jointly guaranteed by this proof system and the external signature scheme.

Moreover, we prove the first equation using the vector $\mathbf{z}_\mathbf{A}$ to commit elements in $\mathbb{G}_2$, and the last three equations are proved using $\mathbf{z}_\sigma$ to commit elements in $\mathbb{G}_2$. Moreover, we have $\mathbf{z} = \mathbf{z}_\mathbf{A} + \mathbf{z}_\sigma \notin \mathsf{Span}(\mathbf{D})$, which ensures either the first two equations or the last three equations are proved in the soundness setting. Noticing that all commitments in $\mathbb{G}_1$ are in the perfectly binding mode. We give all the commitments as follows

- $\left[\mathbf{C}_\mathbf{w}\right]_2 := \left[\mathbf{D}_2\right]_2 \mathbf{R}_\mathbf{w} + \left[\mathbf{z}_\mathbf{A}\right]_2 \mathbf{w}^\intercal$

[21] Note that x inherently consists of group elements in $\mathbb{G}_1$, but we explicitly use the notation $[\cdot]_1$ here to emphasize its group properties for clarity. We adopt this consistent notation throughout the subsequent presentation.

– $\left[\mathbf{C_k}\right]_2 := \left[\mathbf{D}_2\right]_2 \mathbf{R_k} + \left[\mathbf{z}_\sigma\right]_2 (\bar{\mathbf{k}}^\star)^\intercal$
– $\left[\mathbf{C}_\sigma\right]_2 := \left[\sigma\right]_2 \mathbf{R_s} + \left[\mathbf{z}_\sigma\right]_2 \sigma^\intercal$

The verification equations are given as follows:

– $\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}_\mathbf{w}^\intercal\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[\mathbf{z}_\mathbf{A}^\intercal\right]_2 = \left[\mathbf{P_A}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
– $\left[\underline{\mathbf{A}}_i^\intercal\right]_1 \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1 \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathbf{A}}^\intercal\right]_1 \bullet \left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 = \left[\mathbf{P_d}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
– $\left[\underline{\mathsf{x}}^\intercal\right]_1 \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1 \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathsf{x}}^\intercal\right]_1 \bullet \left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 = \left[\mathbf{P}_\mathsf{x}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
– $\left[\mathbf{B}^\intercal\right]_1 \bullet \left[\mathbf{C}_\sigma^\intercal\right]_2 - \left[\mathbf{C}^\intercal\right]_1 \bullet \left[\begin{pmatrix} 1 \\ \mathsf{hv} \end{pmatrix} \mathbf{z}_\sigma^\intercal\right]_2 = \left[\mathbf{P}_\sigma\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$

We give the full construction of the one-time special simulation sound tLM-NIZK proof system in Figure 11. We show in Figure 12 that the construction is malleable with respect to the transformation set $\mathcal{H}$.

Alg $\mathsf{Setup}(1^\lambda)$:
01 $\mathbf{D} \hookleftarrow \mathcal{D}_k$
02 $\mathbf{z} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D})\right)$
03 $\mathbf{B} \hookleftarrow \mathcal{D}_k$; $\mathbf{K} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{2\times(k+1)}\right)$
04 $\mathbf{C} = \mathbf{KB}$
05 $\mathsf{crs} := (\left[\mathbf{D}\right]_2, \left[\mathbf{z}\right]_2, \left[\mathbf{B}^\intercal\right]_1, \left[\mathbf{C}^\intercal\right]_1)$
06 **return** crs

Alg $\mathsf{Ver}(\mathsf{crs}, \mathsf{t}, \mathsf{x}, \pi)$:
07 $\mathsf{hv} := \mathsf{H}(\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \mathsf{t})$
08 $\left[\mathbf{z}_\sigma\right]_1 := \left[\mathbf{z}\right]_1 - \left[\mathbf{z}_\mathbf{A}\right]_1$
09 **check** $\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}_\mathbf{w}^\intercal\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[\mathbf{z}_\mathbf{A}^\intercal\right]_2 = \left[\mathbf{P_A}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
10 **check** $(\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathbf{A}}^\intercal\right]_1 \bullet \left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 = \left[\mathbf{P_d}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
11 **check** $(\left[\underline{\mathsf{x}}^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathsf{x}}^\intercal\right]_1 \bullet \left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 = \left[\mathbf{P}_\mathsf{x}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
12 **check** $\left[\mathbf{B}^\intercal\right]_1 \bullet \left[\mathbf{C}_\sigma^\intercal\right]_2 - \left[\mathbf{C}^\intercal\right]_1 \bullet \left[\begin{pmatrix} 1 \\ \mathsf{hv} \end{pmatrix} \mathbf{z}_\sigma^\intercal\right]_2 = \left[\mathbf{P}_\sigma\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
13 **return** 1

Alg $\mathsf{Prove}(\mathsf{crs}, \mathsf{t}, \mathsf{x}, \mathbf{w})$:
14 $\left[\underline{\mathbf{A}}^\star\right]_1 \hookleftarrow \mathsf{U}\left(\mathbb{G}_1^{1\times k}\right)$
15 $\left[\underline{\mathsf{x}}^\star\right]_1 \hookleftarrow \mathsf{U}\left(\mathbb{G}_1^{1\times 1}\right)$
16 $\mathsf{hv} = \mathsf{H}(\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \mathsf{t})$
17 $\mathbf{v}_\sigma \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^k\right)$; $\left[\mathbf{z}_\sigma\right]_2 := \left[\mathbf{D}\right]_2 \mathbf{v}_\sigma$
18 $\left[\mathbf{z}_\mathbf{A}\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_\sigma\right]_2$
19 $\mathbf{R_w} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k\times k}\right)$
20 $\mathbf{R_k}, \mathbf{R}_\sigma \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1\times k}\right)$
21 $\left[\mathbf{C}_\mathbf{w}^\intercal\right]_2 := \mathbf{R_w} \left[\mathbf{D}^\intercal\right]_2 + \mathbf{w} \left[\mathbf{z}_\mathbf{A}^\intercal\right]_2$
22 $\left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 := \mathbf{R_k} \left[\mathbf{D}^\intercal\right]_2$
23 $\left[\mathbf{C}_\sigma^\intercal\right]_2 := \mathbf{R}_\sigma \left[\mathbf{D}^\intercal\right]_2$
24 $\left[\mathbf{P_A}\right]_1 := \left[\mathbf{A}_i\right]_1 \mathbf{R_w}$
25 $\left[\mathbf{P_d}\right]_1 := (\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1)\mathbf{v}_\sigma^\intercal - \left[\bar{\mathbf{A}}^\intercal\right]_1 \mathbf{R_k}$
26 $\left[\mathbf{P}_\mathsf{x}\right]_1 := (\left[\underline{\mathsf{x}}^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1)\mathbf{v}_\sigma^\intercal - \left[\bar{\mathsf{x}}^\intercal\right]_1 \mathbf{R_k}$
27 $\left[\mathbf{P}_\sigma\right]_1 := \left[\mathbf{B}^\intercal\right]_1 \mathbf{R}_\sigma - \left[\mathbf{C}^\intercal\right]_1 \left(\begin{smallmatrix} 1 \\ \mathsf{hv} \end{smallmatrix}\right) \mathbf{v}_\sigma^\intercal$
28 $\pi := (\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \left[\mathbf{z}_\mathbf{A}\right]_1, \left[\mathbf{C_w}\right]_2, \left[\mathbf{C_k}\right]_2, \left[\mathbf{C}_\sigma\right]_2, \left[\mathbf{P_A}\right]_1, \left[\mathbf{P_d}\right]_1, \left[\mathbf{P}_\mathsf{x}\right]_1, \left[\mathbf{P}_\sigma\right]_1)$
29 **return** π

Fig. 11. Construction of one-time special simulation sound tLM-NIZK proof system.

We give the language malleability algorithm in Figure 12 and the construction of the simulator $\mathcal{S} = (\mathsf{SSetup}, \mathsf{Sim})$ for the $\mathbf{\Pi}_{\mathsf{OT}}$ scheme as detailed in Figure 13.

The correctness of the proof system $\mathbf{\Pi}_{\mathsf{OT}}$ is straightforward. We provide the tight proof for the zero-knowledge property Theorem 6 and special simulation

Alg $\mathsf{Trans}(\mathsf{crs}, \mathsf{t}, \mathsf{x}, \pi, \mathsf{d}_{i,j})$:

01 **parse** $(\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \left[\mathbf{z}_\mathbf{A}\right]_1, \left[\mathbf{C}_\mathbf{w}\right]_2, \left[\mathbf{C}_\mathbf{k}\right]_2, \left[\mathbf{C}_\sigma\right]_2, \left[\mathbf{P}_\mathbf{A}\right]_1, \left[\mathbf{P}_\mathbf{d}\right]_1, \left[\mathbf{P}_\mathsf{x}\right]_1, \left[\mathbf{P}_\sigma\right]_1) =: \pi$

02 $\left[\mathbf{P}_\mathbf{A}\right]_1' := \boldsymbol{\Delta}_{i,j} \left[\mathbf{P}_\mathbf{A}\right]_1$; $\left[\mathbf{C}_\mathbf{k}\right]_2' := \left[\mathbf{C}_\mathbf{k}\right]_2 + \bar{\mathsf{d}}_{i,j} \left[\mathbf{z}_\mathbf{A}^\intercal\right]_2$

03 $\pi' = (\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \left[\mathbf{z}_\mathbf{A}\right]_1, \left[\mathbf{C}_\mathbf{w}\right]_2, \left[\mathbf{C}_\mathbf{k}\right]_2', \left[\mathbf{C}_\sigma\right]_2, \left[\mathbf{P}_\mathbf{A}\right]_1', \left[\mathbf{P}_\mathbf{d}\right]_1, \left[\mathbf{P}_\mathsf{x}\right]_1, \left[\mathbf{P}_\sigma\right]_1)$

04 **return** π'

Fig. 12. Language malleability of the one-time special simulation sound tLM-NIZK proof system. Notice that $\boldsymbol{\Delta}_{i,j}$ can be efficiently computed from $\mathsf{d}_{i,j}$.

Alg $\mathsf{SSetup}(1^\lambda)$:

01 $\mathbf{D} \leftarrow\!\!\$\ \mathcal{D}_k$

02 $\mathbf{z} \leftarrow\!\!\$\ \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D})\right)$

03 $\mathbf{B} \leftarrow\!\!\$\ \mathcal{D}_k$; $\mathbf{K} \leftarrow\!\!\$\ \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{2\times(k+1)}\right)$

04 $\mathbf{C} = \mathbf{K}\mathbf{B}$

05 $\mathsf{crs} := (\left[\mathbf{D}\right]_2, \left[\mathbf{z}\right]_2, \left[\mathbf{B}^\intercal\right]_1, \left[\mathbf{C}^\intercal\right]_1)$

06 $\mathsf{simK} := (\mathbf{K})$

07 **return** $(\mathsf{crs}, \mathsf{simK})$

Alg $\mathsf{Sim}(\mathsf{crs}, \mathsf{t}, \mathsf{simK}, \mathsf{x})$:

08 $\bar{\mathbf{k}}^\star \leftarrow\!\!\$\ \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1}\right)$

09 $\left[\underline{\mathbf{A}}^\star\right]_1 = \left[\underline{\mathbf{A}}_i\right]_1 - (\bar{\mathbf{k}}^\star)^\intercal \left[\bar{\mathbf{A}}\right]_1$

10 $\left[\underline{\mathsf{x}}^\star\right]_1 = \left[\underline{\mathsf{x}}\right]_1 - (\bar{\mathbf{k}}^\star)^\intercal \left[\bar{\mathsf{x}}\right]_1$

11 $\mathsf{hv} := \mathsf{H}(\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \mathsf{t})$

12 $\sigma := \mathbf{K}^\intercal \binom{1}{\mathsf{hv}}$

13 $\mathbf{v}_\mathbf{A} \leftarrow\!\!\$\ \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^k\right)$; $\left[\mathbf{z}_\mathbf{A}\right]_2 := \left[\mathbf{D}\right]_2 \mathbf{v}_\mathbf{A}$

14 $\left[\mathbf{z}_\sigma\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_\mathbf{A}\right]_2$

15 $\mathbf{R}_\mathbf{w} \leftarrow\!\!\$\ \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k\times k}\right)$

16 $\mathbf{R}_\mathbf{k}, \mathbf{R}_\sigma \leftarrow\!\!\$\ \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1\times k}\right)$

17 $\left[\mathbf{C}_\mathbf{w}^\intercal\right]_2 := \mathbf{R}_\mathbf{w} \left[\mathbf{D}^\intercal\right]_2$

18 $\left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 := \mathbf{R}_\mathbf{k} \left[\mathbf{D}^\intercal\right]_2 + \bar{\mathbf{k}}^\star \left[\mathbf{z}_\sigma^\intercal\right]_2$

19 $\left[\mathbf{C}_\sigma^\intercal\right]_2 := \mathbf{R}_\sigma \left[\mathbf{D}^\intercal\right]_2 + \sigma \left[\mathbf{z}_\sigma^\intercal\right]_2$

20 $\left[\mathbf{P}_\mathbf{A}\right]_1 := \left[\mathbf{A}_i\right]_1 \mathbf{R}_\mathbf{w} - \left[\mathsf{x}\right]_1 \mathbf{v}_\mathbf{A}^\intercal$

21 $\left[\mathbf{P}_\mathbf{d}\right]_1 := -\left[\bar{\mathbf{A}}^\intercal\right]_1 \mathbf{R}_\mathbf{k}$

22 $\left[\mathbf{P}_\mathsf{x}\right]_1 := -\left[\bar{\mathsf{x}}^\intercal\right]_1 \mathbf{R}_\mathbf{k}$

23 $\left[\mathbf{P}_\sigma\right]_1 := \left[\mathbf{B}^\intercal\right]_1 \mathbf{R}_\sigma$

24 $\pi := (\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \left[\mathbf{z}_\mathbf{A}\right]_1, \left[\mathbf{C}_\mathbf{w}\right]_2, \left[\mathbf{C}_\mathbf{k}\right]_2, \left[\mathbf{C}_\sigma\right]_2, \left[\mathbf{P}_\mathbf{A}\right]_1, \left[\mathbf{P}_\mathbf{d}\right]_1, \left[\mathbf{P}_\mathsf{x}\right]_1, \left[\mathbf{P}_\sigma\right]_1)$

25 **return** π

Fig. 13. Simulator of the one-time special simulation sound tLM-NIZK proof system.

soundness of the proof system $\boldsymbol{\Pi}_{\mathsf{OT}}$ in Theorem 7. The detailed proof of the theorems can be found in F.

Theorem 6 (Zero-Knowledge). *The one-time* tLM-NIZK *proof system in Figure 11 is zero-knowledge. Concretely, for all* PPT *adversaries* $\mathcal{A}$*, there exists an* PPT *adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\mathsf{ZK}}_{\boldsymbol{\Pi}_{\mathsf{OT}},\mathcal{A}}(\lambda) \leq 2\mathsf{Adv}^{\mathsf{KerMDH}}_{\mathcal{D}_k,\mathsf{GGen},\mathcal{B}}(\lambda).$$

Theorem 7 (Special Simulation Sound). *The one-time* tLM-NIZK *proof system presented in Figure 11 satisfies special simulation soundness.*

E.2 Special Simulation Sound tLM-NIZK

This section details the construction of an unbounded special simulation sound tLM-NIZK scheme $\mathbf{\Pi}_{\mathsf{U}}$, employing a framework similar to that used for the one-time special simulation sound tLM-NIZK construction in Figure 13. We replace the one-time signature scheme with a tightly-secure UF-CMA signature scheme and change the number of rows in matrix $\bar{\mathbf{A}}$ from $k+1$ to ℓ. The detailed construction of the proof system $\mathbf{\Pi}_{\mathsf{U}}$ is presented in Figure 16.

We first recall the almost tightly-secure UF-CMA signature scheme [GHKP18] in Figure 14. The signature scheme is almost tightly-secure under the $\mathcal{D}_k$-MDDH assumption. We modify the signature scheme to sign a message $\mathsf{m} \in \mathbb{Z}_{\mathsf{p}}$ and the signature is of the form $\sigma = (\left[\mathbf{t}\right]_1, \pi, \left[\mathbf{s}\right]_1)$.

Alg $\mathsf{KGen}(1^\lambda)$:
01 $\mathsf{crs} \xleftarrow{\$} \Pi^{\vee}_{\mathbf{B}_0,\mathbf{B}_1}.\mathsf{Setup}(1^\lambda)$
02 $\mathbf{B}_0, \mathbf{B}_1 \hookleftarrow \mathcal{D}_{2k,k}$
03 $\mathbf{B} \hookleftarrow \mathcal{D}_k$
04 $\mathbf{K}_0, \mathbf{K}_1 \hookleftarrow \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{2k\times(k+1)}\right)$
05 $\mathsf{svk} := (\mathsf{crs}, \left[\mathbf{B}_0\right]_1, \left[\mathbf{B}\right]_2, \left[\mathbf{K}_0\mathbf{B}\right]_2, \left[\mathbf{K}_1\mathbf{B}\right]_2)$
06 $\mathsf{ssk} := (\mathbf{K}_0, \mathbf{K}_1)$
07 **return** $(\mathsf{svk}, \mathsf{ssk})$

Alg $\mathsf{Sig}(\mathsf{ssk}, \mathsf{m} \in \mathbb{Z}_{\mathsf{p}})$:
08 $\mathbf{r} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_{\mathsf{p}}^{k}\right);\ \left[\mathbf{t}\right]_1 := \left[\mathbf{B}_0\right]_1 \mathbf{r}$
09 $\pi \xleftarrow{\$} \Pi^{\vee}_{\mathbf{B}_0,\mathbf{B}_1}.\mathsf{Prove}(\mathsf{crs}, \left[\mathbf{t}\right]_1, \mathbf{r})$
10 $\left[\mathbf{s}\right]_1 := (\mathbf{K}_0 + \mathsf{m}\mathbf{K}_1)^\intercal \left[\mathbf{t}\right]_1$
11 **return** $\sigma := (\left[\mathbf{t}\right]_1, \pi, \left[\mathbf{s}\right]_1)$

Alg $\mathsf{Ver}(\mathsf{svk}, \mathsf{m}, \sigma)$:
12 **check** $\Pi^{\vee}_{\mathbf{B}_0,\mathbf{B}_1}.\mathsf{Ver}(\mathsf{crs}, \left[\mathbf{t}\right]_1, \pi)$
13 **check** $\left[\mathbf{s}\right]_1 \neq \left[\mathbf{0}\right]_1$
14 **check** $\left[\mathbf{s}^\intercal\right]_1 \bullet \left[\mathbf{B}\right]_2 = \left[\mathbf{t}^\intercal\right]_1 \bullet \left[(\mathbf{K_0} + \mathsf{m}\mathbf{K_1})\mathbf{B}\right]_2$
15 **return** 1

Fig. 14. Construction of tightly-secure structure preserving signature scheme. [GHKP18]

For the sake of completeness, we also add the description of the OR proof system $\Pi^{\vee}_{\mathbf{B}_0,\mathbf{B}_1}$ in Figure 15. We define the language as

$$\mathcal{L}^{\vee}_{\mathbf{B}_0,\mathbf{B}_1} = \left\{ \left[\mathbf{t}\right]_2 \mid \exists \mathbf{r} \in \mathbb{Z}_{\mathsf{p}}^{2k}.\ \left[\mathbf{t}\right]_2 = \left[\mathbf{B}_0\right]_2 \mathbf{r} \vee \left[\mathbf{t}\right]_2 = \left[\mathbf{B}_1\right]_2 \mathbf{r} \right\}.$$

Theorem 8 ([GHKP18, Theorem 2]). *Let* $\Pi^{\vee}_{\mathbf{B}_0,\mathbf{B}_1}$ *be a non-interactive zero-knowledge proof. The signature scheme in Figure 14 is tightly-secure under the* $\mathcal{D}_k$-MDDH *and the* $\mathcal{D}_{2k,k}$-MDDH *assumption.*

Concretely, for all PPT *adversaries* $\mathcal{A}$*, there exists* PPT *adversaries* $\mathcal{B}_1, \cdots, \mathcal{B}_4$ *with* $\mathbf{T}(\mathcal{B}) \approx \mathbf{T}(\mathcal{A}) + Q \cdot \mathsf{poly}(\lambda)$ *such that*

$$\mathsf{Adv}^{\mathsf{uf\text{-}cma}}_{\mathsf{Sig},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_k,\mathbb{G}_2,\mathcal{B}_1}(\lambda) + \mathsf{Adv}^{\mathsf{core}}_{\mathcal{B}_2}(\lambda) + \frac{Q}{p}.$$

Alg $\mathsf{Setup}(1^\lambda)$:
$\mathbf{D} \hookleftarrow \mathcal{D}_k$
$\mathbf{z} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D})\right)$
$\mathsf{crs} := (\left[\mathbf{D}\right]_2, \left[\mathbf{z}\right]_2)$
return crs

Alg $\mathsf{Prove}(\mathsf{crs}, \left[\mathbf{t}\right]_1, \mathbf{r})$:
let $j \in \{0,1\}$ **s.t.** $\left[\mathbf{t}\right]_1 = \left[\mathbf{B}_j\right]_1 \cdot \mathbf{r}$
$\mathbf{v} \hookleftarrow \mathbb{Z}_\mathsf{p}^k$
$\left[\mathbf{z}_{1-j}\right]_2 := \left[\mathbf{D}\right]_2 \cdot \mathbf{v}$
$\left[\mathbf{z}_j\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_{1-j}\right]_2$
$\mathbf{S}_0, \mathbf{S}_1 \hookleftarrow \mathbb{Z}_\mathsf{p}^{k\times k}$
$\left[\mathbf{C}_j\right]_2 := \mathbf{S}_j \cdot \left[\mathbf{D}\right]_2^\intercal + \mathbf{r} \cdot \left[\mathbf{z}_j\right]_2^\intercal$
$\left[\mathbf{\Pi}_j\right]_1 := \left[\mathbf{B}_j\right]_1 \cdot \mathbf{S}_j$
$\left[\mathbf{C}_{1-j}\right]_2 := \mathbf{S}_{1-j} \cdot \left[\mathbf{D}\right]_2^\intercal$
$\left[\mathbf{\Pi}_{1-j}\right]_1 := \left[\mathbf{B}_{1-j}\right]_1 \cdot \mathbf{S}_{1-j} - \left[\mathbf{t}\right]_1 \cdot \mathbf{v}^\intercal$
$\pi := (\left[\mathbf{z}_0\right]_2, (\left[\mathbf{C}_i\right]_2, \left[\mathit{\Pi}_i\right]_1)_{i\in\{0,1\}})$
return π

Alg $\mathsf{SSetup}(1^\lambda)$:
$\mathbf{D} \hookleftarrow \mathcal{D}_k, \mathbf{u} \hookleftarrow \mathbb{Z}_\mathsf{p}^k$
$\mathbf{z} := \mathbf{D} \cdot \mathbf{u}$
$\mathsf{crs} := (\mathsf{p}, \left[\mathbf{D}\right]_2, \left[\mathbf{z}\right]_2)$; $\mathsf{td} := \mathbf{u}$
return $(\mathsf{crs}, \mathsf{td})$

Alg $\mathsf{PVer}(\mathsf{crs}, \left[\mathbf{t}\right]_1, \pi)$:
$\left[\mathbf{z}_1\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_0\right]_2$
check $\left[\mathbf{B}_0\right]_1 \bullet \left[\mathbf{C}_0\right]_2 = (\left[\mathbf{\Pi}_0\right]_1 \bullet \left[\mathbf{D}\right]_2^\intercal) + (\left[\mathbf{t}\right]_1 \bullet \left[\mathbf{z}_0\right]_2^\intercal)$
check $\left[\mathbf{B}_1\right]_1 \bullet \left[\mathbf{C}_1\right]_2 = (\left[\mathbf{\Pi}_1\right]_1 \bullet \left[\mathbf{D}\right]_2^\intercal) + (\left[\mathbf{t}\right]_1 \bullet \left[\mathbf{z}_1\right]_2^\intercal)$
return 1

Alg $\mathsf{PSim}(\mathsf{crs}, \mathsf{td}, \left[\mathbf{t}\right]_1)$:
parse $\mathsf{td} =: \mathbf{u}$
$\mathbf{v} \hookleftarrow \mathbb{Z}_\mathsf{p}^k$
$\left[\mathbf{z}_0\right]_2 := \left[\mathbf{D}\right]_2 \cdot \mathbf{v}$
$\left[\mathbf{z}_1\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_0\right]_2$
$\mathbf{S}_0, \mathbf{S}_1 \hookleftarrow \mathbb{Z}_\mathsf{p}^{k\times k}$
$\left[\mathbf{C}_0\right]_2 := \mathbf{S}_0 \cdot \left[\mathbf{D}\right]_2^\intercal$
$\left[\mathbf{\Pi}_0\right]_1 := \left[\mathbf{B}_0\right]_1 \cdot \mathbf{S}_0 - \left[\mathbf{t}\right]_1 \cdot \mathbf{v}^\intercal$
$\left[\mathbf{C}_1\right]_2 := \mathbf{S}_1 \cdot \left[\mathbf{D}\right]_2^\intercal$
$\left[\mathbf{\Pi}_1\right]_1 := \left[\mathbf{B}_1\right]_1 \cdot \mathbf{S}_1 - \left[\mathbf{t}\right]_1 \cdot (\mathbf{u}-\mathbf{v})^\intercal$
$\pi := (\left[\mathbf{z}_0\right]_2, (\left[\mathbf{C}_i\right]_2, \left[\mathbf{\Pi}_i\right]_1)_{i\in\{0,1\}})$
return π

Fig. 15. NIZK argument for the OR language $\mathcal{L}^\vee_{\mathbf{B}_0,\mathbf{B}_1}$.

where

$$\begin{aligned}\mathsf{Adv}^{\mathsf{core}}_{\mathcal{B}_2}(\lambda) := {} & (4k\lceil\log Q\rceil + 2) \cdot \mathsf{Adv}^{\mathsf{MDDH}}_{\mathbb{G}_1,\mathcal{D}_{2k,k},\mathcal{B}_3}(\lambda) \\ & + (2\lceil\log Q\rceil + 2) \cdot \mathsf{Adv}^{\mathsf{ZK}}_{\mathsf{PS},\mathcal{B}_4}(\lambda) \\ & + \lceil\log Q\rceil \cdot \varDelta_{\mathcal{D}_{2k,k}} + \tfrac{4\lceil\log Q\rceil+2}{p-1} + \tfrac{\lceil\log Q\rceil Q}{p}.\end{aligned}$$

We define $\mathbf{u}_i \in \mathbb{Z}_\mathsf{p}^{2k}$ as a vector with all entries equal to zero except for the i-th entry, which is 1. Let $\delta \in \{0,1\}$ and $\mathsf{hv} = \mathsf{H}(\underline{\mathbf{A}}^\star, \underline{\mathsf{x}}^\star, \mathsf{t}) \in \mathbb{Z}_\mathsf{p}$. To construct the special simulation-sound tLM-NIZK proof system, we establish the following relations:

- $\left[\mathbf{A}_i\right]_1 \ulcorner \bar{\mathbf{w}} \urcorner - \left[\mathsf{x}\right]_1 = \left[0\right]_\mathsf{T}$ // $\mathsf{x} \in \mathcal{L}_{\mathbf{A}_i}$
- $(\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1) - \left[\bar{\mathbf{A}}^\intercal\right]_1 \ulcorner \bar{\mathbf{k}}^\star \urcorner = \left[0\right]_1$ // $\mathsf{link}(\mathbf{A}_i, \mathsf{x}, \mathbf{A}^\star, \mathsf{x}^\star)$
- $(\left[\underline{\mathsf{x}}^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1) - \left[\bar{\mathsf{x}}^\intercal\right]_1 \ulcorner \bar{\mathbf{k}}^\star \urcorner = \left[0\right]_1$ // $\mathsf{link}(\mathbf{A}_i, \mathsf{x}, \mathbf{A}^\star, \mathsf{x}^\star)$

- For $j \in [2k]$, $\left[\mathbf{t}^\intercal\right]_1 \bullet \left[\delta \mathbf{u}_j\right]_2 = \left[\delta \mathbf{t}^\intercal\right]_1 \bullet \left[\delta \mathbf{u}_j\right]_2$ // $\delta \in \{0,1\}$
- $\left[\mathbf{s}^\intercal\right]_1 \bullet \left[\mathbf{B}\right]_2 = \left[\delta \mathbf{t}^\intercal\right]_1 \bullet \left[(\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)\mathbf{B}\right]_2$ // $\mathsf{Ver}(\mathsf{svk}, \mathsf{hv}, \sigma) = 1$

To provide intuition for the last two verification equations, consider the goal of enabling the simulation of a proof that demonstrates knowledge of a signature for an honest user. As noted in [GS08], the Groth-Sahai proof for non-homogeneous bilinear equations requires modifications to facilitate simulation. Specifically, we introduce a binary variable $\delta \in \{0,1\}$ and its corresponding variable $\delta\mathbf{t}$ to restore homogeneity to the equation fourth. To ensure the well-formedness of the commitment to $\delta\mathbf{t}$, we incorporate the third equation. However, the quadratic nature of the third statement necessitates $2k$ distinct equations to complete the proof.

Analogous to the one-time special simulation sound tLM-NIZK proof system, we employ $\mathbf{z}_\mathbf{A}$ to prove the first statement and $\mathbf{z}_\sigma$ to prove the last three, ensuring that $\mathbf{z}_\mathbf{A} + \mathbf{z}_\sigma = \mathbf{z} \notin \mathsf{Span}(\mathbf{D})$. This condition enforces that either the first equation or the last four equations have perfect soundness. More specifically, we need to give the commitments $\left[\mathbf{C}_\mathbf{w}\right]_2, \left[\mathbf{C}_\mathbf{k}\right]_2, \left[\mathbf{C}_\mathbf{t}\right]_2, \left[\mathbf{C}_\mathbf{s}\right]_2$ and the proofs $\left[\mathbf{P}_\mathbf{A}\right]_1, \left[\mathbf{P}_\mathbf{d}\right]_1, \left[\mathbf{P}_\mathsf{x}\right]_1, \{\left[\mathbf{P}_\mathbf{t}^j\right]_1, \left[\mathbf{Q}_\mathbf{t}^j\right]_2\}_{j\in[2k]}, \left[\mathbf{P}_\sigma\right]_2$ verifying the following equations:

- $\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}_\mathbf{w}^\intercal\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[\mathbf{z}_\mathbf{A}^\intercal\right]_2 = \left[\mathbf{P}_\mathbf{A}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
- $(\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathbf{A}}^\intercal\right]_1 \bullet \left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 = \left[\mathbf{P}_\mathbf{d}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
- $(\left[\underline{\mathsf{x}}^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathsf{x}}^\intercal\right]_1 \bullet \left[\mathbf{C}_\mathbf{k}^\intercal\right]_2 = \left[\mathbf{P}_\mathsf{x}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
- For $j \in [2k]$:
  $\left[\mathbf{y}_1\mathbf{t}^\intercal\right]_1 \bullet \left[\mathbf{u}_j\mathbf{z}_\sigma^\intercal\right]_2 - \left[\mathbf{C}_\mathbf{t}\right]_1 \bullet \left[\mathbf{u}_j\mathbf{z}_\sigma^\intercal\right]_2 = \left[\mathbf{P}_\mathbf{t}^j\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2 + \left[\mathbf{U}_1\right]_1 \bullet \left[\mathbf{Q}_\mathbf{t}^j\right]_2$
- $\left[\mathbf{C}_\mathbf{s}\right]_1 \bullet \left[\mathbf{B}\right]_2 - \left[\mathbf{C}_\mathbf{t}\right]_1 \bullet \left[(\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)\mathbf{B}\right]_2 = \left[\mathbf{U}_1\right]_1 \bullet \left[\mathbf{Q}_\sigma\right]_2$

The construction of the special simulation-sound tLM-NIZK proof system is presented in Figure 16. This proof system achieves almost tight security under the $\mathcal{D}_k$-MDDH assumption. Additionally, the detailed simulator $\mathcal{S} = (\mathsf{SSetup}, \mathsf{Sim})$ for the proof system $\mathbf{\Pi}_\mathsf{U}$ is provided in Figure 17.

The correctness of the proof system $\mathbf{\Pi}_\mathsf{U}$ is straightforward. We provide the tight proof for the zero-knowledge property Theorem 9 and special simulation soundness of the proof system $\mathbf{\Pi}_\mathsf{U}$ in Theorem 10. The detailed proof of the theorems can be found in F.

Theorem 9. *The* tLM-NIZK *proof system in Figure 16 is zero-knowledge. Concretely, for all* PPT *adversaries* $\mathcal{A}$*, there exists an* PPT *adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\mathsf{ZK}}_{\mathbf{\Pi}_\mathsf{U},\mathcal{A}}(\lambda) \leq 2\mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_k,\mathsf{GGen},\mathcal{B}}(\lambda).$$

Theorem 10. *The* tLM-NIZK *proof system in Figure 16 is special simulation sound. Concretely, for all* PPT *adversaries* $\mathcal{A}$*, there exists an* PPT *adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\mathsf{SSS}}_{\mathbf{\Pi}_\mathsf{U},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{UF\text{-}CMA}}_{\mathsf{SIG},\mathcal{B}}(\lambda).$$

Alg $\mathsf{Setup}(1^\lambda)$:

$\mathbf{D}_1, \mathbf{D}_2 \hookleftarrow \mathcal{D}_k$
$\mathbf{z} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D}_2)\right)$
$\mathbf{y}_1 \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D}_1)\right)$
$\mathbf{v}_1 \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k}\right)$; $\mathbf{z_1} = \mathbf{D}_1\mathbf{v}_1$
$\left[\mathbf{U}_1\right]_1 = \left[\mathbf{D}_1 \,|\, \mathbf{z}_1\right]_1$
$\mathsf{crs}^\vee \xleftarrow{\$} \Pi^\vee_{\mathbf{B}_0,\mathbf{B}_1}.\mathsf{Setup}(1^\lambda)$
$\mathbf{B}_0, \mathbf{B}_1 \hookleftarrow \mathcal{D}_{2k,k}$
$\mathbf{B} \hookleftarrow \mathcal{D}_k$
$\mathbf{K}_0, \mathbf{K}_1 \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{2k\times(k+1)}\right)$
$\mathsf{svk} := (\mathsf{crs}^\vee, \left[\mathbf{B}_0\right]_1, \left[\mathbf{B}\right]_1, \left[\mathbf{K}_0\mathbf{B}\right]_2, \left[\mathbf{K}_1\mathbf{B}\right]_2)$
$\mathsf{ssk} := (\mathbf{K}_0, \mathbf{K}_1)$
$\mathsf{crs}' := (\left[\mathbf{U}_1\right]_1, \left[\mathbf{D}_2\right]_2, \left[\mathbf{z}\right]_2, \mathbf{y}_1)$
return $\mathsf{crs} = (\mathsf{crs}', \mathsf{svk})$

Alg $\mathsf{Ver}(\mathsf{crs}, \mathsf{t}, \mathsf{x}, \pi)$:

$\mathsf{hv} := \mathsf{H}(\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \mathsf{t})$
$\left[\mathbf{z}_\sigma\right]_2 = \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_\mathbf{A}\right]_2$
check $\Pi^\vee_{\mathbf{B}_0,\mathbf{B}_1}.\mathsf{Ver}(\mathsf{crs}^\vee, \left[\mathbf{t}\right]_1, \pi_\mathbf{t})$
check $\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}^\intercal_\mathbf{w}\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[\mathbf{z}^\intercal_\mathbf{A}\right]_2 = \left[\mathbf{P}_\mathbf{A}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
check $(\left[\underline{\mathbf{A}}_i^\intercal - (\underline{\mathbf{A}}^\star)^\intercal\right]_1) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathbf{A}}^\intercal\right]_1 \bullet \left[\mathbf{C}^\intercal_\mathbf{k}\right]_2 = \left[\mathbf{P}_\mathbf{d}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
check $(\left[\underline{\mathsf{x}}^\intercal - (\underline{\mathsf{x}}^\star)^\intercal\right]_1) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathsf{x}}^\intercal\right]_1 \bullet \left[\mathbf{C}^\intercal_\mathbf{k}\right]_2 = \left[\mathbf{P}_\mathsf{x}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2$
for $j \in [2k]$:
  check $\mathbf{y}_1 \left[\mathbf{t}^\intercal\right]_1 \bullet \left[\mathbf{u}_j\mathbf{z}_\sigma^\intercal\right]_2 - \left[\mathbf{C}_\mathbf{t}\right]_1 \bullet \left[\mathbf{u}_j\mathbf{z}_\sigma^\intercal\right]_2 = \left[\mathbf{P}^j_\mathbf{t}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2 + \left[\mathbf{U}_1\right]_1 \bullet \left[\mathbf{Q}^j_\mathbf{t}\right]_2$
check $\left[\mathbf{C}_\mathbf{s}\right]_1 \bullet \left[\mathbf{B}\right]_2 - \left[\mathbf{C}_\mathbf{t}\right]_1 \bullet \left[(\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)\mathbf{B}\right]_2 = \left[\mathbf{U}_1\right]_1 \bullet \left[\mathbf{Q}_\sigma\right]_2$

Alg $\mathsf{Prove}(\mathsf{crs}, \mathsf{t}, \mathsf{x}, \mathbf{w})$:

$\underline{\mathbf{A}}^\star \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{1\times k}\right)$; $\underline{\mathsf{x}}^\star \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}\right)$
$\mathsf{hv} := \mathsf{H}(\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \mathsf{t})$
$\mathbf{v}_\sigma \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k}\right)$; $\mathbf{z}_\sigma := \left[\mathbf{D_2}\right]_2 \mathbf{v}_\sigma$
$\left[\mathbf{z}_\mathbf{A}\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_\sigma\right]_2$
$\mathbf{w_t} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k}\right)$; $\mathbf{t} := \left[\mathbf{B}_0\right]_1 \mathbf{w_t}$
$\pi_\mathbf{t} \xleftarrow{\$} \Pi^\vee_{\mathbf{B}_0,\mathbf{B}_1}(\mathbf{t}, \mathbf{w_t})$
$\mathbf{R}_\mathbf{w} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k\times k}\right), \mathbf{R}_\mathbf{k} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{\ell\times k}\right)$
$\mathbf{R}_\mathbf{t} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times 2k}\right)$
$\mathbf{R}_\mathbf{s} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times(k+1)}\right)$
$\{\mathbf{\Gamma}_j\}_{j\in[2k]} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times k}\right)$
$\left[\mathbf{C}^\intercal_\mathbf{w}\right]_2 := \mathbf{R}_\mathbf{w}\left[\mathbf{D}_2^\intercal\right]_2 + \mathbf{w} \cdot \left[\mathbf{z}^\intercal_\mathbf{A}\right]_2$
$\left[\mathbf{C}^\intercal_\mathbf{k}\right]_2 := \mathbf{R}_\mathbf{k}\left[\mathbf{D}_2^\intercal\right]_2$
$\left[\mathbf{C}_\mathbf{t}\right]_1 := \left[\mathbf{U}_1\right]_1 \mathbf{R}_\mathbf{t}$
$\left[\mathbf{C}_\mathbf{s}\right]_1 := \left[\mathbf{U}_1\right]_1 \mathbf{R}_\mathbf{s}$
$\left[\mathbf{P}_\mathbf{A}\right]_1 := \left[\mathbf{A}_i\right]_1 \mathbf{R}_\mathbf{w}$
$\left[\mathbf{P}_\mathbf{d}\right]_1 := \left[(\underline{\mathbf{A}}_i^\intercal - (\underline{\mathbf{A}}^\star)^\intercal)\mathbf{v}_\sigma^\intercal - \bar{\mathbf{A}}^\intercal\mathbf{R}_\mathbf{k}\right]_1$
$\left[\mathbf{P}_\mathsf{x}\right]_1 := \left[(\underline{\mathsf{x}}^\intercal - (\underline{\mathsf{x}}^\star)^\intercal)\mathbf{v}_\sigma^\intercal - \bar{\mathsf{x}}^\intercal\mathbf{R}_\mathbf{k}\right]_1$
for $j \in [2k]$:
  $\left[\mathbf{P}^j_\mathbf{t}\right]_1 := \left[\mathbf{y}_1\mathbf{t}^\intercal\mathbf{u}_j\mathbf{v}_\sigma^\intercal\right]_1 - \left[\mathbf{U}_1\mathbf{R}_\mathbf{t}\mathbf{u}_\mathbf{j}\mathbf{v}_\sigma^\intercal\right]_1 - \left[\mathbf{U}_1\mathbf{\Gamma}_j\right]_1$
  $\left[\mathbf{Q}^j_\mathbf{t}\right]_2 := \left[\mathbf{\Gamma}_j\mathbf{D}_2^\intercal\right]_2$
$\left[\mathbf{Q}_\sigma\right]_2 := \left[\mathbf{R}_\mathbf{s}\mathbf{B} - \mathbf{R}_\mathbf{t}(\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)\mathbf{B}\right]_2$
$\pi' = (\mathbf{z}_\mathbf{A}, \mathbf{C}_\mathbf{w}, \mathbf{C}_\mathbf{k}, \mathbf{C}_\mathbf{t}, \mathbf{C}_\mathbf{s}, \mathbf{P}_\mathbf{A}, \mathbf{P}_\mathbf{d}, \mathbf{P}_\mathsf{x}, \{\mathbf{P}^j_\mathbf{t}, \mathbf{Q}^j_\mathbf{t}\}_{j\in[2k]}, \mathbf{Q}_\sigma)$
return $\pi = (\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \pi_\mathbf{t}, \pi')$

Fig. 16. Construction of unbounded special simulation sound tLM-NIZK proof system

F Proof of tLM-NIZK

F.1 Proof of Theorem 6

Theorem 6. *The proof system* $\mathbf{\Pi}_\mathsf{OT}$ *in Figure 11 is zero-knowledge. Concretely, for all* PPT *adversaries* $\mathcal{A}$*, there exists an* PPT *adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\mathsf{ZK}}_{\mathbf{\Pi}_\mathsf{OT},\mathcal{A}}(\lambda) \leq 2\mathsf{Adv}^{\mathsf{KerMDH}}_{\mathcal{D}_k,\mathsf{GGen},\mathcal{B}}(\lambda).$$

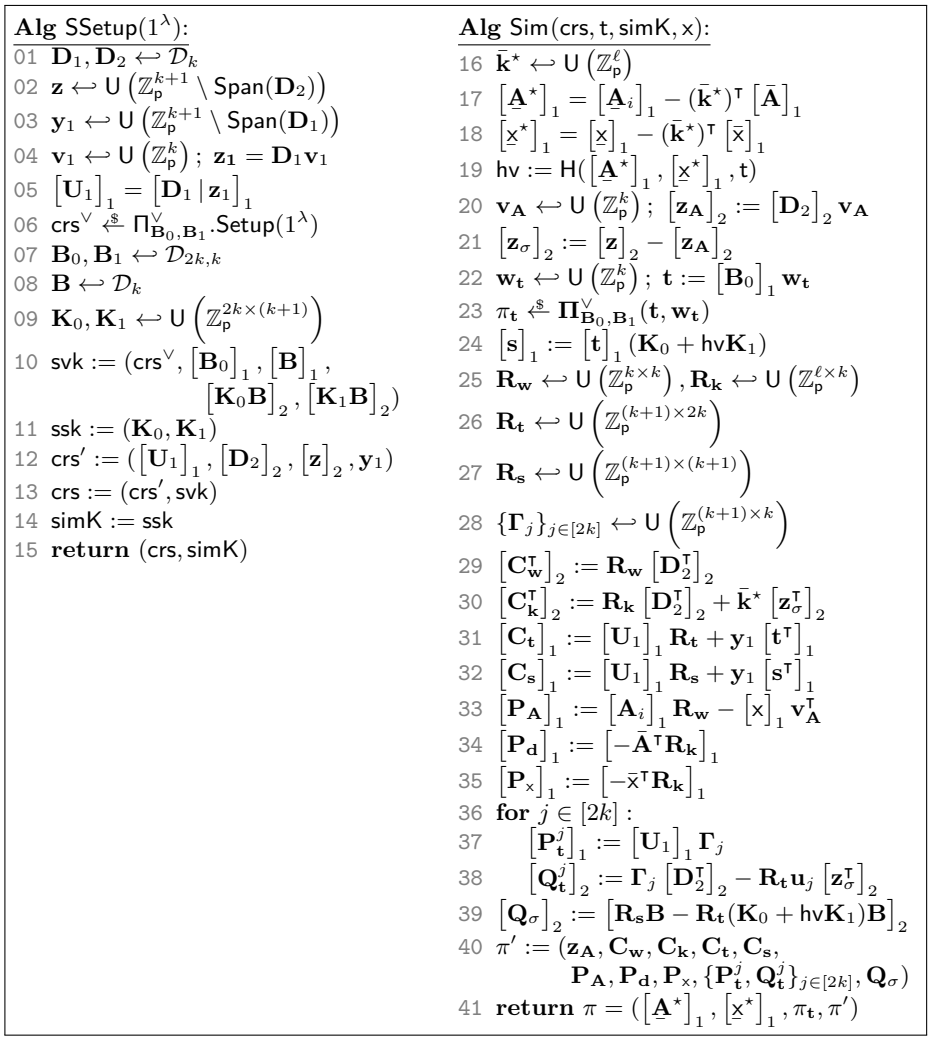

Fig. 17. Construction of the simulator $\mathcal{S} = (\mathsf{SSetup}, \mathsf{Sim})$ for the unbounded special simulation sound tLM-NIZK proof system $\mathbf{\Pi}_\mathsf{U}$.

Proof. We present the construction of the simulator $\mathcal{S} = (\mathsf{SSetup}, \mathsf{Sim})$ in Figure 13. We first observe that the crs generated by SSetup is identical to the one generated by Setup in the real scheme.

We prove the zero-knowledge property of $\mathbf{\Pi}_{\mathsf{OT}}$ by introducing hybrid games $\mathbf{G}_0, \ldots, \mathbf{G}_5$, where in $\mathbf{G}_0$ the adversary interacts with the real proof system, and in $\mathbf{G}_5$ the adversary interacts with the simulated proof system. We show that the view of the adversary in $\mathbf{G}_0$ and $\mathbf{G}_5$ are computationally indistinguishable. The pseudocode of the hybrid games is given in Figure 18.

Game $\mathbf{G}_0$: This is the original security game where $\mathcal{A}$ interacts with the Setup

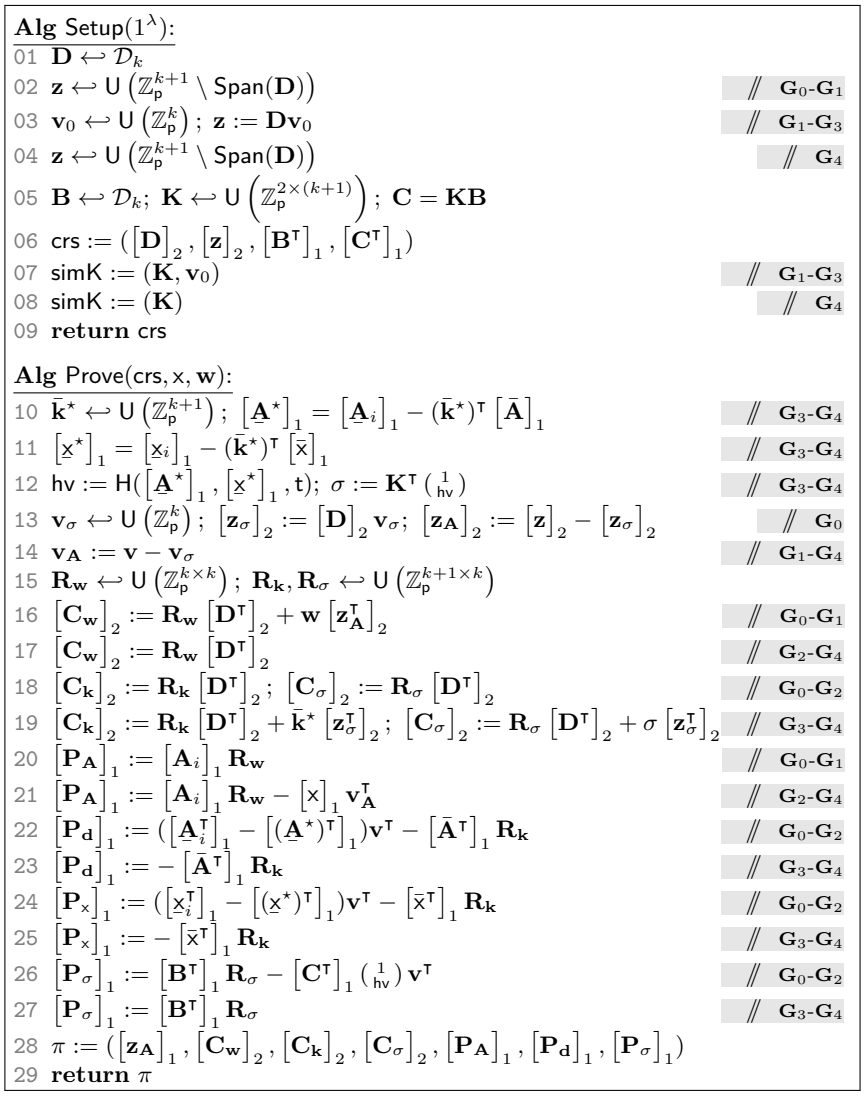

Fig. 18. Hybrid games in proving the zero-knowledge property of one-time special simulation sound tLM-NIZK proof system.

and Prove oracles.

$$\mathsf{pr}_0 = \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Prove}}}(1^\lambda, \mathsf{crs}) = 1 \;\middle|\; \mathsf{crs} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)\right].$$

Game $\mathbf{G}_1$: In this game, instead of sample $\mathbf{z}$ uniformly at random from $\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D})$, we first sample $\mathbf{v}_0$ and set $\mathbf{z} = \mathbf{D}\mathbf{v}_0$. The only difference between $\mathbf{G}_2$

and $\mathbf{G}_1$ is the distribution of $\mathbf{z}$, it reduce to the $\mathcal{D}_k$-MDDH problem.

$$|\mathsf{pr}_1 - \mathsf{pr}_0| \leq \mathsf{Adv}^{\mathcal{D}_k\text{-MDDH}}_{\mathbb{G}_1,\mathcal{B}}(\lambda).$$

Game $\mathbf{G}_2$: In game $\mathbf{G}_2$, we change the way of computing $\left[\mathbf{C_w}\right]_2$ and $\left[\mathbf{P_A}\right]_1$. In the previous game, we have

$$\left[\mathbf{C_w}\right]_2 = \mathbf{R_w}\left[\mathbf{D}^\intercal\right]_2 + \mathbf{w}\left[\mathbf{z}_\mathbf{A}^\intercal\right]_2, \qquad \left[\mathbf{P_A}\right]_1 = \left[\mathbf{A}\right]_1 \mathbf{R_w}.$$

We compute them in $\mathbf{G}_2$ as follows:

$$\left[\mathbf{C_w}\right]_2 = \mathbf{R_w}\left[\mathbf{D}^\intercal\right]_2, \qquad \left[\mathbf{P_A}\right]_1 = \left[\mathbf{A}\right]_1 \mathbf{R_w} - \left[\mathsf{x}\right]_1 \mathbf{v}_\mathbf{A}^\intercal,$$

where $\mathbf{v_A} = \mathbf{v_0} - \mathbf{v}_\sigma$. Since we have $\mathbf{z_A} = \mathbf{D}\mathbf{v_A}$, the change in $\mathbf{G}_2$ can be seen as changing $\mathbf{R_w}$ to $\mathbf{R_w} - \mathbf{w}\mathbf{v}_\mathbf{A}^\intercal$. Therefore, we have

$$|\mathsf{pr}_2 - \mathsf{pr}_1| = 0.$$

Game $\mathbf{G}_3$: We change the way of computing $\left[\mathbf{C_k}\right]_2, \left[\mathbf{C}_\sigma\right]_2, \left[\mathbf{P_d}\right]_1, \left[\mathbf{P}_\mathsf{x}\right]_1, \left[\mathbf{P}_\sigma\right]_1$ in $\mathbf{G}_3$. We first compute two row vectors $\left[\underline{\mathbf{A}}^\star\right]_1$ and $\left[\underline{\mathsf{x}}^\star\right]_1$ and their hash value hv, then compute the one-time signature $\sigma = \mathbf{K}^\intercal\left(\begin{smallmatrix}1\\ \mathsf{hv}\end{smallmatrix}\right)$. We recall that in $\mathbf{G}_3$, we have

$$\begin{aligned}
&\left[\mathbf{C_k}\right]_2 = \mathbf{R_k}\left[\mathbf{D}^\intercal\right]_2, && \left[\mathbf{C}_\sigma\right]_2 = \mathbf{R}_\sigma\left[\mathbf{D}^\intercal\right]_2,\\
&\left[\mathbf{P_d}\right]_1 = (\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1)\mathbf{v}^\intercal - \left[\bar{\mathbf{A}}^\intercal\right]_1 \mathbf{R_k},\\
&\left[\mathbf{P}_\mathsf{x}\right]_1 = (\left[\underline{\mathsf{x}}_i^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1)\mathbf{v}^\intercal - \left[\bar{\mathsf{x}}^\intercal\right]_1 \mathbf{R_k},\\
&\left[\mathbf{P}_\sigma\right]_1 = \left[\mathbf{B}^\intercal\right]_1 \mathbf{R}_\sigma - \left[\mathbf{C}^\intercal\right]_1 \begin{pmatrix}1\\ \mathsf{hv}\end{pmatrix} \mathbf{v}^\intercal.
\end{aligned}$$

We compute them in $\mathbf{G}_3$ as follows:

$$\begin{aligned}
&\left[\mathbf{C_k}\right]_2 = \mathbf{R_k}\left[\mathbf{D}^\intercal\right]_2 + \mathbf{k}^\star\left[\mathbf{z}_\sigma^\intercal\right]_2, && \left[\mathbf{C}_\sigma\right]_2 = \mathbf{R}_\sigma\left[\mathbf{D}^\intercal\right]_2 + \sigma\left[\mathbf{z}_\sigma^\intercal\right]_2,\\
&\left[\mathbf{P_d}\right]_1 = -\left[\bar{\mathbf{A}}^\intercal\right]_1 \mathbf{R_k}, && \left[\mathbf{P}_\mathsf{x}\right]_1 = -\left[\bar{\mathsf{x}}^\intercal\right]_1 \mathbf{R_k},\\
&\left[\mathbf{P}_\sigma\right]_1 = \left[\mathbf{B}^\intercal\right]_1 \mathbf{R}_\sigma.
\end{aligned}$$

Since $\mathbf{z}_\sigma = \mathbf{D}\mathbf{v}$, the above change is oblivious to the adversary.

Therefore, we have

$$|\mathsf{pr}_3 - \mathsf{pr}_2| = 0.$$

Game $\mathbf{G}_4$: We change again the way of computing $\mathbf{z_A}, \mathbf{z}_\sigma$. We can observe that in $\mathbf{G}_3$, we do not use the value of $\mathbf{v}$ to compute $\left[\mathbf{C_k}\right]_2, \left[\mathbf{C}_\sigma\right]_2, \left[\mathbf{P_d}\right]_1, \left[\mathbf{P}_\mathsf{x}\right]_1, \left[\mathbf{P}_\sigma\right]_1$. Therefore, we can set $\mathbf{z}_\sigma$ as $\mathbf{z} - \mathbf{z_A}$ with $\mathbf{z}$ sample uniformly at random from

$\mathbb{Z}_{\mathsf{p}}^{k+1} \setminus \mathsf{Span}(\mathbf{D})$. We can notice that the only change in $\mathbf{G}_4$ is the distribution of $\mathbf{z}$, therefore we have

$$|\mathsf{pr}_4 - \mathsf{pr}_3| \leq \mathsf{Adv}^{\mathcal{D}_k\text{-MDDH}}_{\mathbb{G}_1,\mathcal{B}}(\lambda).$$

Finally, we can observe that Setup and Prove algorithm in $\mathbf{G}_4$ is identical to the SSetup and Sim algorithm. Therefore, we have

$$\mathsf{pr}_4 = \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Sim}}}(1^\lambda, \mathsf{crs}) = 1 \;\middle|\; \mathsf{crs} \xleftarrow{\$} \mathsf{SSetup}(1^\lambda)\right].$$

Using the triangle inequality over all above hybrids, we have

$$|\mathsf{pr}_0 - \mathsf{pr}_4| \leq 2\mathsf{Adv}^{\mathcal{D}_k\text{-MDDH}}_{\mathbb{G}_1,\mathcal{B}}(\lambda).$$

Therefore, we conclude the proof. □

F.2 proof of Theorem 7

Theorem 7. *The one-time* tLM-NIZK *proof system presented in Figure 11 satisfies special simulation soundness.*

Proof. We begin by observing that the vector $\mathbf{z}$ within the common reference string (CRS), generated by the SSetup algorithm, is designed to lie outside the linear span of $\mathbf{D}$. Consequently, either $\mathbf{z}_{\mathbf{A}}$ or $\mathbf{z}_\sigma$ (or both) must also lie outside the linear span of $\mathbf{D}$. This implies the existence of a vector ξ, which the challenger can derive without affecting the adversary's view, such that $\xi^\intercal \mathbf{D} = 0$, and either $\xi^\intercal \mathbf{z}_{\mathbf{A}} = 1$ or $\xi^\intercal \mathbf{z}_\sigma = 1$. We consider these two scenarios separately:

- **Case $\mathbf{z}_{\mathbf{A}} \notin \mathsf{Span}(\mathbf{D})$:** The first verification equation is

$$\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}_{\mathbf{w}}\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[\mathbf{z}_{\mathbf{A}}^\intercal\right]_2 = \left[\mathbf{P}_{\mathbf{A}}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2.$$

  Right-multiplying both sides by ξ yields

$$\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}_{\mathbf{w}} \cdot \xi\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[1\right]_2 = \left[\mathbf{P}_{\mathbf{A}}\right]_1 \bullet \left[0\right]_2.$$

  Therefore, in this case, we conclude that $\mathsf{x} \in \mathsf{Span}(\mathbf{A})$.
- **Case $\mathbf{z}_\sigma \notin \mathsf{Span}(\mathbf{D})$:** In this case, we examine the other three verification equations:

$$\begin{aligned}
\left(\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1\right) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathbf{A}}^\intercal\right]_1 \bullet \left[\mathbf{C}_{\mathbf{k}}\right]_2 &= \left[\mathbf{P}_{\mathbf{d}}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2, \\
\left(\left[\underline{\mathsf{x}}_i^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1\right) \bullet \left[\mathbf{z}_\sigma^\intercal\right]_2 - \left[\bar{\mathsf{x}}^\intercal\right]_1 \bullet \left[\mathbf{C}_{\mathbf{k}}\right]_2 &= \left[\mathbf{P}_{\mathsf{x}}\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2, \\
\left[\mathbf{B}^\intercal\right]_1 \bullet \left[\mathbf{C}_\sigma\right]_2 - \left[\mathbf{C}^\intercal\right]_1 \bullet \left[\begin{pmatrix} 1 \\ \mathsf{hv} \end{pmatrix} \mathbf{z}_\sigma^\intercal\right]_2 &= \left[\mathbf{P}_\sigma\right]_1 \bullet \left[\mathbf{D}_2^\intercal\right]_2.
\end{aligned}$$

  Right-multiplying both equations by ξ, we obtain

$$\begin{aligned}
\left(\left[\underline{\mathbf{A}}_i^\intercal\right]_1 - \left[(\underline{\mathbf{A}}^\star)^\intercal\right]_1\right) \bullet \left[1\right]_2 - \left[\bar{\mathbf{A}}^\intercal\right]_1 \bullet \left[\mathbf{C}_{\mathbf{k}} \cdot \xi\right]_2 &= \left[\mathbf{P}_{\mathbf{d}}\right]_1 \bullet \left[0\right]_2, \\
\left(\left[\underline{\mathsf{x}}_i^\intercal\right]_1 - \left[(\underline{\mathsf{x}}^\star)^\intercal\right]_1\right) \bullet \left[1\right]_2 - \left[\bar{\mathsf{x}}^\intercal\right]_1 \bullet \left[\mathbf{C}_{\mathbf{k}} \cdot \xi\right]_2 &= \left[\mathbf{P}_{\mathsf{x}}\right]_1 \bullet \left[0\right]_2, \\
\left[\mathbf{B}^\intercal\right]_1 \bullet \left[\mathbf{C}_\sigma \cdot \xi\right]_2 - \left[\mathbf{C}^\intercal\right]_1 \bullet \left[\begin{pmatrix} 1 \\ \mathsf{hv} \end{pmatrix}\right]_2 &= \left[\mathbf{P}_\sigma\right]_1 \bullet \left[0\right]_2.
\end{aligned}$$

From these equations, we can extract $[\mathbf{C_k} \cdot \xi]_2$ such that $([\underline{\mathbf{A}}_i^\intercal]_1 - [(\underline{\mathbf{A}}^\star)^\intercal]_1) \bullet [1]_2 = [\bar{\mathbf{A}}^\intercal]_1 \bullet [\mathbf{C_k} \cdot \xi]_2$ and $([\underline{x}_i^\intercal]_1 - [(\underline{x}^\star)^\intercal]_1) \bullet [1]_2 = [\bar{x}^\intercal]_1 \bullet [\mathbf{C_k} \cdot \xi]_2$. Furthermore, $[\mathbf{C}_\sigma \cdot \xi]_2$ constitutes a valid one-time signature for hv. We distinguish between two subcases:

1. If $\mathsf{hv} = \mathsf{H}([\underline{\mathbf{A}}^\star]_1, [\underline{x}^\star]_1, \mathsf{t})$ where $[\underline{\mathbf{A}}^\star]_1$ and $[\underline{x}^\star]_1$ differs from the one generated in the challenge ciphertext, then the adversary has successfully broken the one-time unforgeability of the underlying signature scheme.
2. If $[\underline{\mathbf{A}}^\star]_1$ and $[\underline{x}^\star]_1$ equals the one generated in the challenge ciphertext, the challenger could have generated $[\underline{\mathbf{A}}^\star]_1$ and $[\underline{x}^\star]_1$ as $[\underline{\mathbf{A}}^\star]_1 = (\mathbf{k}')^\intercal [\bar{\mathbf{A}}]_1$ and $[\underline{x}^\star]_1 = (\mathbf{k}'')^\intercal [\bar{x}]_1$. Consequently, $[\mathbf{C_k} \cdot \xi]_2 - [\mathbf{k}']_2$ equals the $\mathbb{G}_2$ representation of the re-encryption token from the challenge user to user j.

Therefore, we conclude that the tLM-NIZK proof system Figure 13 satisfies one-time special simulation soundness. □

F.3 Proof of Theorem 9

Theorem 9. *The* tLM-NIZK *proof system in Figure 16 is zero-knowledge. Concretely, for all* PPT *adversaries* $\mathcal{A}$*, there exists an* PPT *adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\mathsf{ZK}}_{\mathbf{\Pi}_\mathsf{U},\mathcal{A}}(\lambda) \leq 2\mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_k,\mathsf{GGen},\mathcal{B}}(\lambda).$$

Proof. We establish the zero-knowledge property of $\mathbf{\Pi}_\mathsf{U}$ through a sequence of hybrid games. As the verification algorithm remains consistent across these games, we present only the modifications to the Setup and Prove algorithms for conciseness. The pseudocode for these hybrid games is provided in Figure 19 and Figure 20.

Alg $\mathsf{Setup}(1^\lambda)$:
01 $\mathbf{D}_1, \mathbf{D}_2 \hookleftarrow \mathcal{D}_k$
02 $\mathbf{z} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D}_2)\right)$ // $\mathbf{G}_0, \mathbf{G}_6$
03 $\mathbf{v}_0 \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k}\right)$; $\mathbf{z} := \mathbf{D}_2\mathbf{v}_0$ // $\mathbf{G}_1$-$\mathbf{G}_5$
04 $\mathbf{y}_1 \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k+1} \setminus \mathsf{Span}(\mathbf{D}_1)\right)$; $\mathbf{v}_1 \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k}\right)$; $\mathbf{z_1} = \mathbf{D}_1\mathbf{v}_1$
05 $[\mathbf{U}_1]_1 = [\mathbf{D}_1 \,|\, \mathbf{z}_1]_1$; $(\mathsf{svk}, \mathsf{ssk}) \xleftarrow{\$} \mathsf{Sig.Setup}(1^\lambda)$
06 $\mathsf{simK} := (\mathsf{ssk}, \mathbf{v}_0)$ // $\mathbf{G}_2$-$\mathbf{G}_6$
07 **return** $\mathsf{crs} := (\mathsf{crs}' := ([\mathbf{U}_1]_1, [\mathbf{D}_2]_2, [\mathbf{z}]_2, \mathbf{y}_1), \mathsf{svk})$

Fig. 19. Hybrid security games of unbounded special simulation sound tLM-NIZK proof system $\mathbf{\Pi}_\mathsf{U}$ in Setup.

Game $\mathbf{G}_0$: In this initial game, the adversary interacts directly with the original Setup and Prove algorithms. The winning probability of the adversary in $\mathbf{G}_0$ is therefore:

$$\mathsf{pr}_0 = \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Prove}}}(1^\lambda, \mathsf{crs}) = 1 \;\middle|\; \mathsf{crs} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)\right].$$

Alg $\mathsf{Prove}(\mathsf{crs}, \mathsf{t}, \mathsf{x}, \mathbf{w})$:

01 $\underline{\mathbf{A}}^\star \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{1\times k}\right)$ $\underline{\mathsf{x}}^\star \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{1\times k}\right)$ // $\mathbf{G}_0$-$\mathbf{G}_3$

02 $\underline{\mathbf{A}}^\star := \underline{\mathbf{A}}_i - (\bar{\mathbf{k}}^\star)^\intercal \bar{\mathbf{A}}$ $\underline{\mathsf{x}}^\star := \underline{\mathsf{x}}_i - (\bar{\mathbf{k}}^\star)^\intercal \bar{\mathsf{x}}$ // $\mathbf{G}_4$-$\mathbf{G}_6$

03 $\mathsf{hv} := \mathsf{H}(\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \mathsf{t})$

04 $\mathbf{v}_\sigma \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^k\right)$; $\mathbf{z}_\sigma := \left[\mathbf{D}_2\right]_2 \mathbf{v}_\sigma$ // $\mathbf{G}_0$-$\mathbf{G}_4$

05 $\mathbf{v}_\mathbf{A} := \mathbf{v} - \mathbf{v}_\sigma$ // $\mathbf{G}_3$-$\mathbf{G}_4$

06 $\mathbf{v}_\mathbf{A} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^k\right)$; $\mathbf{z}_\mathbf{A} := \left[\mathbf{D}_2\right]_2 \mathbf{v}_\mathbf{A}$ // $\mathbf{G}_5$-$\mathbf{G}_6$

07 $\left[\mathbf{z}_\mathbf{A}\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_\sigma\right]_2$ // $\mathbf{G}_0$-$\mathbf{G}_4$

08 $\left[\mathbf{z}_\sigma\right]_2 := \left[\mathbf{z}\right]_2 - \left[\mathbf{z}_\mathbf{A}\right]_2$ // $\mathbf{G}_5$-$\mathbf{G}_6$

09 $\mathbf{w}_\mathbf{t} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^k\right)$; $\mathbf{t} := \left[\mathbf{B}_0\right]_1 \mathbf{w}_\mathbf{t}$; $\pi_\mathbf{t} \xleftarrow{\$} \boldsymbol{\Pi}^\vee_{\mathbf{B}_0,\mathbf{B}_1}(\mathbf{t}, \mathbf{w}_\mathbf{t})$

10 $\mathbf{R}_\mathbf{w} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{k\times k}\right), \mathbf{R}_\mathbf{k} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{\ell\times k}\right), \mathbf{R}_\mathbf{t} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times 2k}\right)$

11 $\mathbf{R}_\mathbf{s} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times(k+1)}\right), \{\boldsymbol{\Gamma}_i\}_{i\in[2k]} \hookleftarrow \mathsf{U}\left(\mathbb{Z}_\mathsf{p}^{(k+1)\times k}\right)$

12 $\left[\mathbf{C}_\mathbf{w}\right]_2 := \mathbf{R}_\mathbf{w}\left[\mathbf{D}_2^\intercal\right]_2 + \mathbf{w} \cdot \left[\mathbf{z}_\mathbf{A}^\intercal\right]_2$ // $\mathbf{G}_0$-$\mathbf{G}_2$

13 $\left[\mathbf{C}_\mathbf{w}\right]_2 := \mathbf{R}_\mathbf{w}\left[\mathbf{D}_2^\intercal\right]_2$ // $\mathbf{G}_3$-$\mathbf{G}_6$

14 $\left[\mathbf{C}_\mathbf{k}\right]_2 := \mathbf{R}_\mathbf{k}\left[\mathbf{D}_2^\intercal\right]_2$; $\left[\mathbf{C}_\mathbf{t}\right]_1 := \left[\mathbf{U}_1\right]_1 \mathbf{R}_\mathbf{t}$; $\left[\mathbf{C}_\mathbf{s}\right]_1 := \left[\mathbf{U}_1\right]_1 \mathbf{R}_\mathbf{s}$ // $\mathbf{G}_0$-$\mathbf{G}_3$

15 $\left[\mathbf{s}\right]_1 = \left[\mathbf{t}\right]_1 (\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)$; $\left[\mathbf{C}_\mathbf{k}\right]_2 := \mathbf{R}_\mathbf{k}\left[\mathbf{D}_2^\intercal\right]_2 + \bar{\mathbf{k}}^\star\left[\mathbf{z}_\sigma^\intercal\right]_2$ // $\mathbf{G}_4$-$\mathbf{G}_6$

16 $\left[\mathbf{C}_\mathbf{t}\right]_1 := \left[\mathbf{U}_1\right]_1 \mathbf{R}_\mathbf{t} + \mathbf{y}_1\left[\mathbf{t}^\intercal\right]_1$; $\left[\mathbf{C}_\mathbf{s}\right]_1 := \left[\mathbf{U}_1\right]_1 \mathbf{R}_\mathbf{s} + \mathbf{y}_1\left[\mathbf{s}^\intercal\right]_1$ // $\mathbf{G}_4$-$\mathbf{G}_6$

17 $\left[\mathbf{P}_\mathbf{A}\right]_1 := \left[\mathbf{A}_i\right]_1 \mathbf{R}_\mathbf{w}$ // $\mathbf{G}_0$-$\mathbf{G}_2$

18 $\left[\mathbf{P}_\mathbf{A}\right]_1 := \left[\mathbf{A}_i\right]_1 \mathbf{R}_\mathbf{w} - \left[\mathsf{x}\right]_1 \mathbf{v}_\mathbf{A}^\intercal$ // $\mathbf{G}_3$-$\mathbf{G}_6$

19 $\left[\mathbf{P}_\mathbf{d}\right]_1 := \left[(\underline{\mathbf{A}}_i - \underline{\mathbf{A}}^\star)^\intercal\right]_1 \mathbf{v}_\sigma^\intercal - \left[\bar{\mathbf{A}}^\intercal\right]_1 \mathbf{R}_\mathbf{k}$ // $\mathbf{G}_0$-$\mathbf{G}_3$

20 $\left[\mathbf{P}_\mathsf{x}\right]_1 := \left[(\underline{\mathsf{x}} - \underline{\mathsf{x}}^\star)^\intercal\right]_1 \mathbf{v}_\sigma^\intercal - \left[\bar{\mathsf{x}}^\intercal\right]_1 \mathbf{R}_\mathbf{k}$ // $\mathbf{G}_0$-$\mathbf{G}_3$

21 $\left[\mathbf{P}_\mathbf{d}\right]_1 := \left[-\bar{\mathbf{A}}^\intercal \mathbf{R}_\mathbf{k}\right]_1$ $\left[\mathbf{P}_\mathsf{x}\right]_1 := \left[-\bar{\mathsf{x}}^\intercal \mathbf{R}_\mathbf{k}\right]_1$ // $\mathbf{G}_4$-$\mathbf{G}_6$

22 **for** $j \in [2k]$:

23 $\quad \left[\mathbf{P}_\mathbf{t}^j\right]_1 := \left[\mathbf{y}_1 \mathbf{t}^\intercal \mathbf{u}_j \mathbf{v}_\sigma^\intercal\right]_1 - \left[\mathbf{U}_1 \mathbf{R}_\mathbf{t} \mathbf{u}_j \mathbf{v}_\sigma^\intercal\right]_1 - \left[\mathbf{U}_1 \boldsymbol{\Gamma}_j\right]_1$ // $\mathbf{G}_0$-$\mathbf{G}_3$

24 $\quad \left[\mathbf{Q}_\mathbf{t}^j\right]_2 := \left[\boldsymbol{\Gamma}_j \mathbf{D}_2^\intercal\right]_2$ // $\mathbf{G}_0$-$\mathbf{G}_3$

25 $\quad \left[\mathbf{P}_\mathbf{t}^j\right]_1 := \left[\mathbf{U}_1 \boldsymbol{\Gamma}_j\right]_1$; $\left[\mathbf{Q}_\mathbf{t}^j\right]_2 := \left[\boldsymbol{\Gamma}_j \mathbf{D}_2^\intercal\right]_2 - \mathbf{R}_\mathbf{t} \mathbf{u}_j \left[\mathbf{z}_\sigma^\intercal\right]_2$ // $\mathbf{G}_4$-$\mathbf{G}_6$

26 $\left[\mathbf{Q}_\sigma\right]_2 := \left[\mathbf{R}_\mathbf{s}\mathbf{B} - \mathbf{R}_\mathbf{t}(\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)\mathbf{B}\right]_2$

27 $\pi' = (\mathbf{z}_\mathbf{A}, \mathbf{C}_\mathbf{w}, \mathbf{C}_\mathbf{k}, \mathbf{C}_\mathbf{t}, \mathbf{C}_\mathbf{s}, \mathbf{P}_\mathbf{A}, \mathbf{P}_\mathbf{d}, \mathbf{P}_\mathsf{x}, \{\mathbf{P}_\mathbf{t}^j, \mathbf{Q}_\mathbf{t}^j\}_{j\in[2k]}, \mathbf{Q}_\sigma)$

28 **return** $\pi = (\left[\underline{\mathbf{A}}^\star\right]_1, \left[\underline{\mathsf{x}}^\star\right]_1, \pi_\mathbf{t}, \pi')$

Fig. 20. Hybrid security games of unbounded special simulation sound $\mathsf{tLM\text{-}NIZK}$ proof system $\boldsymbol{\Pi}_\mathsf{U}$ in Prove.

Game $\mathbf{G}_1$: In $\mathbf{G}_1$, we change the distribution of $\mathbf{z}$. Instead of being uniformly sampled from $\mathbb{Z}_\mathsf{p}^\ell \setminus \mathsf{Span}(\mathbf{D}_2)$, we sample $\mathbf{v}_0$ uniformly from $\mathbb{Z}_\mathsf{p}^k$ and set $\mathbf{z} = \mathbf{D}_2\mathbf{v}_0$. Under the $\mathcal{D}_{\ell,k}$-MDDH assumption, $\mathbf{G}_0$ and $\mathbf{G}_1$ is indistinguishable for the adversary, and we have:

$$|\mathsf{pr}_1 - \mathsf{pr}_0| \leq \mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G}_2,\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_2$: In this game, we introduce a simulation key simK that corresponds to the signing key of the underlying signature scheme and $\mathbf{v}_0$. Since this simulation key is generated internally and remains inaccessible to the adversary, the adversary's view remains unchanged. Consequently, we have:

$$|\mathsf{pr}_2 - \mathsf{pr}_1| = 0.$$

Game $\mathbf{G}_3$: In $\mathbf{G}_3$, we first compute $\mathbf{v}_\mathbf{A} := \mathbf{v} - \mathbf{v}_\sigma$. We can notice that we have $\mathbf{z}_\mathbf{A} = \mathbf{D}_2\mathbf{v}_\mathbf{A} \in \mathsf{Span}(\mathbf{D}_2)$. Then we generate $\left[\mathbf{C}_\mathbf{w}\right]_2$ as $\mathbf{R}_\mathbf{w}\left[\mathbf{D}_2^\intercal\right]_2$ without using $\mathbf{w}$, and compute $\left[\mathbf{P}_\mathbf{A}\right]_1$ using the value of $\mathbf{v}_\mathbf{A}$ as $\left[\mathbf{P}_\mathbf{A}\right]_1 := \left[\mathbf{A}\right]_1\mathbf{R}_\mathbf{w} - \left[\mathsf{x}\right]_1\mathbf{v}_\mathbf{A}^\intercal$. We can notice that when $\mathbf{z}_\mathbf{A} \in \mathsf{Span}(\mathbf{D}_2)$, $\left[\mathbf{C}_\mathbf{w}\right]_2 = \mathbf{R}_\mathbf{w}\left[\mathbf{D}_2^\intercal\right]_2 + \mathbf{w}\left[\mathbf{z}_\mathbf{A}^\intercal\right]_2$ is a perfectly hiding commitment of $\mathbf{w}$. Therefore, the adversary's view in $\mathbf{G}_2$ and $\mathbf{G}_3$ is identical. Thus, we have:

$$|\mathsf{pr}_3 - \mathsf{pr}_2| = 0.$$

Game $\mathbf{G}_4$: In this game, we generate the proof of the last three equations honestly using $\mathbf{z}_\sigma$ and $\mathbf{w}_\mathbf{t}$, without using the value of $\mathbf{v}_\sigma$. Similar to the argument in the previous game, the adversary's view in $\mathbf{G}_3$ and $\mathbf{G}_4$ is identical. Thus, we have:

$$|\mathsf{pr}_4 - \mathsf{pr}_3| = 0.$$

Game $\mathbf{G}_5$: In this game, we change the way of generating $\mathbf{z}_\mathbf{A}, \mathbf{z}_\sigma$. Instead of computing $\mathbf{z}_\mathbf{A}$ using $\mathbf{v}_\mathbf{A}$, we compute $\left[\mathbf{z}_\mathbf{A}\right]_1$ as $\left[\mathbf{z}\right]_1 - \left[\mathbf{z}_\sigma\right]_1$. Note that the adversary's view is unchanged compared to $\mathbf{G}_4$. Thus, we have:

$$|\mathsf{pr}_5 - \mathsf{pr}_4| = 0.$$

Game $\mathbf{G}_6$: In this game, we change the distribution of $\mathbf{z}$ in the Setup algorithm. If there is an adversary in distinguishing $\mathbf{G}_5$ and $\mathbf{G}_6$, we can construct an adversary $\mathcal{B}$ that can break the $\mathcal{D}_k$-MDDH assumption. We have:

$$|\mathsf{pr}_6 - \mathsf{pr}_5| \leq \mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G}_2,\mathcal{B}}(1^\lambda).$$

In $\mathbf{G}_6$, the adversary's view is identical to the one in interacting with the simulator $\mathcal{S}$. The simulator generates the same crs and simK as in $\mathbf{G}_6$, and the proof π is generated using the same method as in $\mathbf{G}_6$. Therefore, we have:

$$\mathsf{pr}_6 = \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Sim}}}(1^\lambda, \mathsf{crs}) = 1 \;\middle|\; \mathsf{crs} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)\right].$$

Summarizing the above games, we have:

$$\begin{aligned}|\mathsf{pr}_6 - \mathsf{pr}_0| &\leq \mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G}_2,\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G}_2,\mathcal{B}}(1^\lambda)\\ &= 2\mathsf{Adv}^{\mathsf{MDDH}}_{\mathcal{D}_{\ell,k},\mathbb{G}_2,\mathcal{B}}(1^\lambda).\end{aligned}$$

Thus, we conclude that the $\mathsf{tLM\text{-}NIZK}$ proof system $\mathbf{\Pi}_\mathsf{U}$ is zero-knowledge. □

F.4 Proof of Theorem 10

Theorem 10. *The* tLM-NIZK *proof system in Figure 11 is special simulation sound. Concretely, for all* PPT *adversaries* $\mathcal{A}$*, there exists an* PPT *adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\mathsf{SSS}}_{\mathbf{\Pi}_{\cup},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{Unforg}}_{\mathsf{SIG},\mathcal{B}}(\lambda).$$

Proof. We begin by observing that the vector $\mathbf{z}$ is uniformly sampled from $\mathbb{Z}_{\mathsf{p}}^{\ell} \setminus \mathsf{Span}(\mathbf{D}_2)$. Consequently, at least one of $\mathbf{z}_{\mathbf{A}}$ or $\mathbf{z}_{\sigma}$ does not belong to $\mathsf{Span}(\mathbf{D}_2)$. This implies the existence of a vector ξ, which the challenger can derive without affecting the adversary's view, such that $\xi^{\intercal}\mathbf{D}_2 = 0$, and either $\xi^{\intercal}\mathbf{z}_{\mathbf{A}} = 1$ or $\xi^{\intercal}\mathbf{z}_{\sigma} = 1$. Similarly, there exists a vector ξ_2 such that $\xi_2^{\intercal}\mathbf{y}_1 = 1$ and $\xi_2^{\intercal}\mathbf{U}_1 = 0$. We analyze these two cases separately:

- **Case $\mathbf{z}_{\mathbf{A}} \notin \mathsf{Span}(\mathbf{D}_2)$:** In this scenario, the first verification equation is given by:

$$\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}_{\mathbf{w}}\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[\mathbf{z}_{\mathbf{A}}^{\intercal}\right]_2 = \left[\mathbf{P}_{\mathbf{A}}\right]_1 \bullet \left[\mathbf{D}_2^{\intercal}\right]_2.$$

  Right-multiplying both sides by ξ yields:

$$\left[\mathbf{A}_i\right]_1 \bullet \left[\mathbf{C}_{\mathbf{w}} \cdot \xi\right]_2 - \left[\mathsf{x}\right]_1 \bullet \left[1\right]_2 = \left[\mathbf{P}_{\mathbf{A}}\right]_1 \bullet \left[0\right]_2.$$

  Therefore, in this case, we conclude that $\mathsf{x} \in \mathsf{Span}(\mathbf{A})$.
- **Case $\mathbf{z}_{\sigma} \notin \mathsf{Span}(\mathbf{D}_2)$:** In this scenario, we examine the last three verification equations:
  1. $(\left[\underline{\mathbf{A}}_i^{\intercal}\right]_1 - \left[(\underline{\mathbf{A}}^{\star})^{\intercal}\right]_1) \bullet \left[\mathbf{C}_{\mathbf{k}}\right]_2 - \left[\bar{\mathbf{A}}^{\intercal}\right]_1 \bullet \left[\mathbf{z}_{\sigma}^{\intercal}\right]_2 = \left[\mathbf{P}_{\mathbf{d}}\right]_1 \bullet \left[\mathbf{D}_2^{\intercal}\right]_2$,
  2. $(\left[\underline{\mathsf{x}}^{\intercal}\right]_1 - \left[(\underline{\mathsf{x}}^{\star})^{\intercal}\right]_1) \bullet \left[\mathbf{C}_{\mathbf{k}}\right]_2 - \left[\bar{\mathsf{x}}^{\intercal}\right]_1 \bullet \left[\mathbf{z}_{\sigma}^{\intercal}\right]_2 = \left[\mathbf{P}_{\mathsf{x}}\right]_1 \bullet \left[\mathbf{D}_2^{\intercal}\right]_2$,
  3. For $j \in [k+1]$:

$$\left[\mathbf{y}_1\mathbf{t}^{\intercal}\right]_1 \bullet \left[\mathbf{u}_j\mathbf{z}_{\sigma}^{\intercal}\right]_2 - \left[\mathbf{C}_{\mathbf{t}}\right]_1 \bullet \left[\mathbf{u}_j\mathbf{z}_{\sigma}^{\intercal}\right]_2 = \left[\mathbf{P}_{\mathbf{t}}^{j}\right]_1 \bullet \left[\mathbf{D}_2^{\intercal}\right]_2 + \left[\mathbf{U}_1\right]_1 \bullet \left[\mathbf{Q}_{\mathbf{t}}^{j}\right]_2,$$

  4. $\left[\mathbf{C}_{\mathbf{s}}\right]_1 \bullet \left[\mathbf{B}\right]_2 - \left[\mathbf{C}_{\mathbf{t}}\right]_1 \bullet \left[(\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)\mathbf{B}\right]_2 = \left[\mathbf{U}_1\right]_1 \bullet \left[\mathbf{Q}_{\sigma}\right]_2$.

  By right-multiplying ξ to all three equations and left-multiplying $\xi_2^{\intercal}$ to the second equation, we obtain:
  1. $(\left[\underline{\mathbf{A}}_i^{\intercal}\right]_1 - \left[(\underline{\mathbf{A}}^{\star})^{\intercal}\right]_1) \bullet \left[\mathbf{C}_{\mathbf{k}}\xi\right]_2 - \left[\bar{\mathbf{A}}^{\intercal}\right]_1 \bullet \left[1\right]_2 = \left[\mathbf{P}_{\mathbf{d}}\right]_1 \bullet \left[0\right]_2$,
  2. $(\left[\underline{\mathsf{x}}^{\intercal}\right]_1 - \left[(\underline{\mathsf{x}}^{\star})^{\intercal}\right]_1) \bullet \left[\mathbf{C}_{\mathbf{k}}\xi\right]_2 - \left[\bar{\mathsf{x}}^{\intercal}\right]_1 \bullet \left[1\right]_2 = \left[\mathbf{P}_{\mathsf{x}}\right]_1 \bullet \left[0\right]_2$,
  3. For $j \in [k+1]$:

$$\left[\mathbf{t}^{\intercal}\right]_1 \bullet \left[\mathbf{u}_j\right]_2 - \left[\xi_2^{\intercal}\mathbf{C}_{\mathbf{t}}\right]_1 \bullet \left[\mathbf{u}_j\right]_2 = \left[\mathbf{P}_{\mathbf{t}}^{j}\right]_1 \bullet \left[0\right]_2 + \left[0\right]_1 \bullet \left[\mathbf{Q}_{\mathbf{t}}^{j}\right]_2,$$

  4. $\left[\xi_2^{\intercal}\mathbf{C}_{\mathbf{s}}\right]_1 \bullet \left[\mathbf{B}\right]_2 - \left[\xi_2^{\intercal}\mathbf{C}_{\mathbf{t}}\right]_1 \bullet \left[(\mathbf{K}_0 + \mathsf{hv}\mathbf{K}_1)\mathbf{B}\right]_2 = \left[0\right]_1 \bullet \left[\mathbf{Q}_{\sigma}\right]_2$.

  From the above equations, the challenger can derive the vector $\left[\mathbf{C}_{\mathbf{k}}\xi\right]_2$, which represents a re-encryption token from $\left[\underline{\mathbf{A}}_i\right]_1$ to $\left[(\underline{\mathbf{A}}^{\star})^{\intercal}\right]_1$ and $\left[\underline{\mathsf{x}}_i\right]_1$ to $\left[(\underline{\mathsf{x}}^{\star})^{\intercal}\right]_1$. Furthermore, the second equation implies that $\left[\mathbf{C}_{\mathbf{t}}\right]_1$ is a commitment to $\mathbf{t}$, and $\left[\xi_2^{\intercal}\mathbf{C}_{\mathbf{s}}\right]_1$ and $\left[\xi_2^{\intercal}\mathbf{C}_{\mathbf{t}}\right]_1$ constitute a valid signature for $\mathsf{hv} = \mathsf{H}(\left[\underline{\mathbf{A}}^{\star}\right]_1, \left[\underline{\mathsf{x}}^{\star}\right]_1, \mathsf{t})$. Therefore, unless the adversary can break the unforgeability of the underlying signature scheme, the adversary must have derived an invalid re-encryption token.

□

G Security Proof for Single-Challenge aCCA-Secure PRE

Theorem 15. *If we instantiate the general construction of* $\mathsf{PRE}_{\mathsf{aCCA}}$ *Figure 6 with*

- SMP*: a hard subset membership language distribution*
- VSHPS*: a vector space hash proof system* VSHPS *with* (ε, d)*-universal, with* $d \geq 1$*.*
- OTS*: an one-time signature scheme* OTS *that is one-time strongly unforgeable.*
- tLM-NIZK*: an one-time special simulation-sound tag-based language-malleable* NIZK *proof system* tLM-NIZK *for the language corresponded to* SMP *and* VSHPS*.*

Then the resulting $\mathsf{PRE}_{\mathsf{aCCA}}$ *scheme is single-challenge* aCCA *secure against any* PPT *adversary* $\mathcal{A}$*, with advantage*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{SC\text{-}aHRA}}_{\mathsf{PRE},\mathcal{A}}(\lambda) \leq & \binom{\mathsf{N_U}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{StUnforg}}_{\mathsf{OTS},\mathcal{B}}(1^\lambda) \\ & + \mathsf{Adv}^{\mathsf{Collision}}_{\mathsf{H},\mathcal{B}}(1^\lambda) + 2 \cdot \mathsf{Adv}^{\mathsf{SSS}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{HSM}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) \\ & + \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).\end{aligned}$$

Proof. We prove the above theorem via the following hybrid security games $\mathbf{G}_0, \ldots, \mathbf{G}_{10}$. The detailed security games are presented in Figure 21.

Game $\mathbf{G}_0$: This is the original SC-aCCA security game. By definition, the adversary's success probability is

$$\mathsf{pr}_0 \coloneqq \Pr\left[\mathsf{Exp}^{\mathsf{SC\text{-}aCCA}}_{\mathsf{PRE},\mathcal{A}}(1^\lambda) = 1\right].$$

Game $\mathbf{G}_1$: This game is identical to $\mathbf{G}_0$, except that the challenger aborts if a collision occurs among the generated public keys. Let $\mathsf{Bad}_{\mathsf{Col}}$ be the event that there exist distinct $i, j \in [\mathsf{N_U}]$ such that $\mathsf{pk}_i = \mathsf{pk}_j$. If $\mathsf{Bad}_{\mathsf{Col}}$ occurs, the game aborts.

The difference in the adversary's success probability between these two games is bounded by the probability of the event $\mathsf{Bad}_{\mathsf{Col}}$. Using a standard union bound argument, we have:

$$|\mathsf{pr}_0 - \mathsf{pr}_1| \leq \Pr[\mathsf{Bad}_{\mathsf{Col}}] \leq \binom{\mathsf{N_U}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_2$: In this game, we modify the way of generating the public parameters. Instead of sampling the projection key projk uniformly from the set $\mathcal{PK}_{\mathsf{lpk}}$, we first sample the hash key hk using the hash proof system's setup algorithm, then compute the projection key as $\mathsf{projk} := \mathsf{PKGen}(\mathsf{hk}, \mathcal{L}_{\mathsf{lpk}})$. Since both methods of generating projk yield the same distribution, we have

$$\mathsf{pr}_2 = \mathsf{pr}_1.$$

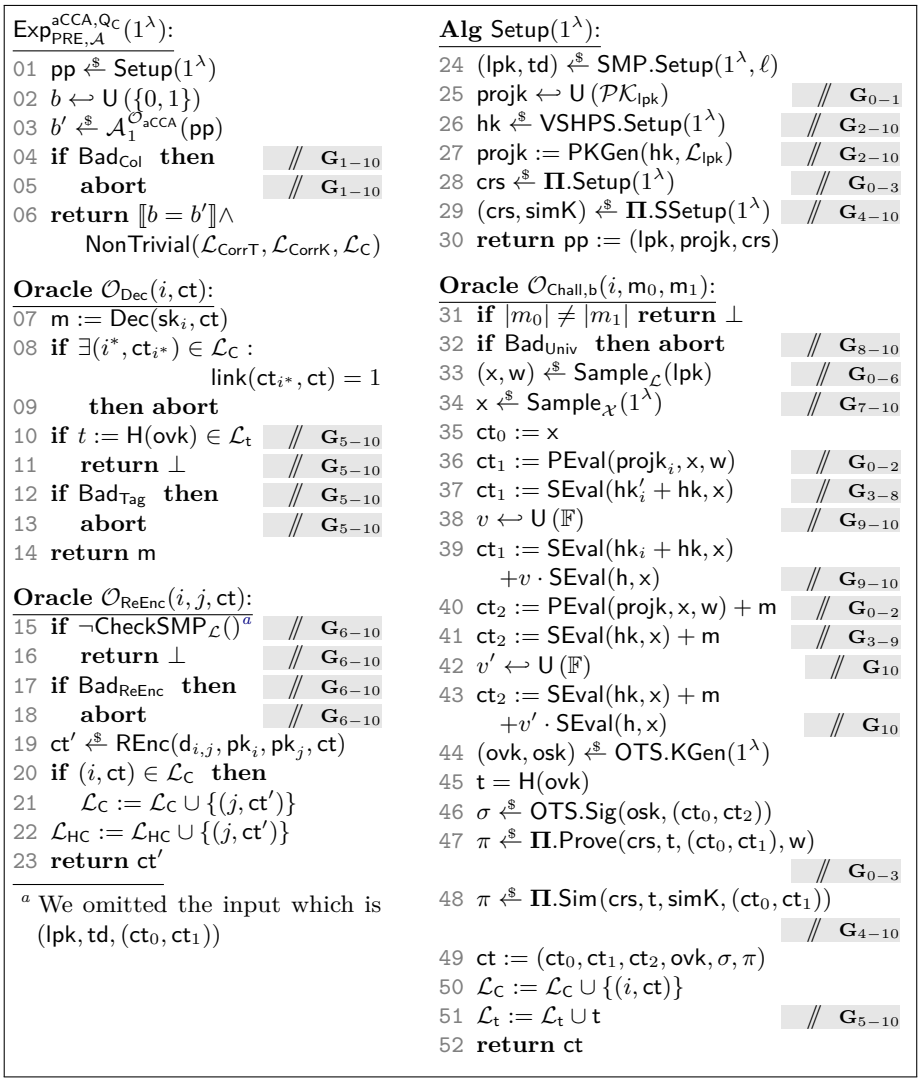

Fig. 21. Hybrid games for SC-aCCA security. We denote by $\mathsf{h} \in \mathcal{HK}$ the base vector for kernel space corresponds to the dimension k linear language with support in $\mathbb{F}^\ell$. Note that such vector can be generated together with the linear language.

Game $\mathbf{G}_3$: The only difference between $\mathbf{G}_3$ and $\mathbf{G}_2$ is that, the challenge ciphertext is encrypted using the hash key hk instead of using the public key projk. We generate the challenge ciphertext as

$$\mathsf{ct}_0 \xleftarrow{\$} \mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}) \quad \mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}'_i + \mathsf{hk}, \mathsf{ct}_0) \quad \mathsf{ct}_2 := \mathsf{SEval}(\mathsf{hk}, \mathsf{ct}_0) + \mathsf{m}_b$$

since $\mathsf{ct}_0 \in \mathcal{L}_{\mathsf{lpk}}$ with witness w, by the perfect correctness of HPS, the challenge ciphertext in $\mathbf{G}_3$ and $\mathbf{G}_2$ is identical. Therefore, we have

$$\mathsf{pr}_3 = \mathsf{pr}_2.$$

Game $\mathbf{G}_4$: In this game, we change the way of generating the zero-knowledge proof $\pi^\star$ in the challenge ciphertext. Instead of generating $\pi^\star$ using the witness w of ct_0, we generate $\pi^\star$ using the simulation trapdoor simK. By the zero-knowledge property of the proof system, we have

$$|\mathsf{pr}_4 - \mathsf{pr}_3| \leq \mathsf{Adv}^{\mathsf{ZK}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_5$: In this game, we introduce a list $\mathcal{L}_{\mathsf{t}}$ in $\mathcal{O}_{\mathsf{Chall}}$ to store the Tag of the challenge ciphertexts, then we reject any decryption query that uses the same tag as the challenge ciphertext. Let $\mathsf{Bad}_{\mathsf{Tag}}$ be the event that the adversary submits a decryption query with the challenge tag that would otherwise be accepted by the decryption oracle. If $\mathsf{Bad}_{\mathsf{Tag}}$ occurs, the game aborts. As in the general construction, this bad event occurs with probability at most $\mathsf{Adv}^{\mathsf{StUnforg}}_{\mathsf{OTS},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{Collision}}_{\mathsf{H},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{SSS}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda)$. Thus, we have

$$|\mathsf{pr}_5 - \mathsf{pr}_4| \leq \mathsf{Adv}^{\mathsf{StUnforg}}_{\mathsf{OTS},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{Collision}}_{\mathsf{H},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{SSS}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_6$: In this game, we reject any re-encryption query that ct_0 is not sampled from the $\mathcal{L}_{\mathsf{lpk}}$ with witness w or ct_1 is not computed as $\mathsf{ct}_1 := \mathsf{PEval}(\mathsf{projk}_i, \mathsf{ct}_0, \mathsf{w})$. Since the challenger knows the trapdoor td, it can always check the validity of the re-encryption queries. Let $\mathsf{Bad}_{\mathsf{ReEnc}}$ be the event that the adversary submits a re-encryption query that would otherwise be accepted by the re-encryption oracle. If $\mathsf{Bad}_{\mathsf{ReEnc}}$ occurs, the game aborts. By the special simulation-soundness of the proof system, we have

$$|\mathsf{pr}_6 - \mathsf{pr}_5| \leq \mathsf{Adv}^{\mathsf{SSS}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_7$: In this game, we change the way the challenge ciphertext is generated. Instead of sampling ct_0 from the language using $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk})$, we sample it from the statement set $\mathsf{ct}_0 \xleftarrow{\$} \mathsf{Sample}_{\mathcal{X}}(1^\lambda)$. Since the bad event $\mathsf{Bad}_{\mathsf{Col}}$ does not occur, the adversary cannot simply check if $\mathsf{ct}_0 \in \mathcal{L}_{\mathsf{lpk}}$ by checking if $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_j, \mathsf{ct}_0) = 0$ where i, j are any two collision users. [22] By the hard subset membership property of the language, we have

$$|\mathsf{pr}_7 - \mathsf{pr}_6| \leq \mathsf{Adv}^{\mathsf{HSM}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

[22] If there is a collision between two public keys pk_i and pk_j, then the adversary can simply gets $\mathsf{hk}_i - \mathsf{hk}_j \in \mathcal{HK}''$ by querying the oracle $\mathcal{O}_{\mathsf{CorrT}}(i, j)$, and check if $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_j, \mathsf{ct}_0) = 0$. If yes, then $\mathsf{ct}_0 \in \mathcal{L}_{\mathsf{lpk}}$, otherwise $\mathsf{ct}_0 \notin \mathcal{L}_{\mathsf{lpk}}$.

Game $\mathbf{G}_8$: In $\mathbf{G}_8$, we introduce a bad event $\mathsf{Bad}_{\mathsf{Univ}}$ that captures the failure of the universal hash property. Specifically, we define

$$\mathsf{Bad}_{\mathsf{Univ}} \coloneqq \{\forall \mathsf{hk}'' \in \mathcal{HK}'' : \mathsf{SEval}(\mathsf{hk}'', \mathsf{x}) = 0 \mid \mathsf{ct}_0 \xleftarrow{\$} \mathsf{U}(\mathcal{X})\}.$$

We recall that $\mathcal{HK}' \subset \mathcal{HK}$ is the affine vector space, such that we can change hk to any element $\mathsf{hk}' \in \mathcal{HK}'$ without changing its projection over $\mathcal{L}_{\mathsf{lpk}}$. $\mathcal{HK}''$ is the vector space that corresponds to the affine vector space $\mathcal{HK}'$. Using the universality of VSHPS, we have

$$|\mathsf{pr}_8 - \mathsf{pr}_7| \leq \Pr[\mathsf{Bad}_{\mathsf{Univ}}] \leq \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_9$: In this game, we modify the generation of the challenge ciphertext. We sample $v \hookleftarrow \mathsf{U}(\mathbb{F})$ at the beginning of the experiment and compute the challenge ciphertext as $\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}_i, \mathsf{ct}_0) + v \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) + \mathsf{SEval}(\mathsf{hk}, \mathsf{ct}_0)$ where h is a basis of the $\mathcal{HK}''$ over the linear language $\mathcal{L}_{\mathsf{lpk}}$.

Since the game does not abort, we are guaranteed that $\mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) \neq 0$. As v is chosen uniformly at random from $\mathbb{F}$, the term $v \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0)$ acts as a one-time pad, perfectly masking the ciphertext component. Thus, we can also modify ct_1 to be $\mathsf{ct}_1 \hookleftarrow \mathsf{U}(\mathbb{F})$ without changing the adversary's view.

We first give the value of $|\mathsf{pr}_9 - \mathsf{pr}_8|$, then we explain why the above modification is valid in Lemma 5. We have

$$|\mathsf{pr}_9 - \mathsf{pr}_8| = 0.$$

Game $\mathbf{G}_{10}$: In this game, we modify the generation of the challenge ciphertext. We sample $v' \hookleftarrow \mathsf{U}(\mathbb{F})$ at the beginning of the experiment and compute the challenge ciphertext as $\mathsf{ct}_2 := \mathsf{SEval}(\mathsf{hk}, \mathsf{ct}_0) + v' \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) + \mathsf{m}$.

The proof process is similar to that of $\mathbf{G}_9$, we first translate hk to $\mathsf{hk} + v' \cdot \mathsf{h}$, then delete all the $v' \cdot \mathsf{h}$ components from other oracle except $\mathcal{O}_{\mathsf{Chall}}$. Since hk is independent with re-encryption keys $\mathsf{d}_{i,j}$, we can do this without considering a selective adversary first, which means S2A lemma is not need. Above all, we have

$$|\mathsf{pr}_{10} - \mathsf{pr}_9| = 0.$$

Finally, we observe that in $\mathbf{G}_9$, the challenge ciphertext is computed as

$$\begin{aligned}
&\mathsf{ct}_0 \xleftarrow{\$} \mathsf{Sample}_{\mathcal{X}}(1^\lambda) \\
&\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}_1 + \mathsf{hk}, \mathsf{ct}_0) + v \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) \\
&\mathsf{ct}_2 := \mathsf{SEval}(\mathsf{hk}, \mathsf{ct}_0) + v' \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) + \mathsf{m}_b
\end{aligned}$$

Since v' is chosen uniformly at random from $\mathbb{F}$ and $\mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) \neq 0$, the term $v' \cdot \mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0)$ acts as a one-time pad, perfectly masking the message m_b. Therefore, the adversary's success probability is exactly $1/2$, and we have

$$\mathsf{pr}_9 = \frac{1}{2}.$$

Taking union bound over all the hybrid games, we have

$$\begin{aligned}\mathsf{Adv}^{\mathsf{SC\text{-}aHRA}}_{\mathsf{PRE},\mathcal{A}}(\lambda) \leq & \binom{\mathsf{N_U}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{StUnforg}}_{\mathsf{OTS},\mathcal{B}}(1^\lambda) \\ & + \mathsf{Adv}^{\mathsf{Collision}}_{\mathsf{H},\mathcal{B}}(1^\lambda) + 2 \cdot \mathsf{Adv}^{\mathsf{SSS}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{HSM}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) \\ & + \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).\end{aligned}$$

□

Lemma 5. *Games $\boldsymbol{G}_8$ and $\boldsymbol{G}_9$ are perfectly indistinguishable. That is, for any* PPT *adaptive adversary $\mathcal{A}$, we have*

$$\left|\Pr\left[\boldsymbol{G}_9^{\mathcal{A}} \Rightarrow 1\right] - \Pr\left[\boldsymbol{G}_8^{\mathcal{A}} \Rightarrow 1\right]\right| = 0,$$

Proof. We prove the lemma via a reduction from adaptive to selective security. The argument is structured in two parts, formalized in Lemma 6 and Lemma 3.

Lemma 6 establishes that no PPT selective adversary can distinguish between $\mathbf{G}_8$ and $\mathbf{G}_9$ with any advantage. Subsequently, by Lemma 3, we can extend the result to any PPT adaptive adversary $\mathcal{A}$ that no PPT adaptive adversary $\mathcal{A}$ can distinguish between $\mathbf{G}_8$ and $\mathbf{G}_9$ with any advantage which completes the proof. □

Lemma 6. *For any* PPT *selective adversary $\mathcal{B}$, which by definition needs to commit to all its oracle queries before the game begins, we have:*

$$\left|\Pr\left[\boldsymbol{G}_9^{\mathcal{B}} \Rightarrow 1\right] - \Pr\left[\boldsymbol{G}_8^{\mathcal{B}} \Rightarrow 1\right]\right| = 0.$$

Proof. We prove the lemma by introducing three intermediate games, $\boldsymbol{G}_{8.1}$, $\boldsymbol{G}_{8.2}$ and $\boldsymbol{G}_{8.3}$, and show that for any selective adversary $\mathcal{B}$, the following chain of perfect indistinguishability holds: $\boldsymbol{G}_8 \approx_p \boldsymbol{G}_{8.1} \approx_p \boldsymbol{G}_{8.2} \approx_p \boldsymbol{G}_{8.3} \approx_p \boldsymbol{G}_9$. This implies that $\boldsymbol{G}_8$ and $\boldsymbol{G}_9$ are perfectly indistinguishable for $\mathcal{B}$.

The simulation strategy for these games, detailed in Figure 22, leverages the selective nature of $\mathcal{B}$. Since all oracle queries are known beforehand, the challenger can simulate the environment without knowing most secret keys corresponding to challenge queries. Without loss of generality, we re-index users such that all challenge queries in $\mathcal{L}_{\mathsf{C}}$ are for users $i \in \{1, \ldots, |\mathsf{V_{CL}}|\}$.

Game $\boldsymbol{G}_{8.1}$: *This game, detailed in Figure 22, is identical to $\boldsymbol{G}_8$ but uses a different simulation strategy. The challenger first generates a key pair $(\mathsf{hk}_1, \mathsf{projk}_1)$ and for each user $i \in \{2, \ldots, |\mathsf{V_{CL}}|\}$, it generates $\mathsf{projk}_i := \mathsf{projk}_1 + \mathsf{PKGen}(\mathsf{d}_{1,i}, \mathcal{L}_{\mathsf{lpk}})$ via the re-encryption key $\mathsf{d}_{1,i}$. Keys for non-challenge users $i > |\mathsf{V_{CL}}|$ are generated independently as in the original game. The challenger can answer all oracle queries using the known secret keys of non-challenge users and the re-encryption keys for challenge users.*

The challenge ciphertext is generated using hk_1 as in $\boldsymbol{G}_8$. Since the distribution of keys and ciphertexts is identical to $\boldsymbol{G}_8$, we have

$$|\mathsf{pr}_{8.1} - \mathsf{pr}_8| = 0.$$

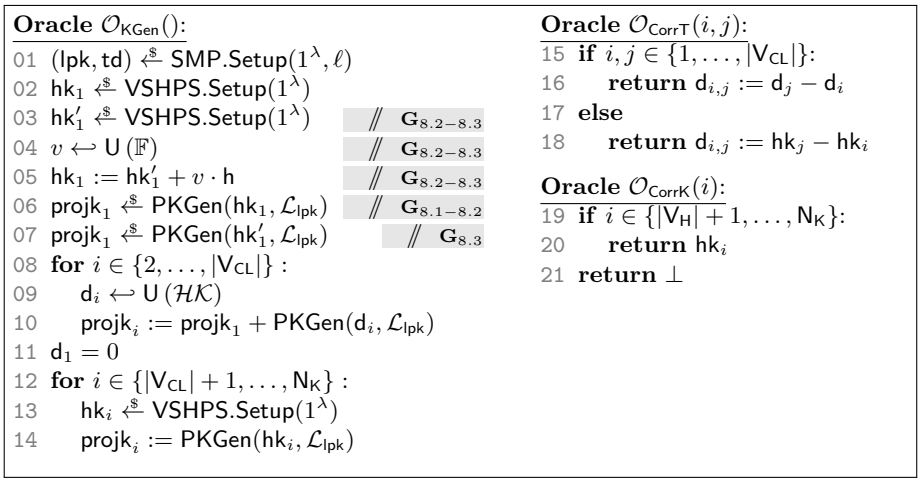

Fig. 22. Description of the hybrid games $\mathbf{G}_{8.1}$, $\mathbf{G}_{8.2}$ and $\mathbf{G}_{8.3}$ in a selective adversary. The figure first show the difference between $\mathbf{G}_{8.1}$ under selective adversary and $\mathbf{G}_8$ under adaptive adversary. Then it illustrates the changes in the key generation process from $\mathbf{G}_{8.2}$ and $\mathbf{G}_{8.3}$. Without loss of generality, we re-index keys such that challenge queries in $\mathcal{L}_{\mathsf{C}}$ correspond to the first $|\mathsf{V_{CL}}|$ users. The key generation oracle $\mathcal{O}_{\mathsf{KGen}}$ is executed only once at the start.

Game $G_{8.2}$: *In this game, we modifies the generation of* hk_1. *Let* $\mathsf{h} \in \mathcal{HK}''$ *be a hash key such that* $\mathsf{SEval}(\mathbf{h}, \mathsf{ct}_0) \neq 0$ *for the challenge statement* ct_0. *Such an* h *is guaranteed to exist by the* $(\varepsilon, 1)$*-universality of the* VSHPS. *The challenger now computes* $\mathsf{hk}_1 := \mathsf{hk}'_1 + v \cdot \mathsf{h}$, *where* $\mathsf{hk}'_1 \xleftarrow{\$} \mathcal{HK}$ *and* $v \hookleftarrow \mathsf{U}(\mathbb{F})$. *Since* $\mathcal{HK}$ *is a vector space over* $\mathbb{F}$, *the distribution of* hk_1 *remains uniform over* $\mathcal{HK}$. *Furthermore, by the definition of* $\mathcal{HK}''$, *we have* $\mathsf{PKGen}(\mathsf{hk}_1, \mathcal{L}_{\mathsf{lpk}}) = \mathsf{PKGen}(\mathsf{hk}'_1, \mathcal{L}_{\mathsf{lpk}})$. *Thus, the adversary's view is identical to that in* $\boldsymbol{G}_{8.1}$, *and we have*

$$|\mathsf{pr}_{8.2} - \mathsf{pr}_{8.1}| = 0.$$

Game $G_{8.3}$: *This game modifies the generation of the projection key* projk_1 *by using the hash key* hk'_1 *instead of* hk_1. *Moreover, we answer all the oracle queries except* $\mathcal{O}_{\mathsf{Chall}}$ *without knowing* v, *which is only used in the challenge ciphertext generation.*

By the definition of $\mathcal{HK}''$, *we have* $\mathsf{PKGen}(\mathsf{hk}_1, \mathcal{L}_{\mathsf{lpk}}) = \mathsf{PKGen}(\mathsf{hk}'_1, \mathcal{L}_{\mathsf{lpk}})$. *Thus, the public key* projk_1 *remains unchanged. Since* $\mathcal{O}_{\mathsf{CorrK}}$ *is not queried for users* $i \in [|\mathsf{V_{CL}}|]$, $\mathcal{O}_{\mathsf{CorrT}}$ *queries for other challenged users* $i \in \{2, \cdots, |\mathsf{V_{CL}}|\}$ *are answered using* d_i, *and for non-challenged users* $j > |\mathsf{V_{CL}}|$, $\mathcal{O}_{\mathsf{CorrT}}$ *queries are answered using their known secret keys, the adversary gets no information about* hk_1 *or* hk'_1 *from* $\mathcal{O}_{\mathsf{CorrK}}$ *and* $\mathcal{O}_{\mathsf{CorrT}}$. *As for* $\mathcal{O}_{\mathsf{REnc}}$ *and* $\mathcal{O}_{\mathsf{Dec}}$, *by the changes in* $\boldsymbol{G}_5$ *and* $\boldsymbol{G}_6$, *the ciphertexts components* ct_0 *and* ct_1 *in the re-encryption queries and decryption queries are guaranteed to be in the language* $\mathcal{L}_{\mathsf{lpk}}$ *with some witness.*

Therefore, all the information the adversary can learn from these oracles is through the evaluation of $\mathsf{SEval}(\mathsf{hk}_i - \mathsf{hk}_1, \mathsf{ct}_0)$ *or* $\mathsf{SEval}(\mathsf{hk}_1, \mathsf{ct}_0)$ *for some users* i*. Since* $\mathsf{SEval}(\mathsf{h}, \mathsf{ct}_0) = 0$*, We have* $\mathsf{SEval}(\mathsf{hk}_1, \mathsf{ct}_0) = \mathsf{SEval}(\mathsf{hk}'_1, \mathsf{ct}_0) + v \cdot \mathsf{SEval}(\mathsf{hk}, \mathsf{ct}_0) = \mathsf{SEval}(\mathsf{hk}'_1, \mathsf{ct}_0)$*, the answer remains unchanged. Therefore, the adversary's view in* $\boldsymbol{G}_{8.2}$ *and* $\boldsymbol{G}_{8.3}$ *is identical, leading to*

$$\mathsf{pr}_{8.3} = \mathsf{pr}_{8.2}.$$

In $\boldsymbol{G}_{8.3}$*, the challenge ciphertext component* ct_1 *is computed as:*

$$\mathsf{ct}_1 := \mathsf{SEval}(\mathsf{hk}'_1 + v \cdot \mathsf{h}, \mathsf{ct}_0)$$

This is identical to the challenge ciphertext generation in $\boldsymbol{G}_9$*. The only difference lies in the generation of secret keys for users* $i \in \{2, \cdots, |\mathsf{V_{CL}}|\}$*. In* $\boldsymbol{G}_{8.3}$*, these keys are implicitly defined as* $\mathsf{hk}_i = \mathsf{hk}'_1 + \mathsf{d}_i$*, while in* $\boldsymbol{G}_9$*, they are sampled uniformly from* $\mathcal{HK}$*. However, since* d_i *is uniformly random in* $\mathcal{HK}$ *and independent of* hk'_1*, the distribution of* hk'_i *remains uniform over* $\mathcal{HK}$*. Thus, the distribution of all secret keys in both games is identical. Therefore, we have*

$$\mathsf{pr}_9 = \mathsf{pr}_{8.3}.$$

Combining the above results, we conclude that

$$\left|\Pr\left[\boldsymbol{G}_9^{\mathcal{B}} \Rightarrow 1\right] - \Pr\left[\boldsymbol{G}_8^{\mathcal{B}} \Rightarrow 1\right]\right| = 0.$$

□

H Security Proof for Multi-Challenge aCCA-Secure PRE

Theorem 16. *If we instantiate the general construction of* $\mathsf{PRE}_{\mathsf{aCCA}}$ *Figure 6 with*

- SMP*: a hard subset membership language distribution*
- VSHPS*: a vector space hash proof system* VSHPS *with* (ε, d)*-universal, with* $d \geq k$*.*
- OTS*: an one-time signature scheme* OTS *that is one-time strongly unforgeable.*
- tLM-NIZK*: an one-time special simulation-sound tag-based language-malleable* NIZK *proof system* tLM-NIZK *for the language corresponded to* SMP *and* VSHPS*.*

Then the resulting $\mathsf{PRE}_{\mathsf{aCCA}}$ *scheme is multi-challenge* aCCA *secure against any* PPT *adversary* $\mathcal{A}$*, with advantage*

$$\begin{aligned}
\mathcal{A}^{\mathsf{aCCA},\mathsf{Q_C}}_{\mathsf{PRE}_2,\mathcal{A}}(\lambda) = |\mathsf{pr}_0 - \mathsf{pr}_{14}| &\leq \sum_{i=0}^{13} \left|\mathsf{pr}_i - \mathsf{pr}_{i+1}\right| \\
&\leq \binom{\mathsf{N_U}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{StUnforg}}_{\mathsf{OTS},\mathcal{B}}(1^\lambda) \\
&+ \mathsf{Adv}^{\mathsf{Collision}}_{\mathsf{H},\mathcal{B}}(1^\lambda) + 2 \cdot \mathsf{Adv}^{\mathsf{SSS}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + 3 \cdot \mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) \\
&+ \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).
\end{aligned}$$

Proof. The proof proceeds via a sequence of hybrid games, $\mathbf{G}_0, \ldots, \mathbf{G}_{14}$. The transitions from $\mathbf{G}_0$ to $\mathbf{G}_{10}$ mirror those in the proof of Theorem 15, but are adapted for the multi-challenge setting. The key differences are as follows:

- The transition from $\mathbf{G}_6$ to $\mathbf{G}_7$ relies on the $\mathsf{Q_C}$-Hard Subset Membership ($\mathsf{Q_C}$-HSM) property. All $\mathsf{Q_C}$ challenge statements are switched from being sampled in the language via $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk})$ to being sampled uniformly from the ambient space via $\mathsf{Sample}_{\mathcal{X}}(1^\lambda)$. This introduces a difference of at most $\mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda)$.
- In $\mathbf{G}_8$, the bad event $\mathsf{Bad}_{\mathsf{Univ}}$ is defined with respect to the (ε, d)-universality of the VSHPS. The difference is bounded by $|\mathsf{pr}_8 - \mathsf{pr}_7| \leq \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda)$.
- In $\mathbf{G}_9$ and $\mathbf{G}_{10}$, the challenge ciphertext is randomized. Let $\mathbf{H} \in \mathbb{F}^{d\times\ell}$ be a basis of the $\mathcal{HK}''$ over linear language $\mathcal{L}_{\mathsf{lpk}}$. We sample random vectors $\mathbf{v}, \mathbf{v}' \in \mathbb{F}^d$ and compute the challenge ciphertext as $\mathsf{ct}_1^j := \mathsf{SEval}(\mathsf{hk}_i + \mathbf{v}^\intercal\mathbf{H} + \mathsf{hk}, \mathsf{ct}_0^j)$ and $\mathsf{ct}_2^j := \mathsf{SEval}(\mathsf{hk} + \mathbf{v}'\mathbf{H}, \mathsf{ct}_0^j) + \mathsf{m}_b^j$ for $j \in [\mathsf{Q_C}]$ and $i \in [\mathsf{N_U}]$ indexing the challenge users. The proof of this step is similar to the single-challenge case, but requires a more detailed explanation.
  - In $\mathbf{G}_{8.1}$, we index the first challenge user as 1 and generate its secret key hk_1 normally. We add a relative token d_i for each other challenge user i [23], so we can still simulate all the oracle queries perfectly without knowing any other user's secret key in list $\mathsf{V_{CL}}$.
  - In $\mathbf{G}_{8.2}$ and $\mathbf{G}_{8.3}$, we do the same modification as in the single-challenge case, except that we now sample a random vector $\mathbf{v} \in \mathbb{F}^d$ and compute $\mathsf{hk}_1 := \mathsf{hk}'_1 + \mathbf{v}^\intercal \cdot \mathbf{H}$, where $\mathsf{hk}'_1 \xleftarrow{\$} \mathcal{HK}$. We note that since all the challenge users share the same hash key hk_1, the random vector $\mathbf{v}^\intercal\mathbf{H}$ is also shared among all challenge ciphertexts.
  - In $\mathbf{G}_{10}$, since all the challenge ciphertexts share the same hash key hk, the random vector $\mathbf{v}'\mathbf{H}$ is also shared among all challenge ciphertexts.

  As in the single-challenge case, this change is purely conceptual, so we have $\mathsf{pr}_9 = \mathsf{pr}_8$ and $\mathsf{pr}_{10} = \mathsf{pr}_9$.

The subsequent games, $\mathbf{G}_{11}$ through $\mathbf{G}_{14}$, are detailed in Figure 9 and complete the reduction.

Game $\mathbf{G}_{11}$: In this game, we change the distribution of the statement component ct_0 for all challenge ciphertexts. Instead of sampling ct_0 uniformly from the ambient space via $\mathsf{Sample}_{\mathcal{X}}(1^\lambda)$, we sample it from the language $\mathcal{L}_{\mathsf{lpk}_1}$ via $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}_1)$, where $\mathsf{lpk}_1 \xleftarrow{\$} \mathsf{SMP.Setup}(1^\lambda, \ell)$ is new language public key. The difference between $\mathbf{G}_{10}$ and $\mathbf{G}_{11}$ is bounded by the $\mathsf{Q_C}$-HSM property of the language.

$$|\mathsf{pr}_{11} - \mathsf{pr}_{10}| \leq \mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

Game $\mathbf{G}_{12}$: This game is a conceptual rewrite of $\mathbf{G}_{11}$. Since ct_2 is similar to ct_1, we only explain the change for ct_1.

[23] The adversary may not queries a $\mathcal{O}_{\mathsf{CorrT}}$ between two challenge users which make the graph of $\mathsf{V_{CL}}$ is not connected

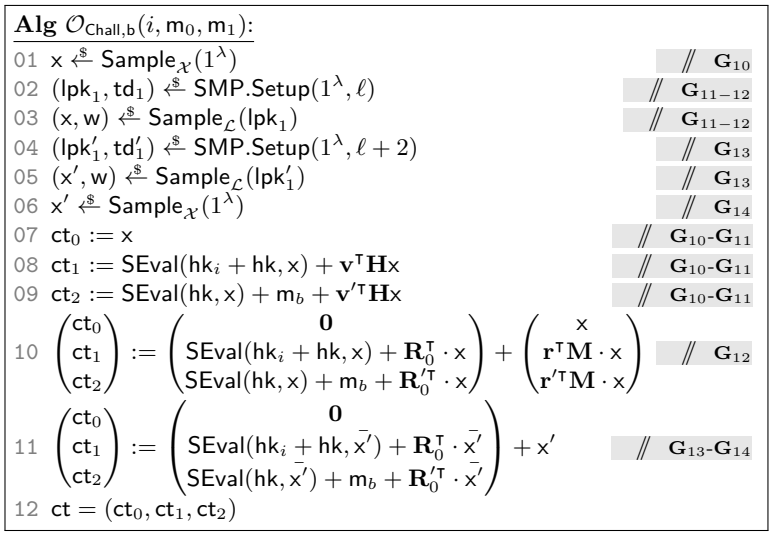

Fig. 23. Hybrid games for aCCA security from $\mathbf{G}_9$ onwards.

We first express the challenge ciphertext generation in matrix form, which does not change the adversary's view. Then we rewrite the term $\mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H},\mathsf{x})$ as $\mathbf{R}^\intercal\cdot\mathsf{x}$ where $\mathbf{R}^\intercal := \mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H},\mathbf{u}_1)|\cdots|\mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H},\mathbf{u}_\ell)$ is a row vector in $\mathbb{F}^{1\times\ell}$ and $\mathbf{u}_1,\ldots,\mathbf{u}_\ell$ is the standard basis of $\mathbb{F}^\ell$. Since SEval is linear in its second argument, we have $\mathsf{SEval}(\mathbf{v}^\intercal\mathbf{H},\mathsf{x}) = \mathbf{R}^\intercal\cdot\mathsf{x}$. Finally, we modify the vector $\mathbf{R}$ to be uniformly random in a subspace of dimension d of $\mathbb{F}^{\ell\times 1}$, where d is the dimension of the hash key space $\mathcal{HK}''$. This change is valid because $\mathbf{v}\in\mathbb{F}^d$ is uniformly random, $\mathbf{H}\in\mathbb{F}^{d\times\ell}$ has full row rank d, and the vectors $\mathbf{u}_1,\ldots,\mathbf{u}_\ell$ are linearly independent. By the d-MinEntropy property of VSHPS, $\mathbf{R}$ has full min-entropy $d\cdot\log|\mathbb{F}|$. Thus we can rewrite $\mathbf{R}$ as $\mathbf{r}^\intercal\mathbf{M}+\mathbf{R}_0$ for some fixed $\mathbf{R}_0\in\mathbb{F}^{\ell\times 1}$, where $\mathbf{r}\xleftarrow{\$}\mathsf{U}(\mathbb{F}^d)$ and $\mathbf{M}\in\mathbb{F}^{d\times\ell}$ is a full row rank matrix, the change is purely conceptual.

Thus, the games are perfectly indistinguishable, we have

$$|\mathsf{pr}_{12}-\mathsf{pr}_{11}| = 0$$

Game $\mathbf{G}_{13}$: This game reinterprets the structure of the challenge ciphertext. In $\mathbf{G}_{11}$, the vector added to the core encryption is $\begin{pmatrix}\mathsf{x}\\ \mathbf{r}^\intercal\mathbf{M}\cdot\mathsf{x}\\ \mathbf{r}'^\intercal\mathbf{M}\cdot\mathsf{x}\end{pmatrix} = \begin{pmatrix}\mathbf{I}_\ell\\ \mathbf{r}^\intercal\mathbf{M}\\ \mathbf{r}'^\intercal\mathbf{M}\end{pmatrix}\cdot\mathsf{x}$. Since $\mathbf{I}_\ell$ is the $\ell\times\ell$ identity matrix, $\mathbf{r}$ and $\mathbf{r}'$ are random vectors in $\mathbb{F}^d$ with $d\geq k = dim(\mathcal{L}_{\mathsf{lpk}_1})$ and $\mathbf{M}\in\mathbb{F}^{d\times\ell}$ is a full row rank matrix, the matrix $\begin{pmatrix}\mathbf{I}_\ell\\ \mathbf{r}^\intercal\mathbf{M}\\ \mathbf{r}'^\intercal\mathbf{M}\end{pmatrix}$ is a isomorphism from $\mathcal{L}_{\mathsf{lpk}_1}$ to a new linear space of dimension k over an ambient

space of dimension $\ell+2$. We denote this new linear space as $\mathcal{L}_{\mathsf{lpk}'_1}$, and rewrite the challenge ciphertexts as $\begin{pmatrix}\mathsf{ct}_0\\ \mathsf{ct}_1\\ \mathsf{ct}_2\end{pmatrix} := \begin{pmatrix}\mathbf{0}\\ \mathsf{SEval}(\mathsf{hk}_i, \bar{\mathsf{x}'}) + \mathbf{R}_0^{\intercal}\cdot\bar{\mathsf{x}'}\\ \mathsf{SEval}(\mathsf{hk}, \bar{\mathsf{x}'}) + \mathsf{m}_b + \mathbf{R}_0'^{\intercal}\cdot\bar{\mathsf{x}'}\end{pmatrix} + \mathsf{x}'$, where $(\mathsf{x}', \mathsf{w}) \xleftarrow{\$} \mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}'_1)$ and $\bar{\mathsf{x}'}$ is the first ℓ components of x'.

This change is purely conceptual, so the games are perfectly indistinguishable, we have

$$\mathsf{pr}_{13} = \mathsf{pr}_{12}.$$

Game $\mathbf{G}_{14}$: In this final game, we change how x' is generated for all challenge ciphertexts. Instead of sampling x' from the language $\mathcal{L}_{\mathsf{lpk}'_1}$ via $\mathsf{Sample}_{\mathcal{L}}(\mathsf{lpk}'_1)$, we sample it uniformly at random from the ambient space $\mathbb{F}^{\ell+2}$ via $\mathsf{Sample}_{\mathcal{X}}(1^\lambda)$. The indistinguishability between $\mathbf{G}_{13}$ and $\mathbf{G}_{14}$ is guaranteed by the $\mathsf{Q_C}$-HSM property of the language $\mathcal{L}_{\mathsf{lpk}'_1}$.

$$|\mathsf{pr}_{14} - \mathsf{pr}_{13}| \le \mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda).$$

In $\mathbf{G}_{14}$, by the above game transitions, the challenge ciphertexts is computed as $\begin{pmatrix}\mathsf{ct}_0\\ \mathsf{ct}_1\\ \mathsf{ct}_2\end{pmatrix} := \begin{pmatrix}\mathbf{0}\\ \mathsf{SEval}(\mathsf{hk}_i + \mathsf{hk}, \bar{\mathsf{x}'}) + \mathbf{R}_0^{\intercal}\cdot\bar{\mathsf{x}'}\\ \mathsf{SEval}(\mathsf{hk}, \bar{\mathsf{x}'}) + \mathsf{m}_b + \mathbf{R}_0'^{\intercal}\cdot\bar{\mathsf{x}'}\end{pmatrix} + \mathsf{x}'$, where x' is a uniformly random vector in $\mathbb{F}^{\ell+2}$. This vector acts as a one-time pad, making the resulting ciphertext $(\mathsf{ct}_0, \mathsf{ct}_1, \mathsf{ct}_2)$ uniformly random and statistically independent of the challenge bit b. Consequently, the adversary's advantage is zero.

$$\mathsf{pr}_{14} = \frac{1}{2}.$$

Combining the bounds from all game transitions yields the final security advantage:

$$\begin{aligned}\mathcal{A}^{\mathsf{aCCA},\mathsf{Q_C}}_{\mathsf{PRE}_2,\mathcal{A}}(\lambda) = |\mathsf{pr}_0 - \mathsf{pr}_{14}| &\le \sum_{i=0}^{13} \left|\mathsf{pr}_i - \mathsf{pr}_{i+1}\right| \\ &\le \binom{\mathsf{N_U}}{2} \cdot \mathsf{Adv}^{\mathsf{KCol}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + \mathsf{Adv}^{\mathsf{StUnforg}}_{\mathsf{OTS},\mathcal{B}}(1^\lambda) \\ &\quad + \mathsf{Adv}^{\mathsf{Collision}}_{\mathsf{H},\mathcal{B}}(1^\lambda) + 2\cdot\mathsf{Adv}^{\mathsf{SSS}}_{\mathsf{tLM\text{-}NIZK},\mathcal{B}}(1^\lambda) + 3\cdot\mathsf{Adv}^{\mathsf{HSM},\mathsf{Q_C}}_{\mathsf{SMP},\mathcal{B}}(1^\lambda) \\ &\quad + \mathsf{Adv}^{\mathsf{BadUniv}}_{\mathsf{VSHPS},\mathcal{B}}(1^\lambda).\end{aligned}$$

□